

RISC-V Atomic Cryptography Extension (ZL)

Roberto Avanzi, Ruud Derwig, Luis Fiolhais, Radim Krčmář

Table of Contents

List of tables	5
List of listings	5
Acronyms	5
Copyright and license information	8
Contributors	8
Introduction	10
Features	10
Non-Goals	11
TODOs / Open Discussion Points	12
1. Book 1: Unprivileged Architecture, Instructions and Architectural States	13
1.1. Introduction	13
1.1.1. Fundamental Concepts and Definitions	13
1.1.2. Architectural Model	14
1.2. Extensions Overview	15
1.3. Programmer-Visible Architectural State	19
1.3.1. Context Registers	19
1.3.1.1. Representation of Cryptographic Registers for Accessing their Contents	20
1.3.1.2. The Context Register File	21
1.3.2. Providing Inputs and Outputs	21
1.3.2.1. RVV/RVV-mini	21
1.3.2.2. ACEIOBUF	22
1.3.3. Metadata Formats	23
1.3.3.1. <i>SystemFormat</i> Bit	25
1.3.3.2. <i>Algorithm</i> and <i>AlgorithmPolicy</i> Fields	25
1.3.3.3. <i>SCProtection</i> Field	25
1.3.3.4. <i>KeyType</i> Field	25
1.3.3.5. <i>State</i> and <i>StateExtension</i> Fields	26
1.3.3.6. <i>ConfigStatus</i> Field	30
1.3.3.7. <i>ImpDataLen</i> Field	30
1.3.3.8. <i>Locality</i> Field	30
1.3.3.9. <i>ExpirationDate</i> Field	33
1.3.3.10. <i>AlgorithmUse</i> Field	33
1.3.4. Memory Alignment Constraints	33
1.3.5. Memory Model	33
1.3.6. CSRs	34
1.3.6.1. <i>acevendorid</i> , <i>acearchid</i> , and <i>aceimpid</i>	34

1.3.6.2. <code>acestart</code>	35
1.3.6.3. <code>acemaxiobufen</code>	36
1.3.6.4. <code>aceiobufen</code>	36
1.3.6.5. <code>aceiobufen</code>	36
1.3.6.6. <code>acenonce0</code> and <code>acenonce1</code>	37
1.4. Instructions	38
1.4.1. Provisioning, Import, and Export	38
1.4.2. <code>ace.load</code>	39
1.4.3. <code>ace.store</code>	40
1.4.4. <code>ace.exec</code>	41
1.4.5. <code>ace.mv</code>	42
1.4.6. <code>ace.setst</code>	44
1.4.7. <code>ace.mgmt</code>	46
1.4.8. <code>ace.getmdv/ace.getmdl/ace.getmdh</code>	50
1.4.9. <code>ace.restrictv/ace.restrictl/ace.restricth</code>	51
1.4.10. <code>ace.clone</code>	54
1.4.11. <code>ace.derive</code>	55
1.4.12. <code>ace.size</code>	56
1.4.13. <code>ace.input</code>	57
1.4.14. <code>ace.output</code>	58
1.5. Pseudo-Instructions	59
1.5.1. <code>ace.clear/ace.reset</code>	59
1.5.2. <code>ace.getst</code>	59
1.5.3. <code>ace.avail</code>	60
1.6. Using <code>ace.input</code> and <code>ace.output</code> to Replace Vector Inputs and Outputs	60
1.7. Interruptibility and Resumability of ACE instructions	62
1.7.1. Management Operations	62
1.7.2. Resumption and Memory Model	63
1.8. Code for CR Configuration and Export Operations	64
1.9. External Cryptographic Context Formats	68
1.9.1. Formats of Provisioning Input and Sealed Cryptographic Context	68
1.9.2. Single-Share Key Export Rule for Threshold implementations	70
1.9.3. System Keys and Randomly Generated Key Values	70
1.9.4. Notation and Representation of CR Internals	71
1.10. Management of Sealed Cryptographic Contexts	71
1.10.1. AES-GCM-SIV Key Derivation and POLYVAL	72
1.10.2. Generation of an SCC from a Fully Configured CR	73
1.10.3. Decryption and Authentication of a complete SCC into a CR	74
1.10.4. Import and Data-Independent Execution Latency	75
2. Book 2: Algorithms	75
2.1. Definition of an Algorithm in ACE	78
2.2. Generic Rules	79
2.2.1. Notation in the Algorithm Descriptions	79
2.2.2. Representation of System Keys in the Data Structures	80
2.3. Padding	80
2.4. States for the Symmetric Algorithms Defined in this Specification	80
2.4.1. Electronic Codebook Modes (Direct Encryption/Decryption)	82

2.4.2. Chaining/Feedback Modes	83
2.4.3. CTR/LFSR/XCTR/XLFSR Keystream Generation	83
2.4.4. XEX/XTS modes (one or two keys)	85
2.4.5. Generic Tweakable Block Ciphers	87
2.4.6. GCM Mode	88
2.4.7. GCM with Set IV	92
2.4.8. GCM-SIV Mode	93
2.4.9. OCB Mode	97
2.4.10. CMAC Functions	102
2.4.11. Hash and MAC Functions, and XOFS	104
2.4.12. HMAC	108
2.4.13. SHA-3 Family Specific Rules	108
2.4.14. Ascon-AEAD128	110
2.4.15. Ascon-AEAD128 with set Nonce	114
2.4.16. Ascon-AEAD128 with Nonce Masking	114
2.4.17. Ascon-Hash256	115
2.4.18. Ascon-XOF128	117
2.4.19. Ascon-CXOF128	117
2.5. Elliptic Curve Asymmetric Algorithms	117
2.6. Post Quantum Public-Key Cryptography	123
2.6.1. ML-KEM	123
2.6.2. ML-DSA	126
3. Book 3: Privileged Architecture	130
3.1. Extensions Overview	130
3.2. Dependencies on Other Extensions	131
3.3. Exceptions Raised by ACE	131
3.4. CSRs	132
3.4.1. ACE field in <code>misa</code>	133
3.4.2. ACE Context Status in <code>mstatus</code> and <code>sstatus</code>	133
3.4.3. ACE Context Status in <code>vsstatus</code>	134
3.4.4. <code>scrstatus</code> for Dirtiness Tracking	135
3.4.5. <code>vscrstatus</code> for Dirtiness Tracking	136
3.4.6. <code>mcrstatus</code> for Dirtiness Tracking	136
3.4.7. Use of <code>*crstatus</code> 's "Other" Value for Lazy Context Switching and CR Loading	136
3.4.8. Propagation of <code>*crstatus</code> Status Values	137
3.4.9. <code>macecsk</code> Group	137
3.4.10. <code>macePhysBootScrt</code> Group	138
3.4.11. <code>haceVirtBootScrt</code> Group	138
3.4.12. <code>maceLocality</code> Group	138
3.4.13. <code>haceLocality</code> Group	138
3.4.14. <code>saceLocality</code> Group	139
4. Book 4: Pseudocode Examples	139
4.1. Electronic Codebook Modes (Direct Encryption/Decryption)	139
4.2. CTR/LFSR/XCTR/XLFSR Keystream Generation	140
4.3. XEX/XTS modes (one or two keys)	140
4.4. GCM Mode	141

4.5. GCM-SIV Mode	142
4.6. OCB Mode	144
4.7. CMAC Functions	145
Index	146
Bibliography	146

List of tables

list-of::table[hide_empty_section=true, enhanced_rendering=true]

List of listings

list-of::listing[hide_empty_section=true, enhanced_rendering=true]

Acronyms

Acronym	Description
ACE	Atomic Cryptography Extension
ACEIOBUF	ACE Input/Output Buffer
ADS	Additional Data Section
AE	Authenticated Encryption
AEAD	Authenticated Encryption with Associated Data
AES	Advanced Encryption Standard
ASID	Application Space Identifier
CBC	Cipher Block Chaining (mode of encryption)
CC	Cryptographic Context
CFB	Cipher Feedback (mode of encryption)
CMAC	Cipher-based Message Authentication Code
CR	Context Register
CPU	Central Processing Unit
CRF	Context Register File
CSK	Context Sealing Key
CSR	Control and Status Register
CTR	Counter (mode of encryption)
DIEL	Data-Independent Execution Latency
ECB	Electronic Codebook (mode of encryption)
EEW	Effective Element Width (Vector Crypto Extension)
EGW	Element Group Width (Vector Crypto Extension)
EPC	Exception Program Counter
ESG	Element Group Size (Vector Crypto Extension)
FIB	Focused Ion Beam
FP	Floating-Point
GCM	Galois Counter Mode
GPA	Guest Physical Address
GPR	General-Purpose Register
GVA	Guest Virtual Address

Acronym	Description
hart	Hardware Thread
HV	Hypervisor
ISE	Instruction Set Extension
IV	Initialization Vector
impdef	Implementation-defined
HLOS	High-Level Operating System
HV	Hypervisor
KDF	Key Derivation Function
KEM	Key Encapsulation Mechanism
KMA	Key Management Applet
LF	Locker File
LFSR	Linear Feedback Shift Register (also: LFSR-based mode of operation)
LS	Locality Secrets
LST	Locality Secrets Table
MAC	Message Authentication Code
MDH	Metadata Header
MM	Machine Mode
NOP	No Operation
OFB	Output Feedback (mode of encryption)
OS	Operating System
PA	Physical Address
PCBC	Propagating Cipher Block Chaining (mode of encryption)
PI	Provisioning Input
PRF	Pseudo-Random Function
PRP	Pseudo-Random Permutation
PRW	Privileged Read-Write
RUP	Release of Unencrypted Plaintext
SCA	Side-Channel Analysis
SCC	Sealed Cryptographic Context
SEM	Scanning Electron Microscope
SIV	Synthetic Initialization Vector
SKID	System Key Identifier
SKS	System Key Store
TBC	Tweakable Block Cipher
TEE	Trusted Execution Environment
TPM	Trusted Platform Module
VA	Virtual Address
VMID	Virtual Machine Identifier

Acronym	Description
WARL	Write Anything Read Legal
WARZ	Write Anything Read-as Zero
XCTR	xored Counter (mode of encryption)
XEX	Xor-Encrypt-Xor mode of operation
XLFSR	xored Linear Feedback Shift Register (mode of encryption)
XOF	eXtendable Output Function
XTS	XEX mode with cipherText Stealing

Pseudocode Notation

- 0 denotes a single bit equal to zero.
- 1 denotes a single bit equal to one.
- $\text{bin}(n, m)$ = binary representation of n to m bits (m least significant bits, or sign extended).
- $\text{Galoismul}(a, b)$ = multiplication of a and b in a finite field implied by the context.
- $\text{len_in_bits}(M)$ = length of M in bits, i.e., $\text{ceil}(\log_2(M))$.
- $\text{len_in_bytes}(M)$ = length of M in bytes, i.e., $\text{ceil}(\log_2(M)) / 8$.
- $\text{lsb}(x)$ = least significant bit of x .
- $m[i \dots j]$ = bytes from the i -th to the j -th of byte array m .
- $m[j:i]$ = bit-slice from the i -th to the j -th bit of m .
- $\text{Montmul}(a, b)$ = Montgomery multiplication of a and b in a ring implied by the context.
- $\text{msb}(x)$ = most significant bit of x .
- $\text{ones}(i)$ = sequence of i ones.
- $\text{zeros}(i)$ = sequence of i zeros.

Copyright and license information

This specification is licensed under the Creative Commons Attribution 4.0 International License (CC-BY 4.0). The full license text is available at <https://creativecommons.org/licenses/by/4.0/>.

Copyright 2025 by RISC-V International.

This document is in the [Development state](#).

- !** Expect potential changes. This draft specification is likely to evolve before it is accepted as a standard. Implementations based on this draft may not conform to the future standard.
- Currently, there is no separation between the ISA and non-ISA parts of the architecture, and the language may not reflect Risc-V's guidelines.

Contributors

This RISC-V specification has been contributed to directly or indirectly by:

- Roberto Avanzi <mailto:ravanzi@qti.qualcomm.com>
- Nicolas Brunie <mailto:nicolas.brunie@sifive.com>
- Ruud Derwig <mailto:ruud.derwig@globalfoundries.com>
- Luis Fiolhais <mailto:luis.fiolhais@ext.semidynamics.com>
- Nick Kossifidis <mailto:mick@ics.forth.gr>
- Radim Krčmář <mailto:rkrmar@qti.qualcomm.com>
- G. Richard Newell <mailto:richard.newell@microchip.com>

- Jan Thoma <mailto:jan.thoma@codasip.com>
- Tolga Yalcin <mailto:tolga.yalcin@qti.qualcomm.com>
- Yan Yan <mailto:yan.yan@codasip.com>

Introduction

This brief introduction to the *Atomic Cryptography Extension* (ACE) is not part of the specification.

The ratification plan can be found [here](#).

Features

ACE defines an interface to high-performance cryptographic operations without giving software access to the keys.

Its development has been prompted by use cases such as content protection (including local LLM decryption), link and storage encryption, secure boot, and attestation.

ACE's main features are:

- To mitigate key misuse, ACE binds cryptographic secrets (such as keys) and metadata into an indivisible structure called a *Cryptographic Context* (CC). The architecture uses the metadata to restrict each secret to a specific algorithm and, optionally, to enforce usage constraints based on the process, VM, security state, boot cycle, an expiration date, or other policies.
- To implement CCs, ACE introduces *Context Registers* (CRs)—architectural containers that conceal their secrets from software. A CC is created by *provisioning* (i.e., writing) secrets and metadata into a CR. ACE instructions operate on CRs instead of explicit key values. As a result, software can use keys without holding them explicitly in the process's memory, while also benefiting from higher performance and side-channel-protected implementations.
- Operations such as full AES block encryption/decryption or absorption of data blocks into a hash function are performed *atomically*—i.e., in a single step.
- To enable CC reuse and support OS features like context switching, ACE provides a secure mechanism to store a CC to memory via authenticated encryption, producing a *Sealed Cryptographic Context* (SCC). Once generated, an SCC can later be re-imported into a CR. If an SCC is passed to a process in place of the key value, the key is not exposed in that process's memory.
- ACE provides a unified interface to multiple cryptographic primitives, avoiding the need for separate instructions per primitive.
 - Supporting only primitives is insufficient for modes like AES-GCM-SIV, XTS, and OCB, because leakage of derived keys or intermediate values can compromise security. Therefore, ACE also defines modes of operation and secure key derivation.
 - In addition to GCM-SIV, XTS, and OCB, ACE defines modes such as CTR, GCM, and CMAC. Where applicable, encrypt-only or decrypt-only versions and versions with a preset IV or nonce are also defined to reduce the potential

for misuse. These modes can be instantiated not only with AES-128/256 but also with other block ciphers such as SM4.

- Support for various hash functions.
 - The interface supports SCA-resistant implementations of the algorithms.
 - The interface can also be used to provide access to public-key and post-quantum algorithms.
 - Not all the algorithms defined in the specification must be supported by an ACE implementation: Following Risc-V practice, we defined several sub-extensions and delegate the choices to RV profiles, which would be defined by a separate process.
- ACE scales from embedded to server cores with a unified ISA and ABI.
 - ACE is compatible with sleep, hibernation, and VM migration.

Rationale for atomic operations Assuming the primitive is secure, the key cannot be recovered from the input and output of the full cryptographic operation. Typical round-based ISAs, which require keys to be supplied between rounds or MAC stages, allow for easy key recovery from the input and output, even if the key came from a hidden register. In fact, even if an instruction computed 4 or 5 rounds of the AES, an attacker with access to the instruction oracle would be able to efficiently recover the key cite:[Rijndael-revised].

Non-Goals

ACE intentionally excludes the following:

- **Debugging support:** The internal representation of a CR remains concealed even during debug.
- **Variable-length complete ciphertext and plaintext inputs:** Plaintext and ciphertexts, as well as messages of which digests are computed, are processed in fixed-size blocks.
- **Complete key management:** ACE provides only architectural building blocks that can be used for key management, where the key management logic resides in trusted software (e.g., TEE applets). ACE does not support external schemes for directly provisioning keys into a key store. Instead, it will be a local trusted SW or HW component that manages external schemes and, with the thus obtained keys and policies, then configures CCs for the ACE unit.
- **Key revocation:**
This specification does not define a key revocation mechanism. The rationale is:
 1. A revocation mechanism would add a significant implementation burden and adversely affect performance, conflicting with ACE's emphasis on simplicity and speed.
 2. A universal revocation mechanism is impractical: key providers use diverse protocols, and revocation data may grow without bound. Managing revocation within the ACE unit would require dynamic access to secure memory. Such functionality is better suited to isolated software components (e.g., enclaves, realms, or TEE applets)

that integrate with the local software stack while maintaining strong security guarantees.

3. Revocation typically applies to root keys, not the derived keys (e.g., session keys) used by user software. It can therefore be handled by the trusted entity managing the root keys. The expiration-date mechanism and *Localities* can be used to limit the lifetime and scope of derived keys.
 4. Applications requiring direct software access to root keys (e.g., link or storage encryption) typically execute in protected environments that can handle their own revocation mechanisms.
- **Full prevention of misuse by compromised processes:** ACE cannot fully prevent misuse by compromised processes because they retain access to their own CCs. ACE complements, but does not replace, existing software vulnerability mitigations and process isolation techniques. Additionally, certain modes (e.g., counter mode encryption) use the same operation for both encryption and decryption, which may enable misuse by malicious software. This risk is inherent to such modes, and cannot be mitigated.

TODOs / Open Discussion Points

- Finalize RVV-mini.
- We could use “k1” in this document instead of “ace.”: k1 means ACE is a subset of crypto extensions, the “l” is also the initial of “Locker”, and Cryptographic Locker could replace Cryptographic Register.
- Do we want to prevent, by default, an execution environment to prevent a Mode to *provision* a key that could be used by a higher privileged Mode? And how?

Structure of the ACE Specification: Separation of Architecture and Data Formats

The ACE ISA is split into three normative Books and a non-normative one.

- Book 1 defines the fixed part of the Unprivileged Architecture (see [Book 1: Unprivileged Architecture, Instructions and Architectural States](#)).
The instructions and architectural states are specified independently of the exact data formats they operate on.

Book 2 defines the Data Structures and Algorithms of the Unprivileged Architecture (see [Book 2: Algorithms](#)).

Within the RISC-V architecture, conformance to Book 1 requires full conformance to Book 2.

In other words, each non-custom algorithm implemented must exhibit the behavior fully specified in Book 2.

Book 2 is kept separate from Book 1 because the data formats used to manage keys are intended to be interoperable across different ISAs that implement ACE or equivalent functionality, and are therefore not specific to RISC-V. Those ISAs must support the format of the Metadata Header and the semantics of its fields verbatim as well.

- Book 3 defines the Privileged Architecture (see [Book 3: Privileged Architecture](#)).
- Book 4 provides non-normative pseudocode examples illustrating the intended use of the algorithms defined in Book 2.

1. Book 1: Unprivileged Architecture, Instructions and Architectural States

1.1. Introduction

ACE relies on certain fundamental concepts, which are introduced here.

While this leads to some repetitions, it also brings better clarity to the presentation.

1.1.1. Fundamental Concepts and Definitions

Cryptographic Context

The **Cryptographic Context** (CC) is the foundation of ACE. A CC is an indivisible structure consisting of a **Metadata Header** (MDH) and **Content**; all cryptographic operations performed by ACE execute within the constraints defined by a CC.

The MDH has a fixed format and specifies the cryptographic algorithm and various optional policies associated with the CC.

The Content has an algorithm-specific format and holds sensitive material such as keys and algorithm state.

The Metadata requires integrity; the Content requires both integrity and confidentiality. These requirements are architecturally enforced by ACE and by the cryptographic algorithms it implements. **Context Register**

To create CCs, ACE uses **Context Registers** (CRs), per-hart architectural containers that safeguard their contents.

A CR is **provisioned** by writing Metadata and Content into it; this operation binds the latter to form a CC.

There is no store-like instruction to extract the Content of the CR in a manner that compromises its confidentiality.

A CC may be used for cryptographic operations only while resident in a CR.

Only the ACE unit may access or modify a CC residing in a CR, in accordance with the rules set by the architecture and by the algorithm bound to the CC. **Sealed**

Cryptographic Context

To support context switching while protecting CCs outside architectural state, a CC may be **exported** from a CR as a **Sealed Cryptographic Context** (SCC)—an encrypted and authenticated representation of a CC that can later be **imported** into a CR. Export and import use a *Context Sealing Key* (CSK), which may be programmable by M-mode. An SCC sealed under one CSK cannot be imported under a different CSK, providing cryptographic domain separation across spatial and temporal domains. **Object-**

Orientation

CCs act as **objects** exposing a uniform interface across algorithms.

A CC encapsulates algorithm-specific behavior, enabling interchangeable use of different primitives (e.g., AES-128, AES-256, SM4) in identical modes (e.g., ECB, GCM-SIV, XTS) without requiring changes to compiled code.

Side-channel attack (SCA) resistant variants are supported transparently. **Usage**

Control Policy

ACE distinguishes between CC/CR **management** and **usage** operations.

Management operations include **configuration** (e.g., CR provisioning, SCC import, and cloning) and **export** of a CR as an SCC. Usage operations include cryptographic operations and state changes.

To prevent denial-of-service scenarios, management operations, in particular configuration operations, are far less restricted than usage operations. *Usage Control* policies restrict only certain CC/CR usage operations. **Localities**

ACE can bind CCs to a **Localities**, i.e., restrict usage to specific devices or device classes, hardware configurations, physical or virtual boot cycles, and process domains at various privilege modes. This binding is achieved by using the Localities as *tweaks* in the encryption and decryption of SCCs.

1.1.2. Architectural Model

This section is informative and non-normative.

ACE is architected as an *attached* unit, meaning that ACE instructions act as messages across a tightly coupled coprocessor interface.

The ACE unit receives the operations from the hart, executes them independently of the issuing hart, and returns results.

Beside implementations of ACE within a CPU core, this model also allows a single hardware instance of the ACE unit to be shared by multiple cores, with separate architectural states for each client hart. The specification mandates only behavior, not implementation.

We do not mandate—and cannot mandate—a pure hardware implementation of ACE. Although we generally discourage it, ACE may be implemented wholly or partially via a trap-and-emulate mechanism.

In certain cases, a partial trap-and-emulate approach may even be beneficial.

For example, smaller implementations may implement symmetric cipher operations (cf. [States for the Symmetric Algorithms Defined in this Specification](#)) in hardware while relying on trap-and-emulate for asymmetric cipher operations, such as elliptic curve algorithms (cf. [Elliptic Curve Asymmetric Algorithms](#)).

This approach is particularly advantageous for post-quantum cryptographic algorithms (cf. [Post Quantum Public-Key Cryptography](#)), which require very large states.

A trap-and-emulate implementation in M-mode firmware avoids the need to support large states inside the ACE unit, as long as the relevant CRs are stored in a secure SRAM or some other form of secure memory.

Trap-and-emulate exploits the `*crstatus`'s “Other” values which can be used also for lazy switching (cf. [Use of `\*crstatus`'s “Other” Value for Lazy Context Switching and CR Loading](#)).

The requirement that CR state be never accessible to any form of user software in clear forbids implementations in S/VS-mode or HV-mode.

The ACE unit may lock certain architectural states—e.g., Vector Registers, CRs, and CSRs.

The hart stalls when attempting to read them until the ACE operation using those resources completes.

1.2. Extensions Overview

Extension	Features	Defined in:	Minimum version	Lifecycle state
Zlv	ACE with support for vector registers. ¹	Instructions	v0.0.0	--
Zlio	ACE with support for <code>ace.input</code> and <code>ace.output</code> . ²	Using <code>ace.input</code> and <code>ace.output</code> to Replace Vector Inputs and Outputs	v0.0.0	--

Extension	Features	Defined in:	Minimum version	Lifecycle state
Zlmem	ACE with dedicated memory instructions. ³	ace.load , ace.store	v0.0.0	--
Zlmv	ACE with dedicated move instructions. ³	ace.mv	v0.0.0	--
Zlexpire	ACE with support for the <i>ExpirationDate</i> metadata field.	ExpirationDate Field	v0.0.0	--
Zlaes128p	AES-128 encryption/decryption. ⁴	Electronic Codebook Modes (Direct Encryption/Decryption)	v0.0.0	--
Zlaes192p	AES-192 encryption/decryption. ⁴	Electronic Codebook Modes (Direct Encryption/Decryption)	v0.0.0	--
Zlaes256p	AES-256 encryption/decryption. ⁴	Electronic Codebook Modes (Direct Encryption/Decryption)	v0.0.0	--
Zlsm4p	ShangMi 4 encryption/decryption. ⁴	Electronic Codebook Modes (Direct Encryption/Decryption)	v0.0.0	--
Zlctrm	CTR and CTR with set initial counter. ⁵	CTR/LFSR/XCTR/XLFSR Keystream Generation	v0.0.0	--
Zlxctrm	XCTR and XCTR with set initial counter. ⁵	CTR/LFSR/XCTR/XLFSR Keystream Generation	v0.0.0	--
Zlxexm	XEX construction. ⁶	XEX/XTS modes (one or two keys)	v0.0.0	--

Extension	Features	Defined in:	Minimum version	Lifecycle state
Zlgcmm	GCM and GCM with set IV. ⁵	GCM Mode	v0.0.0	--
Zlgcmsivm	GCM-SIV. ⁵	GCM-SIV Mode	v0.0.0	--
Zlocbm	OCB3 and OCB3 with set IV. ⁶	OCB Mode	v0.0.0	--
Zlcmacm	CMAC. ⁵	CMAC Functions	v0.0.0	--
Zlesha2h	SHA-224, SHA-256, SHA-384, SHA-512, SHA-512/224, and SHA-512/256.	Hash and MAC Functions, and XOFs	v0.0.0	--
Zlesha3h	SHA3-224, SHA3-256, SHA3-384, SHA3-512, SHAKE128, and SHAKE256.	SHA-3 Family Specific Rules	v0.0.0	--
Zlsm3h	ShangMi 3 (SM3) hash function.	Hash and MAC Functions, and XOFs	v0.0.0	--
Zlhmam	HMAC function with hidden key (with SHA2 when Zlesha2h is supported, and with SHA3 when Zlesha3h is supported).	TBD	v0.0.0	--
Zlkmam	KMAC function with hidden key. Depends on Zlesha3h, since KMAC is only defined with SHA3.	TBD	v0.0.0	--
Zlascon	Ascon-AEAD128 (base, with set	Ascon-AEAD128 to	v0.0.0	--

Extension	Features	Defined in:	Minimum version	Lifecycle state
	Nonce, or with Nonce Masking), Ascon-Hash256, Ascon-XOF128 and Ascon-CXOF128.	Ascon-CXOF128		
<i>Point arithmetic and signature generation and verification algorithms for curves in various standards:</i>				
Zlsecp256r1c	-- NIST curve secp256r1.	Elliptic Curve Asymmetric Algorithms	v0.0.0	--
Zlsecp384r1c	-- NIST curve secp384r1.	Elliptic Curve Asymmetric Algorithms	v0.0.0	--
Zlsecp521r1c	-- NIST curve secp521r1.	Elliptic Curve Asymmetric Algorithms	v0.0.0	--
Zled25519c	-- Edwards curve ed25519.	Elliptic Curve Asymmetric Algorithms	v0.0.0	--
Zled448c	-- Edwards curve ed448.	Elliptic Curve Asymmetric Algorithms	v0.0.0	--
ZlbpP256r1c	-- Brainpool curve brainpoolP256r1.	Elliptic Curve Asymmetric Algorithms	v0.0.0	--
ZlbpP384r1c	-- Brainpool curve brainpoolP384r1.	Elliptic Curve Asymmetric Algorithms	v0.0.0	--
ZlbpP512r1c	-- Brainpool curve brainpoolP512r1.	Elliptic Curve Asymmetric Algorithms	v0.0.0	--
ZlSM2c	-- Chinese 256-bit SM2 curve.	Elliptic Curve Asymmetric Algorithms	v0.0.0	--
<i>Post-Quantum Cryptography:</i>				
Zlmlkem512k	-- ML-KEM-512	ML-KEM	v0.0.0	--
Zlmlkem768k	-- ML-KEM-768	ML-KEM	v0.0.0	--
Zlmlkem1024k	-- ML-KEM-1024	ML-KEM	v0.0.0	--

Extension	Features	Defined in:	Minimum version	Lifecycle state
Zlm1dsa44s	-- ML-DISA-44/ML-DISA-44-PH	ML-DISA	v0.0.0	--
Zlm1dsa65s	-- ML-DISA-65/ML-DISA-65-PH	ML-DISA	v0.0.0	--
Zlm1dsa87s	-- ML-DISA-87/ML-DISA-87-PH	ML-DISA	v0.0.0	--
Zlkn	NIST Symmetric Suite. Depends on Zlaes128, Zlaes256, Zlesha2, Zlgcmm, Zlxexm, and Zlgcmsivm.		v0.0.0	--

¹ Requires V (or a suitable subset thereof). Must support the following CSRs: `acestart` (this is a dependency for `Zicsr`); must support the following instructions: `ace.prov`, all four forms of `ace.exec`, all Forms of `ace.setst`, all Forms of `ace.mgmt`, `ace.clear`, `ace.export`, `ace.import`, `ace.clone`, `ace.size/ace.avail`, `ace.getmd{l,h,v}`, and `ace.restrict{l,h,v}`; must support `Zlkn`.

² Must support the following CSRs: `acestart` (this is a dependency for `Zicsr`), `aceiobufen` and `aceiobufen`; must support the following instructions: `ace.prov`, Form D of `ace.exec`, Forms A and B of `ace.setst`, Forms A and B of `ace.mgmt`, `ace.input` and `ace.output`, `ace.clear`, `ace.export`, `ace.import`, `ace.clone`, `ace.size/ace.avail`, `ace.getmd{l,h}`, and `ace.restrict{l,h}`; must support `Zlkn`.

At least of `Zlmem` or `Zlmv` must be implemented. If `Zlmem` is not implemented in hardware, a trap-and-emulate implementation using `Zlmv` is required.

³ Beside defining the basic encryption and decryption operations, these extensions also select which ciphers can be used with the various modes of operations that follow.

⁴ The mode is applicable to all versions of AES and to SM4.

⁵ The mode is applicable to all versions of AES.

At least one of `Zlv` or `Zlio` must be implemented.

1.3. Programmer-Visible Architectural State

1.3.1. Context Registers

ACE defines 32 Context Registers (CRs), numbered 0 to 31 and denoted `k0–k31`.

CRs are per-hart architectural state, like GPRs or vector registers.

CRs are not separated by privilege Mode: when a hart changes Mode, it sees the same set of CRs.

Unlike a conventional register, a CR can be `_unconfigured_` meaning it does not contain a CC.

A CR is addressed either directly, via a 5-bit immediate, or indirectly, via a GPR value. An additional instruction-encoding bit selects the addressing mode. Accessing a CR with an indirect index outside [0..31] raises an illegal instruction exception.

An implementation must wait before performing any operation on a CR that is still being written to asynchronously by the ACE unit.

- i** Indirect CR addressing implies that CR renaming, while still possible, would significantly increase microarchitectural complexity. We posit that CR renaming is not expected to be necessary, and may even be detrimental, for ACE, for the reasons outlined below.
1. CRs are redefined far less frequently than GPRs or FP registers (excluding implicit internal state changes); and
 2. Speculating on the results of cryptographic operations usually leads to security issues.

We list the most important operations on CRs below, to refer to them before they are fully defined.

- Provisioning ([Provisioning, Import, and Export](#)) loads metadata and plaintext Content into a CR, binding them together.
- Exporting ([Provisioning, Import, and Export](#)) exports a CR to memory as an SCC.
- Importing ([Provisioning, Import, and Export](#)) imports a SCC into a CR.
- `ace.clear` ([ace.clear/ace.reset](#)) unconfigures and clears a CR.
- `ace.exec` ([ace.exec](#)) performs cryptographic operations as specified by the algorithm configured in a CR and according to the state of the CR.
- `ace.setst` ([ace.setst](#)) transitions a CC inside a CR between states of a cryptographic algorithm.
- `ace.mgmt` ([ace.mgmt](#)) transitions a CR between states of its management lifecycle.
- `ace.clone` ([ace.clone](#)) creates a copy of a CR into a second CR.

1.3.1.1. Representation of Cryptographic Registers for Accessing their Contents

A CR can be viewed as a register of variable and discoverable length (more details in [Notation and Representation of CR Internals](#)).

While an ACE implementation may work with an internal representation of a CR's content,

ACE provides a way to access a CR as if it were a register into which a Provisioning Input (PI) or Sealed Cryptographic Context (SCC), as defined in [External Cryptographic Context Formats](#) can be loaded.

To this purpose, ACE defines:

- Instructions `ace.load`, to load serialized data into CRs from memory (see [ace.load](#)), and `ace.store` (see [ace.store](#)) to export a CR's serialized content back to memory; and
- Instructions to make this information available to the ACE unit and to convert the internal representation of CRs to a SCC (by using `ace.setst`, see [ace.setst](#)).

1.3.1.2. The Context Register File

CRs reside in a dedicated, per-hart *CR file* (CRF), a memory accessible only to the ACE implementation.

The CRF shall guarantee confidentiality and integrity of its contents, including the inability for an attacker to roll back the latter to a previous state.

We do not require that accesses to this memory are hardened against physical side-channel attacks at the hardware level.

However, the implementation of side-channel-protected algorithms in the ACE unit shall provide resistance to such attacks as well.

The amount of CRF capacity consumed by each CR depends on the algorithm encoded in the MDH, and it may differ from one CR to another.

Insufficient residual CRF capacity may prevent execution of a configuration instruction. Capacity is freed by unconfiguring other CRs.

The CRF capacity is implementation-defined.

Implementations shall provide sufficient capacity for all supported operations.

At a minimum, the CRF must hold the largest possible single CR, or the largest CR pair used in cloning operations, and a small amount of additional authentication and identification information.

This small amount of additional storage space is not replicated per CR but is shared across all CRs per hart and it is used only when importing and exporting SCCs.

Any corruption of the CRF associated with a hart shall cause an unrecoverable error condition.

The entire architectural state of the ACE unit associated with the hart is reset, and exception `ace_guru_meditation` is raised (cf. [Exceptions Raised by ACE](#)).

1.3.2. Providing Inputs and Outputs

1.3.2.1. RVV/RVV-mini

ACE inputs and outputs may use vector registers or an architectural input/output buffer (ACEIOBUF)

that can be loaded from and stored to external memory. Both mechanisms may be supported simultaneously.

When vector registers are used, the full V extension is not required. ACE depends only on a minimal subset, termed *RVV-mini*.

RVV-mini must be fully opcode-compatible with RVV: code written for RVV-mini must execute with identical behavior under RVV.

The current set of features is:

- Vector registers (or groups) wide enough to support the inputs and outputs of the implemented algorithms, that is, currently, 128 bits.
- Configuration-setting instructions (`vsetvli/vsetivli/vsetvl`):
 - Since all ACE operations ignore masks while accessing vector registers, RVV-mini only needs tail and mask agnostic configurations;

- Since ACE operations do not depend on a concept of element (from a logical point of view, they are not performed in the vector unit), they view a vector (group) as a bit string of length $VL * SEW$ with $VL = \min(AVL, VL_{MAX})$, processing in each operation $VL * SEW$ bits.
- Support for LMUL integer values of 1, 2, 4, and 8;
- Unsupported configurations shall raise an illegal instruction exception.
- Unit-strided vector loads and stores; and
- Vector bitwise logical operations: `vxor`, `vand`, and `vor`.

ACE instructions do not use vector masking and do not define masked versions of `ace.exec`.

Therefore, RVV-mini does not need support for masked operations.

We are not using bit 25 of the instruction encodings, though, for potential future adoption of masked operations, should a compelling use case arise.

Floating-point, fixed-point, and general arithmetic vector operations are not required. Complex operations for certain cryptographic algorithms may be performed in GPRs, with data inserted into vector registers and result extracted from them via `vins/vext`.

RVV-mini need not be finalized before ACE ratification begins; it may be defined later, and the ACE specification amended accordingly. This allows additional requirements to be collected, for example, whether to mandate `vc1mul.v[vx]` and `vc1mulh.v[vx]` from Zvbc.

When vector registers are used, ACE determines the lengths of its operands from $VL * SEW$. The specific choice of VL and SEW is irrelevant; for example, $VL = 2$, $SEW = 64$ and $VL = 4$, $SEW = 32$ both define a 128-bit input.

The CPU instruction-decoding side of ACE uses VL and SEW to determine how much data to send to the ACE unit proper, but the ACE unit itself requires only the product $VL * SEW$ (or `aceiobuflen`).

1.3.2.2. ACEIOBUF

ACEIOBUF is an optional architectural input/output buffer that may be used in place of vector registers.

It is not part of the CRF.

The `ace.input` instruction loads data from memory into the ACEIOBUF to supply inputs to other ACE operations, which will write their outputs to the ACEIOBUF.

The `ace.output` instruction is used to store these outputs to memory.

When the ACEIOBUF is used in place of vector registers, the `ace.exec`, `ace.setst`, and `ace.derive` instructions that would otherwise use vector registers for their inputs and outputs are replaced by instruction sequences that combine the no-input/no-output versions of the same operations with `ace.input` and `ace.output`.

These sequences are described in [Using `ace.input` and `ace.output` to Replace Vector Inputs and Outputs](#).

When the ACEIOBUF is used, the length of the input is given in CSR `aceiobuflen` (`aceiobuflen`).

1.3.3. Metadata Formats

ACE recognizes two types of metadata that may be contained in a CC or associated with it.

1. A 128-bit *MDH* that determines the algorithm, the type of key, and various usage policies associated with the key.
This information is always present in a CC.
The fields are arranged so that the first 64 bits suffice to determine the length of a PI or SCC.
2. An optional 128-bit *Implementation Metadata* block that contains copies of the values of three architectural registers which are the ACE analogues of `mvendorid`, `marchid`, and `mimpid` CSRs for the hart implementation (see `acevendorid`, `acearchid`, and `aceimpid`).
The latter may be only present in an SCC and only if the SCC has been extended by the implementation with an implementation-specific content block.
This block is used by an implementation to determine whether it can use the aforementioned implementation-specific content block, even if generated by another implementation.

In this section we only deal with the 128-bit MDH.

Its fields are summarized in [Format of the 128-bit Metadata Header](#).

All *Reserved* bits, as well as bits unused by a given algorithm, must be zero.

Format of the 128-bit Metadata Header

Field	Size	Name	Description
[11:0]	12	<i>Algorithm</i>	This field encodes the cryptographic algorithm (primitive, mode, or protocol) that is bound to the key material. See Algorithm and AlgorithmPolicy Fields.
[13:12]	2	<i>AlgorithmPolicy</i>	Allowed algorithm variants(s), such as, for a symmetric cipher, whether it can be used for encryption, decryption, or both. See Algorithm and AlgorithmPolicy Fields.
[15:14]	2	Reserved	Reserved for future expansion of <i>AlgorithmPolicy</i> field.
[18:16]	3	<i>SCProtection</i>	Side-channel protection level. The levels of side-channel protection are defined in SCProtection Field.
[20:19]	2	<i>KeyType</i>	This field determines whether the key is given by value, is a system-defined key, or randomly generated. See KeyType Field for more details.
[25:21]	5	<i>State</i>	Current state of the CR/algorithm, encoded as an integer. For instance, whether the CR/algorithm is <i>Off</i> (unconfigured), encrypting or decrypting data, absorbing data in a hash/mac/tag, computing elliptic curve point multiples,

Field	Size	Name	Description
			generating or verifying signatures, or in an error state. It can be modified by the user to advance through various phases of an algorithm. See State and StateExtension Fields.
[29:26]	4	<i>StateExtension</i>	May be used by the algorithm to extend the <i>State</i> field with additional information. See State and StateExtension Fields.
[31:30]	2	<i>ConfigStatus</i>	Used to indicate whether the CR is fully configured and ready to operate, or is being provisioned, imported, or exported. See ConfigStatus Field.
[45:32]	14	<i>ImpDataLen</i>	If non-zero, indicates that an additional, variable-length, implementation-dependent data section appended to Content. The length of this additional data section is <i>ImpDataLen</i> * 128 bits (up to 256 KiB).
[61:46]	16	<i>AuxInfo</i>	Currently used only by ML-DSA to encode an auxiliary hashing algorithm. This field has the same format as bits [15:0].
[62:62]	1	<i>Reserved</i>	Reserved for future use.
[63:63]	1	<i>SystemFormat</i>	Value is 0 if an input is in the ACE architected format 1 if it is in a system-defined Format.
[68:64]	5	<i>UsagePolicy</i>	Usage (Control) Policy. Bits 0, 1, 2, 4 and 4 of the field, when set, disallow CC usage in U-mode, (V)S-mode, HS-mode, M-Mode, and Debug mode, respectively. Any bit of this field is ignored if the corresponding mode is not supported or not enabled.
[77:69]	9	<i>Locality</i>	Zero, or a value determining one or more Locality Secrets, used in the export/import encryption and decryption algorithm to bind the CR to specific environments. See Locality Field.
[79:78]	2	Reserved	Reserved for expansion of the <i>Locality</i> field.
[95:80]	16	<i>AlgorithmUse</i>	Reserved for algorithms and implementations to store additional state information.
[115:96]	20	<i>ExpirationDate</i>	If non-zero, the expiration date of this CC. See ExpirationDate Field. Expressed in hours since 00:00, January 1, 2027 (about 119 years and 7 months).
[127:116]	12	Reserved	Reserved for future use.

The fields as described in detail in the following subsections.

1.3.3.1. *SystemFormat* Bit

The use of a system-defined format is signalled by the *SystemFormat* bit MDH[63:0]. The format of the other 63 bits of MDH[63:0], as well as of the part of the input that would otherwise contain MDH[127:64], is entirely system specific, as is the rest of the input.

1.3.3.2. *Algorithm* and *AlgorithmPolicy* Fields

Algorithms are encoded in a 12-bit field.

Values 0–3071 and 4095 are architecture-defined.

The list of encodings is maintained by RVI (see [Book 2: Algorithms](#)).

The remaining values are available for custom algorithms.

For encryption/decryption primitives and modes, the *AlgorithmPolicy* field indicates whether the CC may be used for encryption (lower bit set), decryption (upper bit set), or both. At least one of the bits must be set.

For asymmetric primitives supporting signatures, the lower bit permits signature generation and the upper bit permits verification.

Such an asymmetric primitive can be configured to support primitives such as key agreement or key encapsulation but *not* signatures, and in this case both bits are zero.

For other primitives, the *AlgorithmPolicy* field may encode additional algorithm identifiers to expand the effective encoding space of the *Algorithm* field.

The information contained in the *Algorithm* field may affect the size of an SCC.

1.3.3.3. *SCProtection* Field

The architecture defines the following side-channel protection levels.

Encoding of Side-Channel Protection levels

Value	Description
0	Data-independent execution latency (DIEL) for cryptographic operations.
1	First-order threshold implementation.
2	DIEL; first-order threshold, fault-tolerant implementation.
≥ 3	Reserved/Currently not defined.

Implementations offering additional levels of protection may set *SCProtection* to a value greater than 2:

- Values 3-5 are reserved for future use by the architecture.
- Values 6-7 are reserved for implementation-custom protection levels.
In this case, the generating environment must use localities that prevent the use of the SCC on different architectures.

The information contained in this field may affect the size of an SCC.

1.3.3.4. *KeyType* Field

This field takes the value 0 if the key is given by an explicit value, and 1 if the key is given by a system-specific, 64-bit System Key Identifier (SKID).

Values 2 and 3 are currently reserved. See [System Keys and Randomly Generated Key Values](#).

If the value is 0, keys are provisioned, exported, and imported by value.

If the value is 1, keys are provisioned, exported, and imported as a System Key Identifier (SKID), except for the all-ones (i.e., `ones(64)`) SKID value.

If the SKID is the all-ones value, the key material is generated randomly upon provisioning, and then exported by value with *KeyType* set to 0.

Random generation applies to all keys at once in an algorithm, unless otherwise specified.

The information contained in this field may affect the size of an SCC.

1.3.3.5. *State* and *StateExtension* Fields

The *State* field is used to track the currently state of an algorithm, including certain types of errors.

The architected values of *State* common to all algorithms are defined in the following three tables.

Global CC State Numbers: Off State

Value	Mnemonic	State Name	Description
0	<code>ace_state_off</code>	<i>Off</i>	The CR is not configured.

Global CC State Numbers: Valid States

Value	Mnemonic	State Name	Description
1	<code>ace_state_initial</code>	<i>Initial</i>	The state of a just provisioned CC; ready for operation.
2-21	Defined by the specification of the Algorithm encoded in the <i>Algorithm</i> field.		
22	<code>ace_state_success</code>	<i>Success</i>	Algorithm completed successfully. No further cryptographic operations may be performed until the CR is reconfigured or transitioned to <i>Initial</i> .
23	<code>ace_state_failure</code>	<i>Failure</i>	Failure in an algorithm, usually an authentication failure in AE(AD) algorithms or a signature verification failure. No further cryptographic operations may be performed until the CR is reconfigured or transitioned to <i>Initial</i> .

Global CC State Numbers: Error States

24	<code>ace_state_not_impl</code>	<i>Not_Impl</i>	Not implemented algorithm. It is set by the hardware upon an attempt to provision or import a CR that is not supported by the implementation.
----	---------------------------------	-----------------	---

25	<code>ace_state_invalid</code>	<i>Invalid</i>	Invalid/invalidated state, for instance because of the use of an invalid value (argument, state, or metadata).
26	<code>ace_state_out_of_mem</code>	<i>Out_of_Memory</i>	Insufficient CRF memory for configuration instruction. This may be set by privileged code in case they do not handle the CR allocation.
27	<code>ace_state_CR_import_auth</code>	<i>Auth_Failed</i>	Authentication failure with <code>ace.import</code> .
28	<code>ace_state_priv_viol</code>	<i>Priv_Viol</i>	Violation of Usage Control policies.
29	<code>ace_state_expired</code>	<i>Expired</i>	The CR has expired. This is set by the hardware even if the condition also raises an exception.
30, 31	(Reserved)		Reserved for future use.

State 0 is the *Off* state. It denotes that the CR is unconfigured.

All states in the range 1-23 are termed *valid*. This includes both *Success* and *Failure* states, which must be used for normal termination of an algorithm in the cases of its successful completion or failure.

The immediate in the `ace.setst` instruction ([ace.setst](#)) is a 7-bit value, whereas the *State* field is only 5 bits long.

This allows to encode additional operations related to state changes (such as stages of import, export, or provisioning code) as `ace.setst` with the two high bits of the immediate taking non-zero values.

States 24-31 are termed *Error States*, and are used to report certain specific ACE errors.

In particular, if the State is `ace_state_invalid`, we say that the CR is *invalidated*.

The Error States are currently listed in decreasing severity order, with Error State 24 being the most severe and Error State 29 the least severe.

Error States 30 and 31, if defined, will not necessarily have a lower priority than Error State 29, and may be therefore not in order.

Error conditions that lead to exceptions are treated in [Exceptions Raised by ACE](#).

If an algorithm defines 23 or fewer valid states (including *Initial*, *Success*, and *Failure*), the current state is stored in the *State* field.

For algorithms with more than 23 valid states, additional state information may reside in the Content section in an algorithm-specific format.

The following rules are generic and apply to all algorithms:

1. The MDHs's *State* field must reflect the current state whenever the CC is in *Initial*, *Success*, *Failure*, or any Error State.
This ensures that these states can be unambiguously determined from the *State* field alone, without inspecting the Content.
2. In States *Success* and *Failed*, only the following ACE instructions are permitted: `ace.prov`, `ace.import`, `ace.export`, `ace.clear`, `ace.size`, `ace.setst`, and `ace.getst` (cf. [Instructions](#) for the descriptions of the instructions).
3. In States *Success* and *Failed*, `ace.setst` can only be used permitted to transition to *Initial* or *Off*.
4. A transition from any valid state to *Initial* using instruction `ace.setst` is always permitted.
While an algorithm may want an initial state to be used only once to avoid certain cryptographic attacks, a CR can always be restored from a backup SCC, so preventing transitions to *Initial* are not effective mitigations.
5. Any transition may be caused by an performing an explicit `ace.setst` operation performed while in a given state, or may be induced by certain conditions specified in the algorithm's specification.
6. The State after `ace.setst` is issued may differ from the State specified by the instruction.
Beside error reporting through an Error State, a common example is given by MAC verification: after `ace.setst` is issued with an `ace_state_hash_verify` immediate as the next state and the value to be verified as the additional parameter, the algorithm transitions to either *Success* or *Failure*. This is done to prevent the manipulation of a CR's state and to reduce code size.
An `ace.exec` instruction may also trigger a state change.
7. Upon transitioning to an Error State, the Content of a CR is invalidated and emptied.
Internal resources associated with the Content may also be released. In that case, any additional variable-length data is released as well and the *ImpDataLen* field is set to 0.
The CR then retains only the MDH.
8. The transitions from *Off* to any Error State are a result of configuration instructions, whereas the corresponding transitions from these Error States to *Off* are a result of explicitly clearing the CR.
9. Exporting a CR in an Error State exports only the MDH and the authentication tag(s).
Requesting the SCC's size (`ace.size` [ace.size](#)) will return 32 (bytes).
The resulting SCC can be reimported, allowing software to take appropriate actions even after a context switch.
10. If a CR is in any Error State, it can still be reconfigured, cloned, or cleared. `ace.setst` can only be used to change the Error State (for instance by an exception handler) or transition the CR to *Off*.
11. Bits [63:0] of the MDH of a CR in an Error State can be read (using `ace.getmdl`: [ace.getmdv/ace.getmdl/ace.getmdh](#)).
12. Bits [63:0] of the MDH of a CR in the *Off* State can be read (using `ace.getmdl`: [ace.getmdv/ace.getmdl/ace.getmdh](#)), returning all zero values.

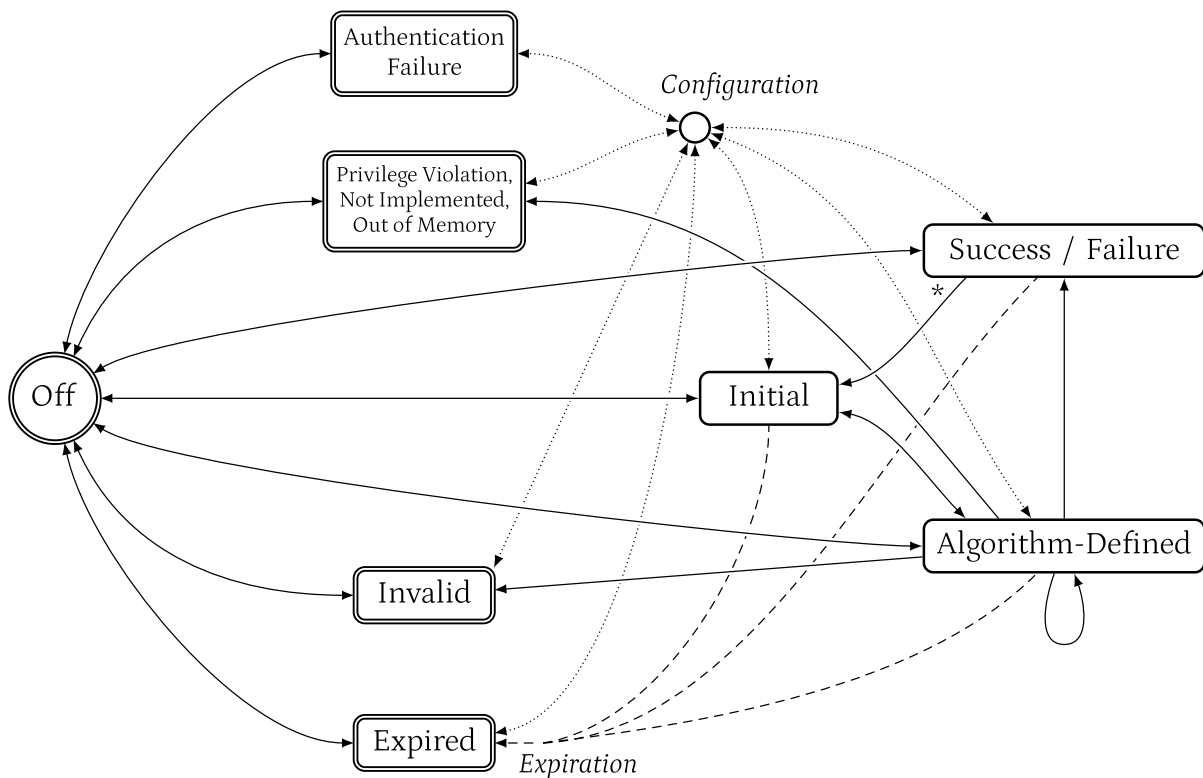
13. Attempts to *use* a CR in any Error State will not perform any operation, and in particular they leave the *State* field unchanged.
14. Any *usage* instruction that would violate the Usage Control policies will not perform its operation, and will set *State* to `ace_state_priv_viol` instead.

In [Default State Transitions](#) we display the default possible state transitions.

The legend for the graph is:

- The double circled state is the *Off* State.
- Valid States have single rounded rectangular borders.
- Error States have double rounded rectangular borders.
- The continuous edges represent state transitions induced by both configuration and usage instructions.
- The dashed edges represent state transitions induced by reaching the expiration date.
- The dotted edges represent state transitions induced by configuration operations.
Completing a provisioning operation can only lead to the *Initial* state.
Completing an import can lead to any state.
- The round *Configuration* “hub” simplifies the graph by expressing that any Error State is always reachable—even from another Error State—through configuration operations (or via `ace.setst`).

Default State Transitions



The *StateExtension* field can be used by an Algorithm to store additional state information.

The information contained in this field may affect the size of an SCC.

1.3.3.6. ConfigStatus Field

The admissible values of this field are given in the table below.

Encoding of ConfigStatus field

Value	Mnemonic	Description
0	<i>CfgStComplete</i>	The CR is fully configured and ready to operate.
1	<i>CfgStProvisioning</i>	The CR is partially configured as a result of an interrupted provisioning operation.
2	<i>CfgStImporting</i>	The CR is partially configured as a result of an interrupted import operation.
3	<i>CfgStExporting</i>	The CR has been placed in exporting mode, i.e., its contents have been encrypted and authenticated for export..

Usage and cloning of a CR whose *ConfigStatus* is not *CfgStComplete* are not allowed.

The information contained in this field shall *not* affect the size of an SCC.

1.3.3.7. ImpDataLen Field

The *ImpDataLen* MDH field records the length of the *Additional Data Section* of the content, which is a variable-size additional part that depends on the implementation.

The format of the *Additional Data Section* is described in [Formats of Provisioning Input and Sealed Cryptographic Context](#).

If *ImpDataLen* is nonzero, the first 128-bit block of this additional section contains a copy of the implementation qualifier

```
IMPQUAL := zeros(32) @ aceimpid @ acearchid @ acevendorid
```

(cf. *acevendorid*, *acearchid*, and *aceimpid*) from the originating machine. Its 16 bytes are counted within the length recorded by the *ImpDataLen* field.

The importing implementation uses this value to determine whether any information in the Implementation-Specific Data Section can be used.

1.3.3.8. Locality Field

The Locality Secrets Table (LST) contains 128-bit Locality Secrets (LS).

Some entries are fixed or only configurable through an implementation-specific authenticated hardware process.

These are shared across all ACE units within the same SoC.

Other entries are SW-programmable per-hart architectural state.

[Architected Localities](#). Bits refer to the *Locality* field in the MDH. lists the architected Localities.

Twelve LST entries are defined: the first six form the *HW Binding Group*, which binds SCCs to the hardware;

entries 6–7 form the *Boot Session Group*; and

entries 8–11 form the *SW Filter Group*.

For each Locality, the table provides: index in the LST; domain (SoC-, device-, VM-wide, or per-process); configurability; bit-field encoding; and value.

Architected Localities. Bits refer to the *Locality* field in the MDH.

Idx	Locality	Description	Domain	Config?	Bits	Value
	No HW Binding	Independent of Chip, OEM, Device	--	--	[3:0]	0
0	<i>SiPScrt</i>	Identifies the designer of the SoC. Permanent.	SoC	No	[1:0]	1
1	<i>ChipFamScrt</i>	Identifies specific chip/ SoC model/family. Permanent.	SoC	No	[1:0]	2
2	<i>ChipScrt</i>	Unique for each SoC. Permanent.	SoC	No	[1:0]	3
3	<i>OEMScrt</i>	Provisioned by OEM to differentiate their products from the competition. Permanent, or reconfigurable via a custom hardware authenticated mechanism.	Device	HW	[3:2]	1
4	<i>ProdScrt</i>	Identifies device/system type/model (product). Permanent or reconfigurable like OEMScrt.	Device	HW	[3:2]	2
5	<i>DevScrt</i>	Unique device (entire system) secret. Shared across all harts in a SoC and shareable across multiple SoCs in a single device using an implementation-defined mechanism. Reconfigurable like OEMScrt.	Device	HW	[3:2]	3
	No Boot Binding	Independent of Physical or Virtual Boot Session	--	--	[5:4]	0
6	<i>PhysBootScrt</i>	Unique per the boot session of the host hardware and its firmware stack. Configured by	Device	M	[5:4]	1

Idx	Locality	Description	Domain	Config?	Bits	Value
		M-mode during the boot process of the host (see macePhysBootScri Group).				
7	<i>VirtBootScri</i>	Unique per the boot session of the virtual hardware and its software stack. Regenerated by H or higher at each virtual boot of a VM (see haceVirtBootScri Group).	VM	H/M	[5:4]	2
No Binding to Modes		Independent of M-, S-, or any VS-mode Localities	--	--	[7:6]	0
8	<i>MLocality</i>	Configurable at M. Useful to separate Supervisor Domains/ Security States.	< M	M	[7:6]	1
9	<i>HLocality</i>	Configurable at HS.	< HS	HS	[7:6]	2
10	<i>SLocality</i>	Configurable at (V)S.	< (V)S	(V)S	[7:6]	3
11	<i>Nonce</i>	Always re-configurable (out-of-band peer-to-peer nonce). Only bits [95:0] are defined.	hart	U	[8:8]	0/1

Implementations are not required to populate every entry in the HW Binding Group, except for *ChipFamScri* and *DevScri*.

If an entry is unconfigured, it is transparently replaced by the next defined entry in its chain: *SiPScri* → *ChipFamScri* → *ChipScri*, and *OEMScri* → *ProdScri* → *DevScri*.

If no entry is configured in these chain but one Locality is requested, the implementation must raise an authentication failure error.

HLocality is active only when the H extension is enabled and V=1; otherwise, it mirrors *MLocality*.

i A system key identifier may refer to keys with unrelated purposes on different devices, leading to unspecified behavior. Implementations shall assign usage policies at least as restrictive as the key's own native policies, and supply an appropriate *Locality* during CR provisioning.

Up to five architected Localities may be active concurrently: two from the HW Binding Group, one of either *PhysBootScri* or *VirtBootScri*, one from the SW Filter Group, and the *Nonce*. The SCC generation and import algorithms described in [Management of Sealed Cryptographic Contexts](#) enforce the policies associated with these Localities.

The information contained in the *Locality* field shall *not* affect the size of an SCC.

1.3.3.9. *ExpirationDate* Field

This field is only in effect of extension *Zlexpire* is supported.

If *Zlexpire* is supported, the ACE implementation must have access to a secure clock.

If *Zlexpire* is not supported, the implementation shall reject PIs and SCCs with a non-zero *ExpirationDate* field.

If non-zero, the 20-bit *ExpirationDate* field specifies the CC's expiration time as hours elapsed since the base date of January 1, 2027. The maximum representable time span is approximately 119 years and 7 months.

Checking the *ExpirationDate* is the first step performed by any *usage* operation. When a usage instruction executes on a CR whose *ExpirationDate* has passed, the hardware immediately sets the MDH's *State* field to `ace_state_expired` and performs all actions prescribed for Error States, including implementation-specific tasks such as releasing the CRF capacity held by the Content. If the ACE privileged architecture is implemented, an `ace_exc_invalid` exception is raised (see [Exceptions Raised by ACE](#)).

This check is *not* performed by management operations (such as exporting, cloning, or configuring), and in particular it is not performed if the *ConfigStatus* is not *CfgStComplete*. This implies that an `ace_exc_invalid` exception is raised only when the expired CR is first used.

The information contained in this field shall *not* affect the size of an SCC.

1.3.3.10. *AlgorithmUse* Field

The *AlgorithmUse* field can be used by an Algorithm to store additional information.

If not used by the Algorithm, the implementation may use this field.

The information contained in this field shall *not* affect the size of an SCC.

1.3.4. Memory Alignment Constraints

If the address accessed by an ACE memory instruction is not naturally aligned to the effective XLEN, the context is transferred successfully or an address-misaligned exception is raised on that address.

There are no atomicity requirements for any ACE memory instruction.

1.3.5. Memory Model

ACE memory instructions appear to execute in program order on the local hart.

ACE memory instructions follow RVWMO at the instruction level. If the *ztso* extension is implemented, ACE memory instructions additionally follow RVTSO at the instruction level.

ACE memory instructions may be decomposed into a set of component memory operations of any granularity.

The ordering of these memory operations within the instructions is not specified, and it depends on the chosen memory model and implementation of the issuing hart, as long as the semantics of the configuration and export instruction and `ace.exec` are respected.

- i** `ztso` only imposes RVTSO at the instruction level; intra-instruction ordering follows RVWMO regardless of whether `ztso` is implemented.

- !** 1. More formal definitions required.
- 2. Discuss whether the ACE Unit may issue XLEN-sized memory operations to the LSU in a prescribed order, to allow proper resumption.

1.3.6. CSRs

ACE provides a number of unprivileged CSRs.

ACE Unprivileged CSRs.

Address	Privilege	Field	Description
0xXXX	RO	<code>acevendorid</code>	Manufacturer of the ACE unit. See <code>acevendorid</code> , <code>acearchid</code> , and <code>aceimpid</code> .
0xXXX	RO	<code>acearchid</code>	Architecture version, per vendor, of the ACE unit. See <code>acevendorid</code> , <code>acearchid</code> , and <code>aceimpid</code> .
0xXXX	RO	<code>aceimpid</code>	Implementation, per vendor, of the ACE unit. See <code>acevendorid</code> , <code>acearchid</code> , and <code>aceimpid</code> .
0xXXX	URW	<code>acestart</code>	ACE start bit index.
0xXXX	URW	<code>aceiobufen</code>	ACE input/output buffer length in bits.
0xXXX	RO	<code>acemaxiobufen</code>	ACE maximum input/output buffer length in bits. Read only.
0xXXX	URW	<code>aceiobufen</code>	ACE input/output buffer limit for current transfers in bits.
0xXXX-XXX	URW	<code>acenonce0</code> , <code>acenonce1</code>	Nonce to be used in export and import operations (see Management of Sealed Cryptographic Contexts).

1.3.6.1. `acevendorid`, `acearchid`, and `aceimpid`

`acevendorid`, `acearchid`, and `aceimpid` identify the version of the ACE unit, analogously to `mvendorid`, `marchid`, `mimpid` CSRs for the hart.

The `acemvvendorid` CSR is a 32-bit read-only register providing the JEDEC manufacturer ID of the provider of the ACE unit (or 0 if it is not commercial).

The `acemarchid` CSR is a 32-bit read-only register encoding the base microarchitecture of the ACE unit. Non-commercial IDs are allocated globally by RVI, while commercial IDs are specific to the `acemvendorid`.

The `acemimpid` CSR is a 32-bit read-only register that provides a unique encoding of the version of the ACE unit implementation. This register must be readable in any implementation, but a value of 0 can be returned to indicate that the field is not implemented. The Implementation value should reflect the design of the ACE unit itself and not any surrounding system.

These three CSRs, together, form the *Implementation Qualifier* (see [ImpDataLen Field](#)).

Included in

Extension	Minimum version	Lifecycle state
Zlv	v0.0.0	--
Zlio	v0.0.0	--

1.3.6.2. `acestart`

`acestart` is an XLEN-bit RW CSR.

`acestart` tracks the progress of resumable ACE instructions, such as `ace.load`, `ace.store`, `ace.input` and `ace.output`.

Progress tracking by `acestart` also applies to `ace.exec` when iteratively processing a multi-block operand, as `ace.exec` may be interrupted between blocks.

Only these instructions honor a nonzero `acestart`; all other instructions ignore it and are restarted, with `acestart` also being cleared.

Since ACE does not have an *intrinsic* concept of element length, `acestart` counts the number of *bytes* processed so far by `ace.exec`, not the number of elements, or read/written by `ace.input/ace.output` and `ace.prov/ace.import/ace.export` OR `ace.store/ace.load`.

Hardware writes this register on interruption.

Software may save and restore the register across context switches.

For certain algorithms, writing zero to `acestart` to force a full restart of `ace.exec` may lead to inconsistent or unpredictable results.

Writing an invalid or non-supported nonzero value raises an illegal instruction exception.

Included in

Extension	Minimum version	Lifecycle state
Zlv	v0.0.0	--
Zlio	v0.0.0	--

1.3.6.3. acemaxiobufen

acemaxiobufen is an XLEN-bit RO CSR that reports the maximum configurable length of the ACEIOBUF.

Included in

Extension	Minimum version	Lifecycle state
Zlv	v0.0.0	--
Zlio	v0.0.0	--

1.3.6.4. aceiobufen

aceiobufen is an XLEN-bit URW CSR that programs the ACEIOBUF length in bits for use by `ace.input` and `ace.output`.

Writing this register—even when re-writing the current value—also zeroes the buffer and sets `aceiobuf_top` to the same length and `ace_start` to zero.

When `aceiobufen` is zero, the ACEIOBUF is *unconfigured*.

Attempting to execute an operation that expects an input or an output in the ACEIOBUF while the latter is unconfigured raises exception `ace_exc_buf_unconf`.

Writing a value larger than `acemaxiobufen` raises exception `ace_exc_out_of_mem`.

- i** As per the Privileged Architecture, the dirtiness of the ACEIOBUF is not tracked except as part of the entire ACE state in `*status`. Context switching code is expected to save and restore the contents of the ACEIOBUF up to `aceiobufen`. We recommend software to set `aceiobufen` to zero once the contents of the ACEIOBUF are no longer needed. Context switching code can use this information to avoid unnecessary save and restore of ACEIOBUF contents.

Included in

Extension	Minimum version	Lifecycle state
Zlio	v0.0.0	--

1.3.6.5. aceiobuf_top

`aceiobuf_top` is an XLEN-bit URW CSR that marks the upper bound in bits of the buffer range used by `ace.input` and `ace.output`.

Together with `ace_start`, it defines the active transfer window.

The constraints $\text{acestart} \leq \text{aceiobuftop} \leq \text{aceiobufen}$ must hold. Violating either bound raises an illegal instruction exception. Modifying `aceiobuftop` does not modify buffer contents.

i `aceiobuftop` enables per-operation length adjustments without the need to reconfigure the ACEIOBUF using `aceiobufen`, which can be expensive due to the fact that it clears the buffer and may lead to internal garbage collection. This can be used similarly to the reconfiguration of `VL*SEW` to provide an partial “last block” or to supply a short IV/nonce to cryptographic algorithms.

Using a alternative sequence for `ace.exec` or `ace.setst` for any Form with an input or output

when `aceiobuftop - acestart` is zero (and `aceiobufen` nonzero) will result in no operation being performed.

Using ACEIOBUF when `aceiobuftop - acestart` is an invalid length raises an illegal instruction exception.

Since context switching code saves and restores the contents of the ACEIOBUF up to `aceiobufen`, the contents between `aceiobuftop` and `aceiobufen` are preserved.

Included in

Extension	Minimum version	Lifecycle state
Zlio	v0.0.0	--

1.3.6.6. `acenonce0` and `acenonce1`

`acenonce0` and `acenonce1` are XLEN-bit URW CSR that define the nonce to be used in `ace.import` and `ace.export` (see [Management of Sealed Cryptographic Contexts](#)). `acenonce0` contains bits [63:0] of the nonce, and `acenonce1` contains bits [95:64] of the nonce.

The most significant 32 bits of `acenonce1` are fixed to zero.

On RV32, 32-bit CSR `acenonce0h` shadows bits [63:32] of `acenonce0`, allowing these bits to be accessed.

Included in

Extension	Minimum version	Lifecycle state
Zlv	v0.0.0	--
Zlio	v0.0.0	--

1.4. Instructions

i The encodings presented here are preliminary.

ACE provides a number of unprivileged instructions. This section specifies their semantics and encodings.

The notation $\kappa\{x_d\}$ denotes the CR whose index is held in GPR x_d .

$\kappa_n | \kappa\{x_n\}$ indicates two alternative encodings: a 5-bit immediate CR index (κ_n) or a CR number supplied by GPR x_n . Use of the zero register x_0 as a CR index register raises an illegal instruction exception unless explicitly indicated otherwise.

i ACE does not have concepts similar to the vector crypto extension's Element Group Width (EGW), Effective Element Width (EEW), and Element Group Size (EGS). Since the architecture decouples instructions like `ace.exec` from the actual operations, the high-level design of ACE needs only the total input and output size, denoted `ACELEN`. This is equal to either `VL * SEW` or the `aceiobuftop` length of the `ACEIOBUF`, as defined in [Notation in the Algorithm Descriptions](#).

1.4.1. Provisioning, Import, and Export

This subsection is informative.

Broadly speaking, and removing error handling, **Provisioning** is performed by the following three steps:

1. Use `ace.mgmt` ([ace.mgmt](#)) to prepare a CR to be provisioned, check the validity of `MDH[63:0]` and load it in the CR.
2. Use `ace.load` ([ace.load](#)) to load the rest of the provisioning information into the CR.
3. Use `ace.mgmt` to make the CR available to software.

Importing a SCC into a CR is similar:

1. Use `ace.mgmt` to prepare a CR to accept a SCC, store `MDH[63:0]` in it, check its validity.
2. Use `ace.load` to load the rest of the SCC information into the CR.
3. Use `ace.mgmt` to authenticate and decrypt the SCC, and make the CR available to software.

Exporting a CR as an SCC consists of the following steps:

1. Use `ace.getmdl` ([ace.getmdv/ace.getmdl/ace.getmdh](#)) to get `MDH[63:0]` of the CR to be exported to be able to determine its size and allocate resources.
2. Store `MDH[63:0]` into memory.
3. Use `ace.mgmt` to prepare a CR to produce a SCC. This includes encrypting its contents and computing the authentication tag(s).
4. Use `ace.store` ([ace.store](#)) to store `MDH[127:64]` and the rest of the SCC to memory.
5. Use `ace.mgmt` to restore the CR in a form available to software.

1.4.2. ace.load

Encoding `ace.load` is an I-type instruction. [wavedrom, , svg]

```
{reg:[
  { bits: 7, name: '0x0b', attr: ['custom-0'] },
  { bits: 5, name: 'context', attr: ['Kd', 'K{Xd}'] },
  { bits: 3, name: 'ace.load', attr: ['0', '0', '0', '0', '0', '1'] },
  { bits: 5, name: 'Xs1' },
  { bits: 12, name: 'immed12' },
]}
```

Mnemonics `ace.load` `Kd|K{Xd}, %offset(Xs1)`

`ace.sysimport Kd|K{Xd}, %offset(Xs1)` *Description*

`ace.load` imports a PI or a SCC into the CR `Kd` or `K{Xd}`, starting with the 8th byte of the MDH.

The memory address of the latter is given in `%offset(Xs1)`.

This instruction requires that the CR is ready to import information and that the first 8, resp., 16 bytes of the MDH have already been loaded by passing them as a parameter in a GPR (pair for RV32), resp., vector register, to the `ace.mgmt` instruction used to open the CR for provisioning or import.

If the *ConfigStatus* field equals *CfgStComplete*, an invalid instruction exception is raised.

`ace.load` can load data larger than the effective XLEN.

`acestart` tracks the number of loaded bytes. If `ace.load` is interrupted, `acestart` contains the offset to the next byte to load upon resumption.

The implementation first determines the size of the input to be loaded from the first 8 bytes of the MDH.

The *ConfigStatus* field determines the input type (PI or SCC), and the fields *Algorithm*, *AlgorithmPolicy*, and *KeyType* determine its length.

`ace.load` then requests the LSU to read the memory contents starting at `%offset(Xs1)`, in XLEN-bit sized loads,

and copies them into the CR starting with the 8th byte (i.e., bits [127:64] of the MDH).

The order in which the LSU issues these transactions is implementation dependent. See [Management Operations](#), [Resumption and Memory Model](#) and [Management of Sealed Cryptographic Contexts](#) for further details.

When the key is in a system-defined format, the semantics of `ace.load` are entirely implementation dependent and do not assume that the CR has been readied to import information or that some metadata has already been loaded. System-defined format is signaled by the *SystemFormat* bit of MDH[63:0], which in this case may have a system-defined length.

The instruction may possibly modify state.

The instruction is not usage-controlled.

Included in

Extension	Minimum version	Lifecycle state
Zlmem	v0.0.0	--

1.4.3. ace.store

Encoding `ace.store` is an S-type instruction. [wavedrom, , svg]

```
{reg:[
  { bits: 7, name: 0x27, attr: ['custom-1'] },
  { bits: 5, name: 'immed[4:0]' },
  { bits: 3, name: 'ace.store', attr: [0x0,0x1] },
  { bits: 5, name: 'Xs1' },
  { bits: 5, name: 'context', attr: ['Ks2','K{Xs2}'] },
  { bits: 7, name: 'immed[11:5]' },
]}
```

Mnemonic `ace.store %offset(Xs1), Ks2|K{Xs2}`

Description

Exports raw data from the CR `Ks2` or `K{Xs2}`, starting with the 8th byte of the MDH, to memory at address `%offset(Xs1)`.

The first 8, resp., all the 16 bytes of the MDH must be read from the CR using `ace.getmdl` into a GPR (pair for RV32), resp., into a vector register, and saved to memory by the software.

`ace.store` can store data that may be larger than the effective XLEN.

`acestart` keeps track of the number of stored bytes, starting from 8 or 16 depending on the use of `ace.getmdl`.

If `ace.store` is

interrupted, `acestart` contains the offset to the next byte to store upon resumption.

i Modifying the CR being used by the interrupted `ace.store` always forces the store to restart. Software that aims to optimize this case will have to inspect `ConfigStatus` in the the metadata of each CR and detect if the store hasn't completed to impede the restart.

If the *ConfigStatus* field is equal to *CfgStComplete*, then an invalid instruction exception is raised.

If the *ConfigStatus* field is not equal to *CfgStComplete*

the implementation first determines the size of the output to be generated from first 8 bytes of the MDH.

The type of the output, i.e. whether it is a PI or a SCC, is determined by the *ConfigStatus* field, and

its length by *Algorithm*, *AlgorithmPolicy*, and *KeyType*.

Then, `ace.store` requests the LSU to store the CR to memory, beginning with the 8th byte (i.e. bits [127:64] of the MDH), in XLEN-bit sized stores, in the memory region starting at address `%offset(Xs1)`. The order in which the LSU issues these transactions is implementation dependent. See [Management Operations](#), [Resumption and Memory Model](#) and [Management of Sealed Cryptographic Contexts](#) for further details.

i We allow to store a PI because it may be possible that a provisioning operation be interrupted, and this allows resuming.

The instruction does not modify state.

The instruction is not usage-controlled.

Included in

Extension	Minimum version	Lifecycle state
Zlv	v0.0.0	--
Zlio	v0.0.0	--

1.4.4. `ace.exec`

Synopsis Perform a cryptographic operation in the ACE unit. *Mnemonic*

The `ace.exec` instruction admits four Forms, namely

1. `ace.exec Vd, Kn|K{Xn}, Vs2` takes an input vector and writes to an output vector.
2. `ace.exec Kn|K{Xn}, Vs2` takes an input vector but does not write to an output vector.
3. `ace.exec Vd, Kn|K{Xn}` does not take an input but writes to an output vector.
4. `ace.exec Kn|K{Xn}` does not take an input vector and does not write to an output vector.

Encoding `ace.exec` is an R-type instruction.

[wavedrom, , svg]

```
{reg:[
  { bits: 7, name: 0x5b, attr: ['custom-2'] },
  { bits: 5, name: 'rd', attr: ['Vd','-', 'Vd','-'] },
  { bits: 3, name: 0x0, attr: ['ace.exec'] },
  { bits: 5, name: 'rs1', attr: ['Ks1|K{Xs1}'] },
  { bits: 5, name: 'rs2', attr: ['Vs2','Vs2','-', '-'] },
  { bits: 1, name: 0x0 },
  { bits: 1, name: 'r' },
  { bits: 1, name: 0x0 },
  { bits: 2, name: 'Form', attr: ['0 0','0 1','1 0','1 1'] },
```

```
{ bits: 2, name: 0x0, attr: ['func2'] },
]}
```

Bit *r* (i.e., bit 26) takes the value 0 if the context is encoded as an immediate, and it takes the value 1 if a GPR (scalar integer value) is used to reference it.

For the four Forms A–D above, the encodings are:

1. Form = 0b00.
2. Form = 0b01 and *rs1* = 0b00000.
3. Form = 0b10 and *rs2* = 0b00000.
4. Form = 0b11 and *rs2* = *rs1* = 0b00000.

Description

Performs a cryptographic operation as defined by the algorithm in CR *ks1* or *K{xs1}*. The instruction may take an input and produce an output.

If the V extension register file is not implemented, only Form D *ace.exec* is supported and *ace.input*, *ace.output* are used to provide inputs and extract outputs from *ace.exec*, as described in [Using *ace.input* and *ace.output* to Replace Vector Inputs and Outputs](#).

The instruction may possibly modify state.

The instruction is usage-controlled.

Included in

Extension	Minimum version	Lifecycle state
z1v (Forms A-D)	v0.0.0	--
z1io (Form D)	v0.0.0	--

1.4.5. *ace.mv*

The *z1mv* subextension adds dedicated move instructions from/to scalar/vector registers to a CR.

The instructions in this extension are encoded as new forms to *ace.exec*.

Mnemonics

The *ace.mv* instructions admits following variants:

1. *ace.mv* *Kd* | *K{Xd}*, *Xs2* copy the contents of a GPR into a CR.
2. *ace.mv* *Kd* | *K{Xd}*, *Vs2* copy the contents of a vector register into a CR.
3. *ace.mv* *Xd*, *Ks1* | *K{Xs1}* extract contents from a CR into a GPR.
4. *ace.mv* *Vd*, *Ks1* | *K{Xs1}* extract contents from a CR into a vector register.

Encodings The hart adds the following encodings to forms B and C of *ace.exec*:

1. `ace.mv Kd|K{Xd}, Xs2.`
 - Encoded using `Form = 0b01` and `rs1 = 0b00001`. `rs2` specifies the source GPR, and `rd` specifies the destination CR.
2. `ace.mv Kd|K{Xd}, Vs2. Form = 0b01 and rs1 = 0b00010.`
 - Encoded using `Form = 0b01` and `rs1 = 0b00010`. `rs2` specifies the source vector register, and `rd` specifies the destination CR.
3. `ace.mv Xd, Ks1|K{Xs1}. Form = 0b10 and rs2 = 0b00001.`
 - Encoded using `Form = 0b10` and `rs2 = 0b00001`. `rs1` specifies the source CR, and `rd` specifies the destination GPR.
4. `ace.mv Vd, Ks1|K{Xs1}. Form = 0b10 and rs2 = 0b00010.`
 - Encoded using `Form = 0b10` and `rs2 = 0b00010`. `rs1` specifies the source CR, and `rd` specifies the destination vector register.

Description

- If `Form = 0b10` and `rs2 = 0b00001`:
the contents of `x[s1]` are moved to the CR starting at offset `acestart`.
`acestart` is updated with $(XLEN/8) + acestart$
The instruction is only valid if the `ConfigStatus` field of the metadata of `ks` is set to `CfgStProvisioning` OR `CfgStImporting`.
Otherwise, an illegal instruction exception is raised.
- If `Form = 0b10` and `rs2 = 0b00010`:
`v1` elements in `v[s1]` are moved to CR starting at offset `acestart`.
`acestart` is updated with $v1 * (EEW/8) + acestart$.
The instruction is only valid if the `ConfigStatus` field of the metadata of `ks` is set to `CfgStProvisioning` OR `CfgStImporting`.
Otherwise, an illegal instruction exception is raised.
An illegal instruction exception is also raised if `V` is not enabled.
- If `Form = 0b01` and `rs1 = 0b00001`:
 $XLEN/8$ bytes at offset `acestart` are moved to `x[s2]`.
`acestart` is updated with $(XLEN/8) + acestart$.
The instruction is only valid if the `ConfigStatus` field of the metadata of `ks` is set to `CfgStExporting`.
Otherwise, an illegal instruction exception is raised.
- If `Form = 0b01` and `rs1 = 0b00010`:
`v1` elements in CR at offset `acestart` are moved to `v[s2]`. `acestart` is updated with $v1 * (EEW/8) + acestart$.
The instruction is only valid if the `ConfigStatus` field of the metadata of `ks` is set to `CfgStExporting`.
Otherwise, an illegal instruction exception is raised.
An illegal instruction exception is also raised if `V` is not enabled.

Included in

Extension	Minimum version	Lifecycle state
Zlmv	v0.0.0	--

1.4.6. ace.setst

Synopsis Change the state of the algorithm encoded in the CC in a CR. *Mnemonic* The ace.setst instruction admits three Forms, namely

1. ace.setst Kd|K{Xd}, #immed7 with no auxiliary input.
2. ace.setst Kd|K{Xd}, #immed7, Xs with an auxiliary input in a GPR.
3. ace.setst Kd|K{Xd}, #immed7, Vs with an auxiliary input in a vector register.

Encoding ace.setst is an R-type instruction.

[wavedrom, ,svg]

```
{reg:[
  { bits: 7, name: 0x5b, attr: ['custom-2'] },
  { bits: 5, name: 'Kd|K{Xd}' },
  { bits: 3, name: 'ace.setst', attr:
    ['0 0 1', '0 1 0', '0 1 1'] },
  { bits: 5, name: 0x0 },
  { bits: 5, name: 'rs1', attr: ["no aux input", 'Xs', 'Vs'] },
  { bits: 7, name: 'immed7' },
]}
```

The encodings of the three Forms are:

1. ace.setst = 0b001. rs1 must be 0b00000.
2. ace.setst = 0b010.
3. ace.setst = 0b011.

Description

The instruction changes the *State* and *StateExtension* fields of the MDH of Kd or K{Xd}. It is used for two purposes:

1. Advance from one stage of an algorithm to another (for instance, from initialization of a hash function to data absorption and then to digest finalization); and
2. Set specific state information of an algorithm (such as configure elliptic curve points or private/public keys), which is intended to be orthogonal to the states mentioned in the previous point.

#immed7 is a constant from a set of admissible values.

These values include both general values, which apply to all Algorithms, as well as values specific to the Algorithm encoded in a CR.

❗ If it is decided to encode ace.mgmt by re-using ace.setst with value of #immed7 at least 96 (so, adding 96 to the values in [Values \(decimal\) of #immed7 to be](#)

used for provisioning, importing, and exporting), then in the description of `ace.setst` we must add that the valid values of `#immed7` are in the range 0 to 95.

Certain state values may require an auxiliary input, which may be passed in a GPR `xs` (a GPR pair in RV32) or a vector (group) `vs`. The vector (group) may be replaced by `ACEIOBUF` if `z1io` is implemented.

Normally, `#immed7` contains the *State* value to be set. In most cases, after the instruction has been issued, the *State* field takes the value specified in `#immed7`. Some states may transition to a different state automatically after completing `ace.setst`.

The allowed state transitions are described in [State and StateExtension Fields](#) and in the algorithms.

Note that:

- It is allowed to issue `ace.setst` instructions to the current state.
When the algorithm needs to prevent this, it will automatically transition to a different state after completing `ace.setst`.
- If `#immed7` is zero, i.e., the value corresponding to the *Off* state, the CR is cleared.
- It is always allowed to use an Error State (see [Global CC State Numbers: Error States](#)) for `#immed7`.
This will also invalidate the contents of the CR. This may trigger a partial or full release of the resources used by the CR.
- Setting an Error State does *not* trigger an exception, even if reaching that state implicitly would normally do.
Further usage of the same CR—for instance, a later `ace.exec` or `ace.state` to a valid state—will raise exceptions in that case.

Certain values of `#immed7`, depending on the Algorithm, may be used to set change *StateExtension* instead, so that it can be used as state information that is somewhat orthogonal to the *State* field.

The full provisioning, export, and import operations are described in detail in [Management of Sealed Cryptographic Contexts](#).

The instruction may possibly modify state.

The instruction is usage-controlled.

Included in

Extension	Minimum version	Lifecycle state
Zlv	v0.0.0	--
Zlio	v0.0.0	--

1.4.7. ace.mgmt

! We can encode this instruction also by re-using `ace.setst` and with the value of `#immed7` at least 96 (so, add 96 to the values in [Values \(decimal\) of #immed7 to be used for provisioning, importing, and exporting](#)).

Synopsis Tracks the states of management operations of a CR. *Mnemonic* The `ace.mgmt` instruction admits three Forms, namely

1. `ace.mgmt Kd|K{Xd}, #immed7` with no auxiliary input.
2. `ace.mgmt Kd|K{Xd}, #immed7, Xs` with an auxiliary input in a GPR.
3. `ace.mgmt Kd|K{Xd}, #immed7, Vs` with an auxiliary input in a vector register.

Encoding `ace.mgmt` is an R-type instruction. Its encoding is identical to that of `ace.setst` except for the most significant bit of `funct3`, which is set to 1.

[wavedrom, svg]

```
{reg:[
  { bits: 7, name: 0x5b, attr: ['custom-2'] },
  { bits: 5, name: 'Kd|K{Xd}' },
  { bits: 3, name: 'ace.mgmt', attr:
    ['1 0 1', '1 1 0', '1 1 1'] },
  { bits: 5, name: 0x0 },
  { bits: 5, name: 'rs1', attr: ["no aux input", 'Xs', 'Vs'] },
  { bits: 7, name: 'immed7' },
]}
```

The encodings of the three Forms are:

1. `ace.mgmt = 0b101`. `rs1` must be `0b00000`.
2. `ace.mgmt = 0b110`.
3. `ace.mgmt = 0b111`.

Description

The instruction changes the *ConfigStatus* field of the MDH of `Kd` or `K{Xd}`.

It is used to initiate and complete provisioning or export/import operations.

The special values of `#immed7` to initiate a provisioning, importing, and exporting process, and conclude it afterwards, are listed in the following table:

Values (decimal) of #immed7 to be used for provisioning, importing, and exporting

Value	Mnemonic	Description
0	<code>ace_CR_provision_start</code>	Prepare a CR for provisioning keys and metadata into it, set <i>ConfigStatus</i> to <i>CfgStProvisioning</i> . Needs an auxiliary input in a GPR (or a GPR pair) with bits [63:0] of MDH, or an auxiliary input into a 128-bit (or longer) vector register to provide the entire MDH.

Value	Mnemonic	Description
1	ace_CR_provision_end	Make the provisioned CR available to software (“sealing”), set <i>ConfigStatus</i> to <i>CfgStComplete</i> . Requires no auxiliary input.
2	ace_CR_import_start	Prepare a CR for importing a SCC into it, set <i>ConfigStatus</i> to <i>CfgStimporting</i> . Needs an auxiliary input in a GPR (or a GPR pair) with bits [63:0] of MDH, or an auxiliary input into a 128-bit (or longer) vector register to provide the entire MDH.
3	ace_CR_import_end	Decrypt and authenticate the imported SCC, make the CR available to software, set <i>ConfigStatus</i> to a provided value.
4	ace_CR_export_start	Prepare a CR for exporting it as a SCC (if not yet in the correct format), set <i>ConfigStatus</i> to <i>CfgExporting</i> . Requires no auxiliary input.
5	ace_CR_export_end	Restore the exported CR in the internal clear format (only if already complete) and make it available to software, set <i>ConfigStatus</i> to a provided value. Uses an auxiliary input in a GPR (or a GPR pair) with bits [63:0] of MDH to restore <i>ConfigStatus</i> saved by software. If the restored value of <i>ConfigStatus</i> is not <i>CfgStComplete</i> , the CR is made available to software.

The semantics of `ace.mgmt` are as follows:

1. **`ace.mgmt` is used to start provisioning, resp., import.**

Before issuing `ace.mgmt`, software must read Bits [63:0] of the MDH from memory if the GPR variant is used, or [127:0] of the MDH from memory if the vector variant is used.

Let `m1` be this value.

Software must pass `m1` as the auxiliary input to `ace.mgmt` in a GPR (or a GPR pair), or a vector register.

`ace.mgmt` then performs following operations:

1. The target CR is cleared.
2. If the combination of *Algorithm*, *AlgorithmPolicy*, and *SCProtection* in `m1` is not supported, the CR transitions to Error State `ace_state_not_impl`, and exception `ace_exc_invalid` is raised as defined in the Privileged Architecture if the latter is implemented.
3. In other cases here where the Metadata is invalid, such as, for instance, a nonzero *State* or reserved field: the CR transitions to Error State `ace_state_invalid`, and

exception `ace_exc_invalid` is raised as defined in the Privileged Architecture if the latter is implemented.

4. The implementations will try to allocate CRF capacity according to the requirements of the *Algorithm*, *AlgorithmPolicy*, and *SCProtection* fields of the MDH. No other field shall affect the amount of capacity required. If capacity allocation is not possible, the CR is transitioned to Error State `ace_state_out_of_mem`, and exception `ace_exc_out_of_mem` is raised as defined in the Privileged Architecture if the latter is implemented.
5. Except for Bits [63:0] of the MDH in the CR, which are set to `m1`, the internal CRF memory corresponding to the target CR is zeroed.
6. *ConfigStatus* is set to *CfgStProvisioning*, resp., *CfgStace_CR_import*. In the latter case, we assume that software has saved the old value of `m1` in its state.
7. `ace.start` is set to 8 if the scalar variant was used, and 16 if the vector variant was used.
8. **`ace.mgmt` is used to terminate a provisioning process.**
 1. The ACE unit performs any necessary readjustment of the CR's content to turn it into the format internally consumed by the ACE Unit.
 2. *ConfigStatus* is set to *CfgStComplete*.
 3. `ace.start` is set to 0.

9. **`ace.mgmt` is used to terminate an import process.**

The software must use Form B of `ace.mgmt` to contextually pass the saved value of `m1`.

1. If `m1.ConfigStatus = CfgStComplete` (we are importing the export of a fully configured CR)
 1. The ACE unit decrypts and authenticates the CR's content.
 2. If authentication fails, the target CR is transitioned to Error State `ace_state_CR_import_auth`. No exception is raised.
 3. The ACE unit performs any necessary readjustment of the CR's content to turn it into the format internally consumed by the ACE Unit.
 4. *ConfigStatus* is set to *CfgStComplete*.
2. `ace.start` is set to 0.
3. If `m1.ConfigStatus ≠ CfgStComplete` (we are either importing the export of a not fully configured CR or a nested CR export):

ConfigStatus is set to `m1.ConfigStatus`.

1. **`ace.mgmt` is used to start an export process.**

Let `m1` ← Bits [63:0] of the MDH of the CR.

Software must save this value in its state to use it when terminating the export process.

Then:

1. If `m1.ConfigStatus = CfgStComplete`:

Prepare a CR to produce a SCC. This includes encrypting its contents and computing the authentication tag(s).

1. If `m1.ConfigStatus` $\neq$ `CfgStComplete` (we are exporting a not fully configured CR or we are initiating a nested CR export):

No encryption or authentication occurs, the information in the CR is exported verbatim, in the same order as they appear in the PI in case the SCC is the export of a partially provisioned CR, or the SCC in case the latter is the export of a partially provisioned, resp., imported, CR, or the CR was previously prepared for export and then the export process itself was interrupted.

1. `ace.start` is set to 8 if the scalar variant was used, and 16 if the vector variant was used..
2. **`ace.mgmt` is used to terminate an export process.**

Let `m1` be the value of `MDH[63:0]` saved by the software upon initiation of the export process.

Software must pass `m1` as the auxiliary parameter to `ace.mgmt` for export termination.

Then:

1. If `m1.ConfigStatus` = `CfgStComplete`:

The implementation must restore the state of the CR to the state it was in when the export process was initiated.

This may involve decrypting its contents or restoring them from an internal backup.

1. If `m1.ConfigStatus` $\neq$ `CfgStComplete`:

No changes to the CR state are made.

1. `ace.start` is set to 0.

Whenever `ace.mgmt`, fails for any other reason, the target CR is cleared.

The full provisioning, export, and import operations are described in detail in [Management of Sealed Cryptographic Contexts](#).

The instruction may possibly modify state.

The instruction is not usage-controlled.

(Z1mv only) When a CR's metadata `ConfigStatus` field is changed from `CfgStImporting` to `CfgStComplete`, the ACE unit verifies the signature of the CR's contents.

(Z1mv only) When a CR's metadata `ConfigStatus` field is changed from `CfgStComplete` to `CfgStExporting`, the ACE unit generates the SCC.

Included in

Extension	Minimum version	Lifecycle state
Z1v	v0.0.0	--
Z1io	v0.0.0	--

1.4.8. `ace.getmdv/ace.getmdl/ace.getmdh`

Synopsis Export the MDH of a CR. *Mnemonics*

```
ace.getmdl Xd, Ks|K{Xs}
ace.getmdh Xd, Ks|K{Xs}
ace.getmdv Vd, Ks|K{Xs}
```

Encoding

`ace.getmd{1,h}` is an R-type instruction and its encoding shares opcode and `func3` with `ace.exec`, but `func2` is set to `0b11`.

[wavedrom, , svg]

```
{reg:[
  { bits: 7, name: 0x5b, attr: ['custom-2'] },
  { bits: 5, name: 'Xd' },
  { bits: 3, name: 0x0, attr: ['ace.exec'] },
  { bits: 5, name: 'rs1', attr: ['Ks|K{Xs}'] },
  { bits: 5, name: 0x00 },
  { bits: 1, name: 0x0 },
  { bits: 1, name: 'r' },
  { bits: 1, name: 0x0 },
  { bits: 1, name: 'h' },
  { bits: 1, name: 'v' },
  { bits: 2, name: 0x3, attr: ['func2'] },
]}
```

Bit `r` (bit 26) is 0 when the context is an immediate and 1 when it is held in a GPR (scalar integer).

If bit `h` (bit 28) is 0, the instruction is `ace.getmdl`; if `h` is 1, it is `ace.getmdh`.

If bit `v` (bit 29) is 0, the result is written to a GPR (GPR pair for RV32).

If bit `v` is 1, the entire MDH is written to the least significant 128 bits of a vector register (group).

In this case, bit `h` must be 0.

Description

On RV64:

- `ace.getmdl` copies bits [63:0] of the MDH of CR `Ks` or `K{Xs}` into GPR `xd`.
- `ace.getmdh` bits [127:64] into GPR `xd`.
- `ace.getmdv` (vector register output version) bits [127:0] into vector register `xv`. This version of the instruction only supports vector register (groups) of 128 bits or longer. If used with a vector register (group) smaller than 128 bits, an illegal instruction exception is raised.

On RV32, if the destinations register is a GPR, `d` must be even, and:

- `ace.getmdl` copies bits [31:0] of the MDH of CR κ_s or $\kappa\{Xs\}$ into GPR x_d and bits [63:32] of the same CR into GPR $x_{[d+1]}$, atomically.
- `ace.getmdh` copies bits [95:64] of the MDH of CR κ_s or $\kappa\{Xs\}$ into GPR x_d and bits [127:96] of the same CR into GPR $x_{[d+1]}$, atomically.
- The semantics of `ace.getmd` are the same as on RV64.

The instruction may not modify (ACE) state.

The instruction is not usage-controlled.

Included in

Extension	Minimum version	Lifecycle state
Zlv: <code>ace.getmdv/ace.getmdl/ace.getmdh</code>	v0.0.0	--
Zlio: <code>ace.getmdl/ace.getmdh</code>	v0.0.0	--

1.4.9. `ace.restrictv/ace.restrictl/ace.restricth`

Synopsis Restrict the policies encoded in a CC in a CR. *Mnemonic*

```
ace.restrictl  Kd|K{Xd}, Xs1
ace.restricth  Kd|K{Xd}, Xs1
ace.restrictv  Kd|K{Xd}, Vs1
```

Encoding

`ace.restrict` is an R-type instruction.

The encoding shares opcode and `func3` with `ace.exec`, but `func2` (i.e., the two msbs of `func7`) is `0b01`.

If bit `h` (bit 28) is 0, the instruction is `ace.restrictl`; if `h` is 1, it is `ace.restricth`.

If bit `v` (bit 29) is 0, the input is taken from a GPR (GPR pair for RV32), and the instruction is `ace.restrictv`.

If bit `v` is 1, the input is taken from the least significant 128 bits of a vector register (group).

In this case, bit `h` must be 0.

[wavedrom, , svg]

```
{reg:[
  { bits: 7, name: 0x5b, attr: ['custom-2'] },
  { bits: 5, name: 'Kd|K{Xd}' },
  { bits: 3, name: 0x0, attr: ['ace.exec'] },
  { bits: 5, name: 0x0 },
  { bits: 5, name: 'Xs1' },
  { bits: 1, name: 0x0 },
  { bits: 1, name: 'r' },
  { bits: 1, name: 0x0 },
  { bits: 1, name: 'h' },
```

```
{ bits: 1, name: 'v' },
{ bits: 2, name: 0x1, attr: ['func2'] },
}]}
```

Bit *r* (i.e., bit 26) takes the value 0 if the context is encoded as an immediate in Bits [11:7], and it takes the value 1 if a GPR (also encoded in [11:7]) is used to reference it.

Description

`ace.restrictl`/`ace.restrictl` can only be used if the CR's state is *Initial*.

Using the instruction in any other state raises an invalid instruction exception.

`ace.restrictl` modifies the field *AlgorithmPolicy* in bits [63:0] of the MDH of the CR.

`ace.restrictl` modifies fields *Locality*, *UsagePolicy*, *ExpirationDate*, and possibly *AlgorithmUse* in bits [127:64] of the MDH of the CR.

Modifying the *Locality* field can be used to add a Locality but also to *replace* a Locality with a stricter one, such as

- *SiPScrt* → *ChipFamScrt*, *ChipScrt*; *ChipFamScrt* → *ChipScrt*;
- *OEMScrt* → *ProdScrt*, *DevScrt*; *ProdScrt* → *DevScrt*.
- *MLocality* → *HLocality*, *SLocality*; and *HLocality* → *SLocality*.

Generalizing Localities is not permitted: Attempting to do so raises exception

`ace_exc_invalid`, but the

State is not automatically set to *ace_state_invalid* (the exception handler may of course do that).

Once the *Locality* field has been modified,

when the CR is exported, it will use the new Locality values to tweak the export algorithm.

Let MDH be the MDH of the CC in the destination CR.

On RV64, the semantics of the instructions are:

- For `ace.restrictl`, *xs* follows the format of bits [63:0] of the MDH, and for `ace.restrictl`, *xs* follows the format of bits [127:64] of the MDH.

Only the fields *AlgorithmPolicy*, *Locality*, *UsagePolicy*, *ExpirationDate* and *AlgorithmUse* of *xs* may be defined.

All other fields must be 0, else exception `ace_exc_invalid` is raised.

- *xs.AlgorithmPolicy* is used to set the value of MDH.*AlgorithmPolicy* of the CR.
 - If the algorithm configured in the CR does not make use of the field *xs.AlgorithmPolicy*, then *xs.AlgorithmPolicy* must be zero, else exception `ace_exc_invalid` is raised.
 - If *xs.AlgorithmPolicy* is used to encode an extension of *xs.Algorithm*, then *xs.AlgorithmPolicy* must be zero, else exception `ace_exc_invalid` is raised.

- No currently disabled operation may be (re)enabled. Any attempt to do so raises exception `ace_exc_invalid`. In some cases, a zero value is not permitted, such as disabling both encryption and decryption.
- If the CR's current state is *only* allowed while performing an operation currently permitted by `MDH.AlgorithmPolicy`, a change to *AlgorithmPolicy* that would disable that operation is still possible.
However, this change will cause exception `ace_exc_invalid` to be raised at the next usage operation on the CR.
- If `xs.Locality` is non-zero, then it is the new value of `MDH.Locality`, subject to the Locality replacement rules above.
- A zero in any of bits `xs.UsagePolicy` means that the corresponding policy is *not* changed.
- A one in any of the bits `xs.UsagePolicy` means that usage by the corresponding mode is not allowed, and the corresponding bit in the Metadata of the destination CC is set to 1.
- If `xs.ExpirationDate` is zero, the corresponding policy in `MDH.ExpirationDate` is *not* changed.
- A non-zero value in `xs.ExpirationDate` will be copied to `MDH.ExpirationDate`, but only if that policy was not already programmed before or the original value of `MDH.ExpirationDate` was equal to the new value or larger. Otherwise exception `ace_exc_invalid` is raised.
- Any change to `MDH.AlgorithmUse` is subject to policies set by the Algorithm in the CR.

On RV32, the only difference is:

- `s` must be even, the register pair `x[s]` and `x[s+1]` is read atomically, and `(x[s+1] @ x[s])` replaces `xs` in the RV64 description.

On both RV32 and RV64, the vector source register version `ace...restrictv` of the instruction modifies the entire MDH, where the 128-bit input has the same format as the MDH.

The instruction may possibly modify state.

The instruction is usage-controlled.

Included in

Extension	Minimum version	Lifecycle state
Zlv: <code>ace.restrictv/ace.restrictl/ace.restricth</code>	v0.0.0	--
Zlio <code>ace.restrictl/ace.restricth</code>	v0.0.0	--

i This instruction enables several practical use cases:

- Enabling side-channel protection when supported by the microarchitecture, even if the original CC was not configured with it.
- Allowing privileged code to give a CC to several processes in a fine-grained manner. This can be achieved by cloning the CR and customizing the Localities for each clone.
- Similarly, enabling privileged code to split a CC capable of both encryption and decryption into two CCs, one only capable of encryption and the other only capable of decryption, thereby creating a one-way channel between processes.

The ability to perform such customizations saves both time and computational resources by avoiding additional round-trips to trusted environments.

1.4.10. `ace.clone`

Synopsis Copy a CR into another CR. *Mnemonic* `ace.clone Kd|K{Xd}, Ks|K{Xs}`

Encoding The encoding shares opcode and `func3` with `ace.exec`, however `func2` is 2.

The instruction is thus an R-type instruction. [wavedrom, , svg]

```
{reg:[
  { bits: 7, name: 0x5b, attr: ['custom-2'] },
  { bits: 5, name: 'Kd|K{Xd}' },
  { bits: 3, name: 0x0, attr: ['ace.exec'] },
  { bits: 5, name: 'Ks|K{Xs}' },
  { bits: 5, name: 0x0 },
  { bits: 2, name: 'R' },
  { bits: 3, name: 0x0 },
  { bits: 2, name: 0x2, attr: ['func2'] },
]}
```

+

Bits [26:25], i.e., field `R`, take the following values depending on how the CRs are given:

+

1. `00` if the source and the output CRs are both encoded as immediate values.
2. `01` if the source CR is encoded as immediate and the output CR is selected using a GPR.
3. `10` if the source CR is selected using a GPR and the output CR is encoded as an immediate value.
4. `11` if the source and the output CRs are both selected using the values of GPRs.

Description Clones source CR κ_s (or the CR indexed by x_s) into destination CR κ_d (or the CR indexed by x_d). A CR whose *ConfigStatus* is not *CfgStComplete* cannot be cloned.

The instruction may possibly modify state.

The instruction is not usage-controlled.

Included in

Extension	Minimum version	Lifecycle state
Zlv	v0.0.0	--
Zlio	v0.0.0	--

1.4.11. ace.derive

i This instruction is not yet used by any algorithm. The encoding is reserved for future use.

Synopsis Derive a CR from a source CR, such as a key derivation. *Mnemonic* The `ace.derive` instruction admits three Forms, namely

1. `ace.derive Kd|K{Xd}, Ks1|K{Xs1}` with no auxiliary input.
2. `ace.derive Kd|K{Xd}, Ks1|K{Xs1}, Xs2` with an auxiliary input in a GPR.
3. `ace.derive Kd|K{Xd}, Ks1|K{Xs1}, Vs2` with an auxiliary input in a vector register.

Encoding The encoding differs from `ace.clone` in that bit 27 is set to 1. The instruction is thus an R-type instruction. [wavedrom, , svg]

```
{reg:[
  { bits: 7, name: 0x5b, attr: ['custom-2'] },
  { bits: 5, name: 'Kd|K{Xd}' },
  { bits: 3, name: 0x0, attr: ['ace.exec'] },
  { bits: 5, name: 'Ks1|K{Xs1}' },
  { bits: 5, name: 'Xs2|Vs2' },
  { bits: 2, name: 'R' },
  { bits: 1, name: 0x1 },
  { bits: 2, name: 'Form' },
  { bits: 2, name: 0x2, attr: ['func2'] },
]}
```

+

Bits [26:25], i.e., field `R`, are encoded exactly as for `ace.clone` ([ace.clone](#)).

+

Bits [29:28], i.e., field `Form`, deals with the optional auxiliary input:

+

1. `00` the instruction admits no auxiliary input.
2. `01` the instruction accepts an auxiliary input in a GPR.
3. `10` the instruction accepts an auxiliary input in a vector register.
4. `11` is reserved for future use.

Description

Derive a new context from an existing one, with its own opaque key material, for instance using a key derivation mechanism.

The behavior of the instruction is not expected to be deterministic.

It can be used multiple times, to generate multiple derived contexts.

The instruction may possibly modify state.

The instruction is usage-controlled.

Included in

Extension	Minimum version	Lifecycle state
Zlv	v0.0.0	--
Zlio	v0.0.0	--

1.4.12. ace.size

Synopsis Return the size of a PI or SCC in bytes, or zero in case of error. *Mnemonics*

There are two Forms of the `ace.size` instruction:

1. `ace.size Xd, Ks|K{Xs}` - Size from a context.
2. `ace.size Xd, Xs|Vs` - Size from MDH.

Encoding

The encoding shares opcode, func3, and func2 (the two msbs of func7) with `ace.exec`, but with bit 27 equal to 1.

[wavedrom, , svg]

```
{reg:[
  { bits: 7, name: 0x5b, attr: ['custom-2'] },
  { bits: 5, name: 'Xd' },
  { bits: 3, name: 0x0, attr: ['ace.exec'] },
  { bits: 5, name: 'Ks/K{Xs}/Xs/Vs' },
  { bits: 5, name: 0x0 },
  { bits: 1, name: 0x0 },
  { bits: 1, name: 'r' },
  { bits: 1, name: 0x1 },
  { bits: 2, name: 'F' },
  { bits: 2, name: 0x0, attr: ['func2'] },
]}
```

The `F` field and the `r` bit may assume the following values:

- `F = 0b00, r = 0` □ Form A: Size from a context given as `Ks`.
- `F = 0b00, r = 1` □ Form A: Size from a context given indirectly as `K{Xs}`.
- `F = 0b01, r = 0` □ Form B: Size from MDH.
- `F = 0b10, r = 0` □ Form C: Size from MDH using vector.

Description

- Form A `ace.size` returns the total size in bytes of the memory buffer that would be necessary to store the exported SCC, including the authentication tag(s) and the additional variable-length data.
Knowing this size is necessary for any feature that relies on `ace.export`, including context switching. The instruction returns 0 if the CR is unconfigured.
- On RV64, Form B `ace.size` returns the total size in bytes of an SCC having Bits [63:0] of the MDH given in `xs` into GPR `xd` including the additional variable-length data. If the algorithm is not supported or the Metadata is invalid, the instruction returns 0.
- On RV32, Form B `ace.size` differs from the RV64 version in that `s` must be even, and Bits [63:0] of the MDH are passed in `x[s+1] @ xs`,
- On systems that implement Zv1128b, Zv1256b, Zv1512b, or Zv11024b, Form C of `ace.size` returns the total size in bytes of an SCC having Bits [127:0] of the MDH given in `vs` into GPR `xd` including the additional variable-length data. If the algorithm is not supported or the Metadata is invalid, the instruction returns 0. Otherwise, Form C raises an illegal instruction exception.

The instruction may not modify state.

The instruction is not usage-controlled.

Included in

Extension	Minimum version	Lifecycle state
Zlv	v0.0.0	--
Zlio	v0.0.0	--

1.4.13. ace.input

Synopsis Load data into the ACEIOBUF. *Mnemonic* `ace.input %offset(Xs), Xl`

Encoding `ace.input` is an S-type instruction. [\[wavedrom, , svg\]](#)

```
{reg:[
  { bits: 7, name: 0x27, attr: ['custom-1'] },
  { bits: 5, name: 'immed[4:0]' },
  { bits: 3, name: 0x2, attr: ['ace.input'] },
  { bits: 5, name: 'Xl' },
  { bits: 5, name: 'Xs' },
  { bits: 7, name: 'immed[11:5]' },
]}
```

Description

If the vector registers are not sufficiently long, or not implemented, inputs can be read directly from memory and outputs written directly to memory.

For this reason, the architecture defines `ace.input` and `ace.output` (cf. next section). `ace.input` loads `x1` bytes from memory starting at address `%offset(Xs)` and copies them to the ACEIOBUF, starting at offset `acestart`.

If `x1 = 0`, the operation does nothing.

If `x1` is greater than `aceiobuftop - acestart`, exception `ace_exc_invalid` is raised.

The intended usage of `ace.input` and `ace.output` is explained in detail in [Using `ace.input` and `ace.output` to Replace Vector Inputs and Outputs](#).

`ace.input` may be used only if the ACEIOBUF is enabled and `aceiobufen` is non-zero. Otherwise, exception `ace_exc_buf_unconf` is raised.

The instruction may possibly modify state.

The instruction is not usage-controlled.

Included in

Extension	Minimum version	Lifecycle state
Zlio	v0.0.0	--

1.4.14. `ace.output`

Synopsis Save the contents of the ACEIOBUF to memory. *Mnemonic* `ace.output`
`%offset(Xd), x1`

Encoding

`ace.output` is an I-type instruction.

[wavedrom, , svg]

```
{reg:[
  { bits: 7, name: 0x27, attr: ['custom-1'] },
  { bits: 5, name: 'Xd' },
  { bits: 3, name: 0x3, attr: ['ace.output'] },
  { bits: 5, name: 'X1' },
  { bits: 12, name: 'immed' },
]}
```

Description

This is the output operation corresponding to `ace.input`.

`ace.output` writes the contents of the input buffer, starting at offset `acestart`, to memory starting at address `%offset(Xs)`.

If `x1 = 0`, the operation does nothing.

If `x1` is greater than `aceiobuftop - acestart`, exception `ace_exc_invalid` is raised.

`ace.output` may be used only if the ACEIOBUF is enabled and `aceiobuflen` is non-zero. Otherwise, exception `ace_exc_buf_unconf` is raised.

The instruction may possibly modify state.

The instruction is not usage-controlled.

Included in

Extension	Minimum version	Lifecycle state
Zlio	v0.0.0	--

1.5. Pseudo-Instructions

1.5.1. `ace.clear/ace.reset`

Synopsis Clears and unconfigures a CR, or the ACE state. *Mnemonic* `ace.clear Kd|K{Xd}`

`ace.reset`

Encoding Pseudo-instructions `ace.clear/ace.reset` are implemented as `ace.setst` (see [ace.setst](#)) with no auxiliary operands and the immediate for *State* set to `ace_state_off` (as defined in [State and StateExtension Fields](#)). *Description*

`ace.clear` unconfigures the state of the CR `Kd` or `K{Xd}` and releases its resources in the CRF.

`ace.reset` unconfigures all the CRs and the ACEIOBUF.

It uses the encoding of `ace.setst` described above with indirect CR addressing and `xd = x0`.

Clearing an *Off* CR does not change dirtiness flags.

The instruction is not usage-controlled.

Included in

Extension	Minimum version	Lifecycle state
Zlv	v0.0.0	--
Zlio	v0.0.0	--

1.5.2. `ace.getst`

Synopsis Returns the *State* field of the metadata of the source CR.

This is useful to determine whether a CR is in an Error State, as defined in [State and StateExtension Fields](#). *Mnemonic* `ace.getst Xd, Ks|K{Xs}`

Encoding The operations is a pseudo-instruction. *Description* `ace.getst` is implemented as a read of the source CR's metadata using `ace.getmdl`

followed by instructions to extract the *State* field of the latter.

For instance, on RV64, we can use:

1. `ace.getmdl Xd, Ks OR K{Xs}`
2. `srlrli Xd, Xd, 20`
3. `andi Xd, Xd, 0x1F`

1.5.3. `ace.avail`

Synopsis Verify whether an algorithm is supported, or the first 64 bits of a metadata block are valid. *Mnemonic* `ace.avail Xd, Xs`

Encoding `ace.avail` is an alias to Form B of `ace.size`, as the latter returns 0 in case of error. *Included in*

Extension	Minimum version	Lifecycle state
Zlv	v0.0.0	--
Zlio	v0.0.0	--

1.6. Using `ace.input` and `ace.output` to Replace Vector Inputs and Outputs

When Zlio is available, some instructions that access vector inputs and outputs may be replaced by instruction sequences without vector inputs and outputs as described in [Alternative sequences for `ace.exec`, `ace.setst`, `ace.mgmt`, and `ace.derive` using `ace.input` and `ace.output`](#).

Alternative sequences for `ace.exec`, `ace.setst`, `ace.mgmt`, and `ace.derive` using `ace.input` and `ace.output`

Instruction and Form	Original Instruction	Alternative Sequence
<code>ace.exec</code> , Form A	<code>ace.exec Vd, Kn K{Xn}, Vs</code>	1: <code>ace.input X1, %offset(Xs)</code> 2: <code>ace.exec Kn K{Xn}</code> 3: <code>ace.output %offset(Xd), X1</code>
<code>ace.exec</code> , Form B	<code>ace.exec Kn K{Xn}, Vs</code>	1: <code>ace.input %offset(Xs), X1</code> 2: <code>ace.exec Kn K{Xn}</code>
<code>ace.exec</code> , Form C	<code>ace.exec Vd, Kn K{Xn}</code>	1: <code>ace.exec Kn K{Xn}</code> 2: <code>ace.output %offset(Xd), X1</code>
<code>ace.setst</code> , Form C	<code>ace.setst Kd K{Xd}, #immed, Vs</code>	1: <code>ace.input %offset(Xs), X1</code> 2: <code>ace.setst Kd K{Xd}, #immed</code>
<code>ace.mgmt</code> , Form C	<code>ace.mgmt Kd K{Xd}, #immed, Vs</code>	1: <code>ace.input %offset(Xs), X1</code> 2: <code>ace.mgmt Kd K{Xd}, #immed</code>

Instruction and Form	Original Instruction	Alternative Sequence
ace.derive, Form C	ace.derive Kd K{Xd}, Ks1 K{Xs1}, Vs2	1: ace.input %offset(Xs2), X1 2: ace.derive Kd K{Xd}, Ks1 K{Xs1}

The sole functions of `ace.input` and `ace.output` are to supply inputs to cryptographic operations and to read out their outputs. No computation is performed on the data by these instructions. All computations and internal state changes of `ace.exec`, `ace.setst`, `ace.mgmt`, and `ace.derive` remain within those operations.

The usage and ordering of the alternative sequences are not enforced architecturally. Thus, the caller may omit `ace.input` or `ace.output`, meaning that the contents of the ACEIOBUF are reused, or the output is ignored.

This does not expand the attack surface, since the caller always retains full control over the data passed to ACE instructions and their outputs.

Instead, the ACE architecture defines the following rules:

1. A Form D `ace.exec` can be used whenever a Form of `ace.exec` with an input and/or an output is instead expected, i.e., any of the Forms A, B, or C `ace.exec`. The implementation shall automatically use the ACEIOBUF if `z1io` is available, and raise an invalid instruction exception otherwise.
2. A Form A `ace.exec` cannot be replaced by a Form B `ace.exec`, where the input is provided by a vector register but an output vector register is not defined, expecting that the output be written to the ACEIOBUF.
Any attempt to do so raises an invalid instruction exception
3. A Form A `ace.exec` cannot be replaced by a Form C `ace.exec`, where the output is provided by a vector register but an input vector register is not defined, expecting that the input be read from the ACEIOBUF.
Any attempt to do so raises an invalid instruction exception
4. Similarly, for `ace.setst`, if a vector input is expected (Form C `ace.setst`) and it is not provided, the implementation shall automatically use the ACEIOBUF if `z1io` is available, and raise an invalid instruction exception otherwise. This instruction replacement is not allowed in the case of a Form B `ace.setst`, i.e., when the auxiliary input should come from a GPR, and an invalid instruction exception is raised regardless of whether `z1io` is available.
5. `ace.mgmt`, the same applies as with `ace.setst`, with only `ace.setst` replaced by `ace.mgmt`.
6. A Form A `ace.derive` can be used whenever a Form C `ace.derive` is expected. The implementation shall automatically use the ACEIOBUF if `z1io` is available, and raise an invalid instruction exception otherwise.
7. Form D `ace.exec` in place of its Forms A-C, Form A `ace.setst` in place of its Form C, and Form A `ace.derive` in place of its Form C may be issued only if the ACEIOBUF is enabled and `aceiobufen` is non-zero. Else, exception `ace_exc_buf_unconf` is raised. If `aceiobufen` is non-zero but the range size `aceiobuftop - acesart` is invalid for the algorithm, exception `ace_exc_invalid` is raised and no cryptographic operation is performed.

i It is because of these rules that, in the Algorithms, we write that a certain Form of `ace.exec`, `ace.setst`, `ace.mgmt` is *expected* instead of *required* (or *only [...] allowed*). The above substitutions are implicit in all algorithms and are not restated for each occurrence.

1.7. Interruptibility and Resumability of ACE instructions

`ace.clear`, `ace.reset`, `ace.setst`, `ace.mgmtt`, `ace.getst`, `ace.restrict`, `ace.clone`, `ace.size` and `ace.avail` are uninterruptible.

Long-running ACE operations may proceed even if the hart is preempted. In this case, when the context-switch handler accesses an output of the operation register, different behaviors are allowed:

1. The hart may wait for the operation to finish;
2. the implementation may adopt a precise exception model, whereby the ACE unit is halted and sets a value in a CSR called `acestart` to allow later resumption;
or
3. the system may impose a timeout before halting.

The architecture permits all these approaches: It only specifies the allowed interruption points for its instructions and algorithms.

`ace.input`, `ace.output`, `ace.load` and `ace.store` may be interrupted and resumed.

`acestart` reflects the amount of data read or written before the interrupt.

In this case resumption will start from that point.

The operation of `ace.exec` on a basic processing unit as defined in the algorithm it is performing is uninterruptible, unless explicitly allowed by the algorithm.

Most cryptographic primitives cannot be interrupted; therefore, state information to encode the round number or a round key value is not needed.

For certain very long running operations, such as public key primitives, an algorithm specific interruptibility and resumability behavior is defined.

When processing multiple basic processing unit iteratively, `ace.exec` handles each block atomically, but it may be interrupted between blocks.

On an interrupt during block processing, the implementation may either complete the current block before yielding to the handler, or discard the partial computation and restore the state from before its processing.

`acestart` is updated accordingly to point at the place of resumption, valid values are multiples of the length of the basic processing unit, with the only possible exceptions defined by the algorithm.

While handlers can set `acestart` to zero to force a restart of an `ace.exec` operation, this may lead to unpredictable behavior when the input and output overlap.

1.7.1. Management Operations

This section is informative and non-normative.

The rules defined in `ace.setst` for the initiation and termination of management operations are necessary because any management operation may be interrupted, and there may be multiple nested interrupts or exceptions, with or without context switches.

Leaving the burden of saving and restoring *ConfigStatus* to the interrupted software simplifies the logic and supports arbitrarily many levels of nested virtualization.

These rules also imply that an interrupted import operation may be resumed on a hart attached to a different ACE Unit implementation.

The architecture is designed to ensure interoperability, provided the importing implementation supports the algorithm encoded in the SCC.

However, the implementation must handle cases where the SCC is too large to fit in a CR due to an optional Implementation VDS—or where that VDS is incompatible—by ignoring the optional data. A different set of optional data may be created by the importing implementation. In these cases, *ImpDataLen* is accordingly updated.

Software must ensure that an interrupted configuration or export operation can be resumed, to support binary compatibility across implementations that provide this feature and implementations that do not.

An implementation is not required to support resumption, and even those that do may ignore the `acestart` value and restart the interrupted operation for any reason.

For this reason, we define specific instruction sequences for the management operations, cf [Code for CR Configuration and Export Operations](#).

The architecture is co-designed with the intended use of the instructions.

A context switch handler can force a restart by clearing partially configured or exported CRs at context save and again at context restore.

This must also be handled by the software.

1.7.2. Resumption and Memory Model

❗ To be discussed with ARC. We would like to know whether instructions issuing long loads or stores can prescribe whether they can access the memory in a prescribed order, i.e., the ACE unit can send to the LSU sequences of reads or writes in ascending memory address order, as if these were separate instructions.

RISC-V implementations may read or write long inputs in an arbitrary order.

Enforcing a strict access ordering for ACE inputs and outputs would be complex on certain microarchitectures.

The ACE unit sends commands to the LSU to read and write memory sequentially—analogously to software looping over an array. The LSU may then issue those reads and writes in any order it chooses.

However, we need that the same code must work across all behaviors.

An implementation always has the option to restart an operation instead of resuming it, for any reason, and may ignore and clear `acestart`. The ACE instructions are designed

so that the same instruction sequences work correctly in all such cases and on every ACE implementation.

For example, the instruction that loads or stores a CR's content starting with the 8th byte may always fail, leaving the CR in an *Off* state. This signals that the import, provisioning, or export operation must be restarted. The software must then prepare the CR for the operation again. For import and provisioning, this entails providing the first 8 bytes of the MDH to the `ace.setst` instruction used to prepare the CR anew.

1.8. Code for CR Configuration and Export Operations

This subsection is informative.

We show here how the architecture expects the entire configuration and export processes to be coded.

The code fragments presented here only include the main operation logic together with the detection of any error condition, not the actual error management, which would be platform specific.

While these code fragments are obviously much longer than monolithic instructions, they can be made into libraries or OS APIs, so they do not affect code size in any major way. The use of indirect CR addressing proves itself very useful in this case.

In these examples,

GPR `t6` points to the memory location where a PI or SCC is stored,

and GPR `t0` contains the number of the CR to be configured or exported.

The code also uses `t1`, `t2`, and `t3` for temporary storage.

Provisioning Code (z1mv)

```
ld      t1, 0(t6)           # load MDH bits [63:0] from memory
li      t3, #23
restart:
ace.mgmt K{t0}, t1, #ace_CR_provision_start
                                # start provisioning, load MDH[63:0] in CR
ace.size a2, K{t0}           # Get data length
subi    a2, a2, 8            # Subtract the 8B already loaded of MDH

vloop:
vsetvli a3, a2, e8, m1, ta, ma # load rest of PI starting with
MDH[127:64]
ace.getst t2, K{t0}          # get _State_ field from MDH[63:0]
bltu     t3, t2, handle_errors # errors (larger than 23, i.e. ≥ 24)
beqz     t2, handle_errors    # handle error (is _Off_ needed?)
vle8.v   v4, 8(t6)           # load data
ace.mv   K{t0}, v4           # Move to CR
add      t6, t6, a3          # Bump pointer
sub      a2, a2, a3          # Decrement number of elements
bnez     a2, vloop
ace.getst t2, K{t0}          # get _State_ field from MDH[63:0]
li      t3, #ace_state_off    # compare to off state
beq      t2, t3, restart      # CR cleared means force restart
ace.mgmt K{t0}, #ace_CR_provision_end # terminate provisioning
```



```

    ace.getst    t2, K{t0}           # and possibly reconstructs data
    bltu        t3, t2, handle_errors # get _State_ field from MDH[63:0]
success:        # handle other errors (> 23, i.e. ≥ 24)

```

Provisioning Code (z1m)

```

    ld          t1, 0(t6)           # load MDH bits [63:0] from memory
    li          t3, #23
restart:
    ace.mgmt    K{t0}, t1, #ace_CR_provision_start
                                # start provisioning, load MDH[63:0] in CR
    ace.getst   t2, K{t0}           # get _State_ field from MDH[63:0]
    bltu        t3, t2, handle_errors # errors (larger than 23, i.e. ≥ 24)
    beqz        t2, handle_errors    # handle error (is _Off_ needed?)
    ace.load    K{t0}, 8(t6)         # load rest of PI starting with
MDH[127:64]
    ace.getst   t2, K{t0}           # get _State_ field from MDH[63:0]
    li          t3, #ace_state_off   # compare to off state
    beq         t2, t3, restart      # CR cleared means force restart
    ace.mgmt    K{t0}, #ace_CR_provision_end # terminate provisioning
                                # and possibly reconstructs data
    ace.getst   t2, K{t0}           # get _State_ field from MDH[63:0]
    bltu        t3, t2, handle_errors # handle other errors (> 23, i.e. ≥ 24)
success:

```

Import Code (z1mv)

```

    ld          t1, 0(t6)           # load MDH bits [63:0] from memory
    li          t3, #23
restart:
    ace.mgmt    K{t0}, t1, #ace_CR_import_start
                                # start provisioning, load MDH[63:0] in CR
    ace.size    a2, K{t0}           # Get data length
    subi a2, a2, 8                 # Subtract the 8B already loaded of MDH

vloop:
    vsetvli a3, a2, e8, m1, ta, ma # load rest of PI starting with
MDH[127:64]
    ace.getst   t2, K{t0}           # get _State_ field from MDH[63:0]
    bltu        t3, t2, handle_errors # errors (larger than 23, i.e. ≥ 24)
    beqz        t2, handle_errors    # handle error (is _Off_ needed?)
    vle8.v      v4, 8(t6)           # load data
    ace.mv      K{t0}, v4           # Move to CR
    add         t6, t6, a3           # Bump pointer
    sub         a2, a2, a3           # Decrement number of elements
    bnez        a2, vloop
    ace.getst   t2, K{t0}           # get _State_ field from MDH[63:0]
    li          t3, #ace_state_off   # compare to off state
    beq         t2, t3, restart      # CR cleared means force restart
    ace.mgmt    K{t0}, #ace_CR_import_end # terminate importing
                                # and possibly reconstructs data
    ace.getst   t2, K{t0}           # get _State_ field from MDH[63:0]
    bltu        t3, t2, handle_errors # handle other errors (> 23, i.e. ≥ 24)
success:

```

Import Code (z1m)

```

ld      t1, 0(t6)                # load MDH bits [63:0] from memory
li      t3, #23
restart:
  ace.mgmt K{t0}, t1, #ace_CR_import_start
                                     # prepare CR for import, load MDH[63:0] in
it
  ace.getst t2, K{t0}              # get _State_ field from CR (after
checks!)
  bltu     t3, t2, handle_errors    # errors (larger than 23, i.e., ≥ 24)
  ace.load K{t0}, 8(t6)             # load all the rest starting with
MDH[127:64]
  ace.getst t2, K{t0}              # get _State_ field from MDH[63:0]
  li      t3, #ace_state_off        # compare to off state
  beq     t2, t3, restart           # CR cleared means force restart
  ace.mgmt K{t0}, #ace_CR_import_end # decrypt/authenticate CR, possibly
complete it
  ace.getst t2, K{t0}              # get _State_ field from MDH[63:0]
  bz      t2, handle_errors         # handle _Off_ state (needed ? double
check)
  bltu     t3, t2, handle_errors    # handle other errors (> 23, i.e., ≥ 24)
success:

```

During the provisioning and import procedures, the validity of bits [63:0] of the metadata field is checked immediately.

The validity of bits [127:64] is deferred to sealing, which in the case of import includes authentication.

If these bits are invalid, the CR will be invalidated upon executing `ace.setst K{t0}, #ace_CR_provision_end/ace_CR_import_end`, and so the following `ace.getst` will catch this situation.

- i** Implementations may interleave decryption and authentication with the load operation.
If an import is interrupted and the partial state exported (assuming we allow that),
the exported data must be the original encrypted data loaded so far, and not a re-encryption
of a partially decrypted payload. This is required for both correctness and security.

Export Code (z1mv)

```

ace.size  t2, K{t0}                # get size of the data to export
{...}    # here goes the code that allocates the buffer to store the exported data

ace.getmd t1, K{t0}

# If _ConfigStatus_ is _CfgStcomplete_, the following ace.setst prepares the
# serialized, encrypted and authenticated internal representation of the CR for
# export, computes the SIV(s), then changes _ConfigStatus_ to _CfgStExporting_.
# Otherwise, it does nothing.

```

```

ace.mgmt    K{t0}, #ace_CR_export_start
sd          0(t6), t1                # Export original _ConfigStatus_ to memory

# The next instruction stores all data starting with the 8th byte of the MDH.
# In case of interrupt/exception, the handler will decide whether to keep
`acestart`
# or clear it to force restart.
# The instruction gets the length of the blob to export from the MDH
# no special handling from the SW is necessary.
vloop:
    vsetvli a3, a2, e8, m1, ta, ma    # load rest of PI starting with
MDH[127:64]
    ace.getst t2, K{t0}                # get _State_ field from MDH[63:0]
    bltu      t3, t2, handle_errors    # errors (larger than 23, i.e. ≥ 24)
    beqz      t2, handle_errors        # handle error (is _Off_ needed?)
    ace.mv     v4, K{t0}                # Move to CR
    vse8.v     v4, 8(t6)                # load data
    add       t6, t6, a3                # Bump pointer
    sub       a2, a2, a3                # Decrement number of elements
    bnez      a2, vloop
    ace.getst t2, K{t0}                # get _State_ field from MDH[63:0]
    li        t3, #ace_state_off        # compare to off state
    beq       t2, t3, restart           # CR cleared means force restart

success:
    # Reset MDH[63:0] to the previously saved value. If the saved _ConfigStatus_ is
    # _CfgStcomplete_, the operation also re-decrypts the state and makes the CR
available
    # to software, otherwise it does nothing and leaves the CR state in the
serialized and
    # encrypted format.

ace.mgmt    K{t0}, t1, #ace_CR_export_end

```

Export Code (z1m)

```

ace.size    t2, K{t0}                # get size of the data to export
{...}      # here goes the code that allocates the buffer to store the exported data

ace.getmd   t1, K{t0}

# If _ConfigStatus_ is _CfgStcomplete_, the following ace.setst prepares the
# serialized, encrypted and authenticated internal representation of the CR for
# export, computes the SIV(s), then changes _ConfigStatus_ to _CfgStExporting_.
# Otherwise, it does nothing.

ace.mgmt    K{t0}, #ace_CR_export_start
sd          0(t6), t1                # Export original _ConfigStatus_ to memory

# The next instruction stores all data starting with the 8th byte of the MDH.
# In case of interrupt/exception, the handler will decide whether to keep
`acestart`
# or clear it to force restart.
# The instruction gets the length of the blob to export from the MDH
# no special handling from the SW is necessary.

```

```

ace.store 8(t6), K{t0}

success:
# Reset MDH[63:0] to the previously saved value. If the saved _ConfigStatus_ is
# _CfgStcomplete_, the operation also re-decrypts the state and makes the CR
available
# to software, otherwise it does nothing and leaves the CR state in the
serialized and
# encrypted format.

ace.mgmt K{t0}, t1, #ace_CR_export_end

```

i If we close the operation prematurely, then we get an authentication error. If we forget to close it, we also get errors because we cannot use the CR unless *ConfigStatus* is *CfgStcomplete*

i Longer instruction sequences based on XLEN-wide writes from GPRs into a CR's state are also possible. However, safely resuming such code after preemption may be more cumbersome. This is feasible and minimally impacts code size, since provisioning, import, and export routines can be placed in a shared library, with indirect CR addressing allowing the target CR to be passed as a parameter. Storing some state information in the caller's context also reduces the need for dedicated CSRs.

1.9. External Cryptographic Context Formats

ACE defines two external CC formats:

1. The *Provisioning Input* (PI), loaded into a CR to create a CC; and
2. The *Sealed Cryptographic Context* (SCC), an encrypted and authenticated representation of a CC used when exporting a CR to memory for later import.

Both formats begin with the 128-bit MDH described in [Metadata Formats](#).

1.9.1. Formats of Provisioning Input and Sealed Cryptographic Context

The format of a Provisioning Input (PI) is shown in [Format of the Provisioning Input \(PI\), all Sections in Order](#).

Format of the Provisioning Input (PI), all Sections in Order

Section	Section Name	Size	Description
1	MDH, Metadata Header	128 bits	See Metadata Formats .
2	Content[], Provisioning Content	Depends only on the <i>Algorithm</i> , <i>AlgorithmPolicy</i> , and <i>KeyType</i> fields of the MDH.	Contains keys and other algorithm-specific information.

The length of a PI, and thus of Content[], must be a multiple of 128 bits.

A CR can contain significantly more information than the Provisioning Content. This additional information falls into three categories:

1. Information necessary for the algorithm to function that cannot be reconstructed from other information—for example, random values generated in the *Initial* state.
2. Information that can be easily reconstructed from other state data. The **algorithm** may choose to omit this from an SCC.
3. Data that optimizes performance on a specific **implementation** and is more expensive to recompute than to encrypt and save to external memory. The implementation may export such data in a custom format within an SCC for efficient reuse. A different receiving implementation may safely ignore this data during import after checking the IMPQUAL value. The originating implementation may silently restart a cryptographic operation if the additional data is corrupted.

The format of a Sealed Cryptographic Context (SCC) is shown in [Plaintext of the Sealed Cryptographic Context \(SCC\) Format, all Sections in Order](#).

Plaintext of the Sealed Cryptographic Context (SCC) Format, all Sections in Order

Section	Section Name	Size	Description
1	MDH, MDH	128 bits	See Metadata Formats .
2	SIV	128 bits	Authentication tag for section 4.
3	Content1: <i>Algorithm Data Section</i>	If the CR is not in an Error State, the length of the SCC depends only on the <i>Algorithm</i> , <i>AlgorithmPolicy</i> , <i>KeyType</i> and <i>StateExtension</i> fields of the MDH.	Contains keys and other algorithm-specific information. Encrypted in memory and during partial import. The data in this section is necessary for the correct functioning of the algorithm.
4	IMPQUAL, Implementation Qualifier	128 bits, if present	Used to identify the implementation of the ACE Unit

Section	Section Name	Size	Description
5	SIV2	128 bits, if present	Authentication tag for sections 4 and 6.
6	Content2: <i>Implementation (Specific) Variable- Length Data Section</i> (Implementation VDS)	Implementation- Specific	The data in this section is <i>not</i> necessary for the correct functioning of the algorithm. Encrypted in memory and during partial import.

The length of each section must be a multiple of 128 bits.

If the context is exported while in an Error State, Sections 3–6 are empty; that is, the SCC consists only of a plaintext MDH and of SIV.

The MDH's *ImpDataLen* field specifies the total size of the Variable-Length Section (SIV2, IMPQUAL, and Content2[]).

1.9.2. Single-Share Key Export Rule for Threshold implementations

Any CC with its algorithm implemented as a threshold implementation will always provision, import and export *all* its keys and secrets as reconstructed values.

The PI and SCC Formats will be the same for non-threshold and threshold implementation

variants of the same algorithm, with the only difference being the value of the *SCProtection* in the MDH.

i These rules assume that reconstructing or re-randomizing a key is an inexpensive operation—for example, XORing with values generated by a PRNG internal to the ACE unit. This approach keeps SCCs compact and guarantees interoperability across architectures that implement side-channel protection differently.

To accelerate key share reconstruction in a threshold variant of an algorithm, some shares of a system key can be included in the content's Additional Data.

For the purposes of the architectural specification, all values are represented as a single share. Side-channel analysis (SCA) protected algorithms are defined as implementation variants of the same unprotected algorithm and are not specified separately. The only difference between a protected and an unprotected algorithm is the value of the *SCProtection* field in the MDH.

1.9.3. System Keys and Randomly Generated Key Values

In a PI or SCC, System Keys are encoded as a 64-bit SKID used to retrieve the key from the a system-specific SKS.

This replaces the key value itself with an usually smaller field, and may reduce the size of the PI or SCC.

For threshold implementations of system keys, ACE may copy key material from the SKS into the CR and expand it into multiple shares.

The key must still be exported as a SKID.

Rule 3 of [Single-Share Key Export Rule for Threshold implementations](#) can be applied to system keys as well, as long as the system key's value cannot be recovered from the included shares.

1.9.4. Notation and Representation of CR Internals

The information contained in a CR comprises at least as many bits as the SCC (see [Plaintext of the Sealed Cryptographic Context \(SCC\) Format, all Sections in Order](#)), excluding SIV, IMPQUAL, and SIV2.

Additional storage may be required for information reconstructed upon successful import.

The i -th CR is denoted by $\kappa(i)$.

When $\kappa(i)$ is being provisioned, resp., imported, it behaves like a contiguous structure, whose length equals that of the PI, resp., the SCC plus an additional 128 bits for the SIV, or 384 bits if also IMPQUAL and SIV2 are required.

The additional capacity of 128 or 384 bits is per hart, not per CR, since allocating an additional 128 bits per CR (or 384 bits if at least one algorithm supports implementation-specific data) would be costly.

The implementation must transparently manage this capacity as though it belongs to the CR.

In principle, this limits the ability to perform concurrent management operations within the same piece of code:

If a second operation is started that may overwrite the storage for SIV, IMPQUAL, and SIV2, the old values of SIV and SIV2 may be irreversibly overwritten.

However, this does not affect any conceivable use case.

The format of this structure follows [Format of the Provisioning Input \(PI\), all Sections in Order](#) for provisioning and [Plaintext of the Sealed Cryptographic Context \(SCC\) Format, all Sections in Order](#) for export and import.

1.10. Management of Sealed Cryptographic Contexts

We now present the cryptographic algorithms used to generate and read SCCs.

This completes the description of the semantics of the management operations given in [Provisioning, Import, and Export](#), in the description of `ace.setst`

(`ace.setst`), and in the code detailing the operations as split by the architecture ([Code for CR Configuration and Export Operations](#)).

See also `ace.load` ([ace.load](#)), `ace.store` ([ace.store](#)), and `ace.getmdl` ([ace.getmdv/ace.getmdl/ace.getmdh](#)).

1.10.1. AES-GCM-SIV Key Derivation and POLYVAL

AES-GCM-SIV cite:[DBLP-journals-iacr-GueronLL17,RFC8452] is used to export and import CRs.

We define the Key Derivation ([RFC8452 Key Derivation](#)) and POLYVAL ([Polyval Hashing Function](#)) functions first. The generation of an SCC from a CR ([Generation of an SCC from a Fully Configured CR](#)) and the import of an SCC into a CR ([Decryption and Authentication of a complete SCC into a CR](#)) are defined next. Then, we define AES-GCM-SIV encryption and decryption.

RFC8452 Key Derivation

```
function RFC8452_KeyDeriv(key : bits(256))
  → struct{enc_key : bits(256), auth_key : bits(128)} =
{
  return struct {
    AESE256(key, (zeros(125) @ 0b101))[63:0] @ AESE256(key, (zeros(125) @ 0b100))
  [63:0] @
    AESE256(key, (zeros(125) @ 0b011))[63:0] @ AESE256(key, (zeros(125) @ 0b010))
  [63:0],
    AESE256(key, (zeros(125) @ 0b001))[63:0] @ AESE256(key, (zeros(125) @ 0b000))
  [63:0]
  }
}
```

i Due to the computational cost of the derivation functions, implementations may cache recently used derived keys.

The SIV (“Synthetic IV”), which also serves as the authentication tag in GCM-SIV, is computed by encrypting the POLYVAL hash of the concatenation of the Locality Secrets, the MDH, and the Content of the CR.

In the POLYVAL pseudocode below, the digested data is represented as an array of 128-bit blocks.

Polyval Hashing Function

```
function POLYVAL(auth_key : bits(128), blocks[0..n-1] : bits(128), n : int)
  → bits(128) =
{
  local tmp : bits(128) ← zeros(128);
  local i : int;
  foreach(i from 0 to n-1) {
    tmp ← tmp xor blocks[i];
    tmp ← Montmul(tmp, auth_key);
  }
  return tmp;
}
```

We now give the notation used for the inputs and outputs of the AES-GCM-SIV encryption and decryption functions, i.e. [AES-GCM-SIV Encryption for SCC Generation](#) and [AES-GCM-SIV Decryption for SCC Import](#):

- $AD[]$ denotes the Associated Data as an array of len_AD 128-bit blocks;
- $P[], resp., C[],$ denotes a Plaintext, resp., a Ciphertext, as arrays of len_PC 128-bit blocks;
- K is a 256-bit Key;
- N is a 96-bit Nonce; and
- SIV is a 128-bit SIV/tag.

AES-GCM-SIV Encryption for SCC Generation

```
function AES_GCM_SIV_Encrypt(AD[] : array of bits(128), N : bits(96),
    P[] : array of bits(128), K : bits(256), len_AD : int, len_PC : int)
    → {SIV : bits(128), C[] : array of bits(128)} =
{
    enc_key, auth_key ← RFC8452_KeyDeriv(K);
    SIV ← POLYVAL(auth_key, AD @ P, len_AD + len_PC);
    SIV[95:0] <- SIV[95:0] xor N[95:0];
    SIV ← AESE(enc_key, 0 @ SIV[126:0]);

    foreach(i from 0 to len_PC - 1)
        C[i] ← P[i] xor AESE(enc_key, 1 @ SIV[126:32] @ (SIV[31:0]+bin(i,32) %
2**32));

    return { SIV, C[] };
}
```

AES-GCM-SIV Decryption for SCC Import

```
function AES_GCM_SIV_Decrypt(AD[] : array of bits(128), N : bits(96), SIV :
bits(128),
    C[] : array of bits(128), K : bits(256), len_AD : int, len_PC : int)
    → {correct : bool, P[] : array of bits(128)} =
{
    enc_key, auth_key ← RFC8452_KeyDeriv(K);

    foreach(i from 0 to len_PC - 1)
        P[i] ← C[i] xor AESE(enc_key, 1 @ SIV[126:32] @ (SIV[31:0]+bin(i,32) % 2**32);

    s ← POLYVAL(auth_key, AD @ P, len_AD + len_PC);
    tmp[95:0] ← s[95:0] xor N[95:0];
    tmp ← AESE(enc_key, 0 @ SIV[126:0]);

    if (tmp != SIV) {
        foreach(i from 0 to len_PC - 1)
            P[i] ← zeros(128);
        return { false, P[] };
    }

    return { true, P[] };
}
```

1.10.2. Generation of an SCC from a Fully Configured CR

If the *ConfigStatus* field is set to *CfgStComplete*, then $K(i)$ is fully provisioned, and opening the CR for export (`ace.setst Kd|K{Xd}, #ace_CR_export_start`) performs the

following steps

to create the serialized, encrypted and authenticated SCC from the CR.

Exporting a Fully Configured CR $\kappa(i)$

1. First, the algorithm-dependent data is processed:
 1. $AD[0] \leftarrow K(i).MDH, len_AD \leftarrow 1$
 2. foreach(j from 0 to 10) :
 1. If $MDH.Locality$ includes Locality # j , then $AD[len_AD] \leftarrow LST[_j_], len_AD \leftarrow len_AD + 1$
 3. If *Nonce* (Locality #11) is included, then $N \leftarrow LST[11]$, else $N \leftarrow \text{zeros}(96)$
 4. $len_PC \leftarrow$ lengths of Sections 3.a and 3.b combined.
 5. $SIV, C[] \leftarrow \text{AES_GCM_SIV_Encrypt}(AD, N, Ki.Content1[], CSK, len_AD, len_PC)$
2. Second, if $VarDataLen \neq 0$, the implementation-dependent data is processed:
 1. $len_AD \leftarrow 2, AD[0] \leftarrow K(i).IMPQUAL, AD[0] \leftarrow SIV.$
 2. $len_PC \leftarrow$ length of Section 6.
 3. $SIV2, C2[] \leftarrow \text{AES_GCM_SIV_Encrypt}(AD, N, Ki.Content2[], CSK, len_AD, len_PC)$
3. Finally, output $MDH, SIV, Content1[],$ and, if $VarDataLen \neq 0, IMPQUAL, SIV2,$ and $Content2[]$.

1.10.3. Decryption and Authentication of a complete SCC into a CR

When the import of the SCC of a fully configured CR is completed,

`ace.setst Kd|K{Xd}, #ace_CR_import_end` decrypts and authenticates the SCC by the following algorithm.

Importing a Fully Configured CR $\kappa(i)$

1. Clear $K(i)$.
2. Load M , the MDH, from the SCC, into $K(i).MDH$.
3. Check correctness of M , and whether the Algorithm is supported.
4. Determine the length of the entire SCC.
5. If the length exceeds the implementation's maximum for the given Algorithm, the SCC contains an Implementation VDS that the importing implementation does not support. The SCC length is therefore adjusted by excluding the Implementation VDS.
6. Load the rest of the SCC, up to the determined length.
7. $AD[0] \leftarrow K(i).MDH, len_AD \leftarrow 1$
8. foreach(j from 0 to 10) :
 1. If $MDH.Locality$ includes Locality # j , then $AD[len_AD] \leftarrow LST[j], len_AD \leftarrow len_AD + 1$
9. If *Nonce* (Locality #11) is included, then $N \leftarrow LST[11]$, else $N \leftarrow \text{zeros}(96)$

10. $\text{len_PC} \leftarrow \text{lengths of Section 3.}$
11. $\text{SIV}, \text{C}[] \leftarrow \text{AES_GCM_SIV_Decrypt}(\text{AD}, \text{N}, \text{Ki.Content1[]} , \text{CSK}, \text{len_AD}, \text{len_PC}).$
12. Second, if $\text{VarDataLen} \neq 0$, the Implementation-dependent data is processed:
 1. $\text{len_AD} \leftarrow 2, \text{AD}[0] \leftarrow \text{K}(i).\text{IMPQUAL}, \text{AD}[1] \leftarrow \text{SIV}.$
 2. $\text{len_PC} \leftarrow \text{length of Section 6.}$
 3. $\text{SIV2}, \text{C2}[] \leftarrow \text{AES_GCM_SIV_Encrypt}(\text{AD}, \text{N}, \text{Ki.Content2[]} , \text{CSK}, \text{len_AD}, \text{len_PC})$
13. Finally, output MDH, SIV, Content1[], and, if $\text{VarDataLen} \neq 0$, IMPQUAL, SIV2, and Content2[].

1.10.4. Import and Data-Independent Execution Latency

This section is informative and non-normative.

In ACE, Data-Independent Execution Latency (DIEL) applies only to the cryptographic operations it performs, including the decryption and authentication of SCC contents.

Loading an SCC into a CR and decrypting-and-authenticating it are performed by separate instructions: `ace.load` is followed by an `ace.setst` instruction that performs no memory accesses. This second instruction is either continued while the requesting hart is preempted or interrupted, or aborted and then resumed. As a result, its timing is unaffected by the data being verified.

There is one dependency, however: an IMPQUAL mismatch. If an SCC contains both architecture-dependent and implementation-dependent parts, whether the latter can be imported on the target machine determines how many AEAD operations are performed, namely, one or two:

1. When the import process is opened, ACE checks the total SCC length. If that length exceeds what is supported on the target machine for the given algorithm, the implementation-dependent part is necessarily unsupported: The SCC length to load and process is adjusted to include only the architecture-dependent part, `ace.load` does not load the implementation-dependent part, and only the architecture-dependent part is decrypted (if the resulting *ConfigStatus* is *CfgStComplete*).
2. When the import process is ended, the implementation qualifier is compared. If there is a mismatch, the implementation-dependent part is discarded, the CR length is adjusted, and only the architecture-dependent part is processed (if the resulting *ConfigStatus* is *CfgStComplete*).

This only allows an attacker to distinguish, in certain cases, between the import of an SCC from a migrated VM and that of a native one. Beyond revealing information that is normally easily obtainable (if not a precondition for other attack types), this is unavoidable and not mitigated by splitting the decryption and authentication of the architecture-dependent and implementation-dependent parts into separate instructions.

2. Book 2: Algorithms

This chapter is normative.

This chapter specifies the behavior of the ACE instructions for the supported cryptographic algorithms. Some specifications define algorithm families instantiable with any primitive meeting certain properties—typically, constraints on block size b and key size k . For example, GCM and GCM-SIV are well defined not only with AES but also with other block ciphers such as SM4. In contrast, other algorithm suites are tied to specific primitives, for instance Ascon.

[Algorithm Type and Mode Encodings](#) defines the encoding of symmetric algorithms in the MDH field *Algorithm*.

The encoding uses two sub-fields: *Type* in bits [5:0] and *Mode* in bits [10:6].

For certain classes of algorithms, *Mode* is used to specify a subtype.

Bit 11 is zero.

Other groups of algorithms may subdivide the *Algorithm* field differently.

The suffix "(reserved)" indicates that the algorithm's specification is not yet complete or contains ambiguities.

Algorithm Type and Mode Encodings

Type	Mode	Algorithm name	Extensions Requirement
0	0	AES128_ECB	Zlaes128p
0	1	AES128_CTR	Zlaes128p and Zlctrm
0	2	AES128_LFSR (reserved)	Zlaes128p and Zlctrm
0	3	AES128_XCTR	Zlaes128p and Zlxcctrm
0	4	AES128_XLFSR (reserved)	Zlaes128p and Zlxcctrm
0	5	AES128_XEX1 (one key)	Zlaes128p and Zlxexm
0	6	AES128_XEX2 (two keys)	Zlaes128p and Zlxexm
0	7	AES128_GCM	Zlaes128p and Zlgcmm
0	8	AES128_GCM_IV	Zlaes128p and Zlgcmm
0	9	AES128_GCM_SIV	Zlaes128p and Zlgcmsivm
0	10	AES128_CMAC	Zlaes128p and Zlcmac
0	11	AES128_OCB	Zlaes128p and Zlocbm
0	12	AES128_OCB_IV	Zlaes128p and Zlocbm
1	0	AES192_ECB	Zlaes192p
1	1–12	<i>Analogues to Modes 1–11 for AES128, but with 192 replacing 128.</i>	
2	0	AES256_ECB	Zlaes256p
2	1–12	<i>Analogues to Modes 1–11 for AES128, but with 256 replacing 128.</i>	
3	0	SM4_ECB	Zlsm4p
3	1–10	<i>Analogues to Modes 1–10 for AES128, but with SM4 and sm4 replacing AES128 and aes128.</i>	
4	0	SHA2-224	Zlesha2h
4	1	SHA2-256	Zlesha2h
4	2	SHA2-384	Zlesha2h
4	3	SHA2-512	Zlesha2h

Type	Mode	Algorithm name	Extensions Requirement
4	4	SHA2-512/224	Zlesha2h
4	5	SHA2-512/256	Zlesha2h
4	6	SHA2-224_HMAC	Zlesha2h and Zlhmamcm
4	7	SHA2-256_HMAC	Zlesha2h and Zlhmamcm
4	8	SHA2-384_HMAC	Zlesha2h and Zlhmamcm
4	9	SHA2-512_HMAC	Zlesha2h and Zlhmamcm
4	10	SHA2-512/224_HMAC	Zlesha2h and Zlhmamcm
4	11	SHA2-512/256_HMAC	Zlesha2h and Zlhmamcm
5	0	SHA3-224	Zlesha3h
5	1	SHA3-256	Zlesha3h
5	2	SHA3-384	Zlesha3h
5	3	SHA3-512	Zlesha3h
5	4	SHAKE128	Zlesha3h
5	5	SHAKE256	Zlesha3h
5	6	SHA3-224_HMAC	Zlesha3h and Zlhmamcm
5	7	SHA3-256_HMAC	Zlesha3h and Zlhmamcm
5	8	SHA3-384_HMAC	Zlesha3h and Zlhmamcm
5	9	SHA3-512_HMAC	Zlesha3h and Zlhmamcm
5	10	KMAC128	Zlesha3h and Zlkmamcm
5	11	KMAC256	Zlesha3h and Zlkmamcm
6	0	SM3	Zlsm3h
7	0	Ascon-AEAD128	Zlascon
7	1	Ascon-AEAD128_Nonce	Zlascon
7	2	Ascon-AEAD128_Nonce_Mask	Zlascon
7	3	Ascon-Hash256	Zlascon
7	4	Ascon-XOF128	Zlascon
7	5	Ascon-CXOF128	Zlascon
8	0	Elliptic Curve Cryptography: secp256r1 (NIST P-256)	Zlsecp256r1c
8	1	Elliptic Curves: secp384r1 (NIST P-384)	Zlsecp384r1c
8	2	Elliptic Curves: secp521r1 (NIST P-521)	Zlsecp521r1c
8	3	Elliptic Curves: ed25519	Zled25519c
8	4	Elliptic Curves: ed448	Zled448c
8	5	Elliptic Curves: brainpoolP256r1	ZlbpP256r1c
8	6	Elliptic Curves: brainpoolP384r1	ZlbpP384r1c
8	7	Elliptic Curves: brainpoolP512r1	ZlbpP512r1c
8	8	Elliptic Curves: SM2	ZlSM2c

Type	Mode	Algorithm name	Extensions Requirement
9	0	Post-Quantum Cryptography: ML - KEM-512	Z1mlkem512k
9	1	Post-Quantum Cryptography: ML - KEM-768	Z1mlkem768k
9	2	Post-Quantum Cryptography: ML - KEM-1024	Z1mlkem1024k
9	3	Post-Quantum Cryptography: ML - DSA-44/HashML - DSA-44	Z1mldsa44s
9	5	Post-Quantum Cryptography: ML - DSA-65/HashML - DSA-65	Z1mldsa65s
9	7	Post-Quantum Cryptography: ML - DSA-87/HashML - DSA-87	Z1mldsa87s

2.1. Definition of an Algorithm in ACE

Each algorithm is fully defined by the following information:

Parameters

These include:

- The *block size* b , which is the bit length of the basic unit of information processed by the algorithm, usually the block size of the main underlying primitive;
- The *key size* k , also in bits, for keyed algorithms; and
- Any other parameter needed to instantiate the algorithm with compatible primitives.

Data Structures

These include:

1. Provisioning Input:

The fields are listed in the order they appear in the PI, each with its length.

2. Internal State:

Information maintained in the internal state of a CC.

Field ordering is unspecified, and field lengths may differ from those listed—for example, in threshold or fault-tolerant implementations.

This information is necessary to be able to define the algorithms and their semantics, and must exist internally, though not necessarily in a prescribed format.

An actual implementation may define the internal state and the behavior at each state in a different way, however the semantics of the algorithm must be equivalent to those defined in this specification.

3. Serialized Context:

The plaintext of the SCC.

The fields are listed in the order they appear in an SCC, each with its length.

Algorithm-Specific Functions Any other function that is specific to the algorithm.
State Machine

The state machine is defined by the following information:

1. A list of **States** in which the internal state machine of the logic is executed, together with their representation according to the rules set in [State and StateExtension Fields](#).
2. A complete list of the **allowed state transitions**, including the precise conditions under which each transition is allowed, the Form of `ace.setst` used for each of these transitions and the meaning of any additional parameter;
3. The enumeration of the **allowed/expected instructions** at each state and their **behavior**.
 Besides configuration and export instructions, the instructions permitted in each algorithm state are typically `ace.exec` and `ace.setst`.
 For `ace.exec`, only one of its Forms (see [ace.exec](#)) is *expected* per state.
 By this we intend that either the specified Form must be issued, or Form D `ace.exec` OR Form C `ace.setst` instead, with appropriate setup instructions for the ACEIOBUF. Issuing the `ace.input/ace.output` instructions are not enforced, as properly handling of the inputs and outputs is the responsibility of the caller.

Pseudocode is provided in [Book 4: Pseudocode Examples](#)

to illustrate the software implementation of ACE-supported algorithms using the ACE ISA.

The pseudocode uses the vector registers and omits the alternative sequences based on ACEIOBUF.

The examples are non-normative.

2.2. Generic Rules

1. Opening a CR for export and import, as well as `ace.clone`, and `ace.clear` are always allowed in any State, even if not explicitly mentioned in an algorithm specification.
2. If a usage instruction that may modify the state of a CR is issued, or a state transition is requested in `ace.setst`, that are not explicitly allowed by the current state of the algorithm encoded in the CC in the CR an *illegal instruction exception* is raised, and the CR is invalidated.

2.2.1. Notation in the Algorithm Descriptions

- **INPUT:** The input data to be processed by `ace.exec` OR `ace.setst`. This can be the input vector register, or the ACEIOBUF, as described in [Using ace.input and ace.output to Replace Vector Inputs and Outputs](#).

- **OUTPUT:** The output data produced by `ace.exec`. This can be the output vector register, or the ACEIOBUF, as described in [Using `ace.input` and `ace.output` to Replace Vector Inputs and Outputs](#).
- **ACELEN:** Either `VL*SEW` or the length `aceiobuftop` of the ACEIOBUF in bits, depending on whether vector registers are used or the ACEIOBUF is used.

2.2.2. Representation of System Keys in the Data Structures

System Keys will change all the algorithms described here in the following way:

1. In a PI and in an SCC, the key field will contain the SKID of the system key in place of the value.
2. This field is reduced from the original key size to a 64-bit value. This saves space in an SCC and allows for faster export and import operations.
3. For algorithms that require more than one key, such as XEX/XTS, a single SKID is used to refer retrieve multiple keys from the SKS.

2.3. Padding

As a general rule, inputs shorter than a full vector length must be handled by the caller. The caller is responsible for proper padding and for passing the actual input length to the algorithm at the appropriate places using the correct instructions, usually `ace.setst`. An exception applies when an algorithm specification defines such handling itself for cryptographic security reasons.

2.4. States for the Symmetric Algorithms Defined in this Specification

The following table, together with [Global CC State Numbers: Off State](#), [Global CC State Numbers: Valid States](#), and [Global CC State Numbers: Error States](#), defines the states used by the symmetric algorithms in this specification.

Constants defining CC states for symmetric algorithms

Value	Mnemonic for the Constant	State Name	Description
2	<code>ace_state_operate</code>	<i>Operate</i>	Generic operational state.
	<code>ace_state_hash_absorb</code>	<i>Hash_Absorb</i>	(Same value as <i>Operate</i> . This is just a more intuitive mnemonic for hash functions.) Start and initialize a hash function of the tag/MAC part of an algorithm and start absorbing blocks of data (in the case of an AEAD algorithm, this

Value	Mnemonic for the Constant	State Name	Description
			usually is the associated data).
3	ace_state_hash_last_block	<i>Hash_Last_Block</i>	In this state additional data may be absorbed, such as partial blocks and lengths, if the hash/tag algorithm processes the last block of its input or of the associated data part of its input in a different way from the other blocks.
4	ace_state_hash_finalize	<i>Hash_Finalize</i>	Finalize the internal state of a hash/tag function.
5	ace_state_hash_verify	<i>Hash_Verify</i>	Compare the internal hash/tag value to the provided one.
6	ace_state_hash_output	<i>Hash_Output</i>	Enter the hash output phase of a hash/tag function.
7	ace_state_encrypt	<i>Encrypt</i>	In this state encryption is performed, as well as an update to the tag computation if this operation is part of the algorithm.
8	ace_state_decrypt	<i>Decrypt</i>	In this state decryption is performed, as well as an update to the tag computation if this operation is part of the algorithm.
9	ace_state_enc_last_block	<i>Enc_Last_Block</i>	For the encryption path of authenticated encryption modes of operation to process the last block separately.
10	ace_state_dec_last_block	<i>Dec_Last_Block</i>	For the decryption path of authenticated encryption modes of operation to process the last block separately.

Value	Mnemonic for the Constant	State Name	Description
11	ace_state_enc_tag_finalize	<i>Enc_Tag_Finalize</i>	Finalize the tag computation for AE mode encryption.
12	ace_state_dec_tag_finalize	<i>Dec_Tag_Finalize</i>	Finalize internal tag computation for AE mode decryption, verify the value.
13	ace_state_set_aux_value	<i>Set_Aux_Value</i>	Set a public/auxiliary value in an algorithm.

2.4.1. Electronic Codebook Modes (Direct Encryption/Decryption)

ECB mode cite:[nist-SP-800-38A-1] is the simplest way to use a block cipher.

Parameters

- b = bit length of a block ($8|b$). ACELEN must be a multiple of b (including b).
- k = bit length of a key ($8|k$).

Data Structures

• Provisioning Input:

Pos.	Size (Bits)	Field
i	128	MDH
ii	k or 64	Key or System Key Identifier

• Internal State:

- Metadata Header.
- key: k bits.

• Serialized Context:

Pos.	Size (Bits)	Field
i	128	MDH
ii	k or 64	Key or System Key Identifier

Algorithm-Specific Functions

- $\text{enc_blk}(K, p) \rightarrow c$ encrypts plaintext block p using the key K , and returns the ciphertext result c .
- $\text{dec_blk}(K, c) \rightarrow p$ decrypts ciphertext block c using the key K , and returns the plaintext result p .

State Machine

• States:

- *Initial*: The CC is ready.
- *Encrypt*: Encrypting blocks of input.

- *Decrypt*: Decrypting blocks of input.
- **Allowed State Transitions:**
 - From any valid state to *Encrypt*, if encryption is allowed.
 - From any valid state to *Decrypt*, if decryption is allowed.
- **Behavior:**
 - In State *Initial*, no `ace.exec` instruction is allowed.
 - In States *Encrypt* or *Decrypt*, Form A `ace.exec` instructions can be issued, i.e.,
“`ace.exec OUTPUT, Kn|K{Xn}, INPUT.`”
- If the state is *Encrypt*, then `OUTPUT <- enc_blk(key, INPUT).`
- If the state is *Decrypt*, then `OUTPUT <- dec_blk(key, INPUT).`
- If `ACELEN > b`, then “`ace.exec OUTPUT, Kn|K{Xn}, INPUT`”
encrypts/decrypts block-by-block. Here, and in the rest of this document,
this means from the blocks in the least significant positions to the most
significant positions:
 1. `foreach(i from 0 to ACELEN-b by b) {`
 1. `OUTPUT[i+b-1:i] <- enc_blk(key, INPUT[i+b-1:i]) }`

In [Simple Example: Encryption with no Authentication](#) we illustrate how to use an ECB CC to encrypt a block of data.

2.4.2. Chaining/Feedback Modes

There is no need to explicitly architect the behavior of chaining modes, such as Cipher Block Chaining (CBC), Cipher Feedback (CFB) and Output Feedback (OFB)—all defined in cite:[nist-SP-800-38A-1]—as they are easily implemented in software on top of the ECB, with no intermediate value requiring confidentiality.

2.4.3. CTR/LFSR/XCTR/XLFSR Keystream Generation

In the “non-X” modes, i.e., CTR mode and its analog with an LFSR replacing the arithmetic counter, the sum of the lengths of IV and counter/LFSR equals the block size of the underlying keyed PRP or PRF. IV and counter/LFSR are concatenated to form the input to the latter.

In the “X” modes, both IV and counter/LFSR have the same size as the block size of the underlying keyed PRP or PRF. IV and counter/LFSR are xored together to form the input to the keyed PRP or PRF.

Parameters

- b = block size ($8|b$). For this algorithm `ACELEN` must be a multiple of b .
- k = key size ($8|k$).
- j = counter size ($8|j$).
- n = IV size ($8|n$).
- If the mode is CTR or LFSR, it is $b = n + j$, else it is $b = n = j$.

Data Structures

- **Provisioning Input:**

Pos.	Size (Bits)	Field
i	128	MDH
ii	k or 64	key or System Key Identifier

- **Internal State:**

- Metadata Header.
- key: k bits.
- IV: n bits.
- ctr: j bits.

- **Serialized Context:**

Pos.	Size (Bits)	Field
i	128	MDH
ii	k or 64	key or System Key Identifier
iii	n	IV
iv	j	ctr

Algorithm-Specific Functions

- `keystream_block(p) -> c`—usually an encryption function.
- `tick_ctr()`—updates the `ctr` (either by a modular increment or a LFSR).

State Machine

- **States:**

- *Initial*: The CC is ready.
- *Operate*: In this state the CC will produce a keystream.

- **Allowed State Transitions:** any.

- **Behavior:**

- In State *Initial*, the `ctr` and `iv` fields are set to 0 (`ctr` is set to 1 if the algorithm is LFSR-based).
- To transition to State *Operate*, a Form C `ace.setst` instruction must be issued.
 - This sets the value of `IV` to `INPUT`.
 - If `ACELEN > b`, only the `b` least significant bits of `INPUT` are considered.
- In State *Operate*, `ace.exec` instructions are expected to be of Form C.

“`ace.exec OUTPUT, Kn|K{Xn}`” performs:

1. If in a CTR or LFSR mode then `tmp ← keystream_block(IV @ ctr)`,
else (i.e., if in a XCTR or XLFSR mode) `tmp ← keystream_block(IV xor ctr)`,
2. `tick_ctr()`, and
3. `OUTPUT ← tmp`.

If $ACELEN$ is a multiple of b , the above three commands are applied to each of the $ACELEN/b$ b -bit blocks of $INPUT$, producing the corresponding b -bit block of $OUTPUT$ each time.

Note that if `ace.setst` is issued to return to State *Initial*, the CC can be reused with a new IV.

Example pseudocode is given in [Encrypt with Keystream Generation](#) (for the case $b = ACELEN = 128$).

2.4.4. XEX/XTS modes (one or two keys)

The XTS mode was designed for storage confidentiality cite:[DBLP-journals-ieeeesp-Martin10] and made into a Recommendation by NIST in 2010 cite:[nist-SP-800-38E]. XTS is based on the XEX construction, formalized by Phillip Rogaway cite:[DBLP-conf-asiacrypt-Rogaway04], to design *Tweakable Block Ciphers* cite:[DBLP-conf-crypto-LiskovRW02]. A Tweakable Block Cipher (TBC) is a block cipher which, besides the key and the plaintext or ciphertext, also takes a tweak as input. The tweak is used together with the key to select the permutation computed by the block cipher.

The XEX construction builds a TBC $\tilde{E}$ from a block cipher E , with key κ , tweak τ , and plaintext P , as follows:

+

$\tilde{E}(K, T, P) = E(K, P \text{ xor } M) \text{ xor } M$ where $M = E(K, T)$ is called a *mask*.

+

In XEX mode, once a mask is computed, it is then updated by using a Galois Multiplication for any successive block instead of updating the tweak and encrypting it again.

The NIST standard uses two different keys, one for the data encryption path and one for the encryption of the tweak. The use of a single key or two independently sampled keys is determined by the algorithm, and it is not a configuration parameter for the algorithm. We first define XEX mode functionality.

Parameters

- b = block size ($8|b$). For this algorithm, $ACELEN$ must be an integer multiple of b .
- k = key size ($8|k$).

Data Structures

- **Provisioning Input:**

Pos.	Size (Bits)	Field
i	128	MDH
ii	k or 64	key1 or System Key Identifier
iii	k or 0	key2 or empty if key1 given

- **Internal State:**

- Metadata Header.
- key1: k bits.
- key2: k bits—omitted if only one key is used or if both keys come from an entry in the SKS.
- mask: b bits.

- **Serialized Context:**

Pos.	Size (Bits)	Field
i	128	MDH
ii	k or 64	key1 or System Key Identifier
iii	k or 0	key2 or empty if key1 given
iv	b	mask

Algorithm-Specific Functions

- `enc_blk( $K$ ,  $p$ )` -> c encrypts block p using the key K , and returns the result c .
- `dec_blk( $K$ ,  $c$ )` -> p decrypts block c using the key K , and returns the result p .
- `update_mask(mask)` -> `mask` updates the mask by a Galois Field multiplication in $GF(2^b)$.

State Machine

- **States:** *Initial*, *Encrypt*, and *Decrypt* (defined as before, cf. [Electronic Codebook Modes \(Direct Encryption/Decryption\)](#)).
- **Allowed State Transitions:**
 - *Initial* $\rightarrow$ *Encrypt*, if encryption is allowed.
 - *Initial* $\rightarrow$ *Decrypt*, if decryption is allowed.
 - *Encrypt*, *Decrypt* $\rightarrow$ *Initial*.
- **Behavior:**
 - In State *Initial*:
 - The `mask` field is set to zero.
 - No `ace.exec` operation is allowed.
 - The `ace.setst` instruction to transition to either *Encrypt* or *Decrypt* is expected to be of Form C.

It sets the tweak, and the `mask` field is used to store it: `mask <- INPUT`.

 - If `ACELEN` $>$ b , only the b least significant bits of `INPUT` are considered.
 - Then, the following operations are performed:
 1. if two independent keys are used then
 1. `mask <- enc_blk(key2, mask)`
 2. else
 1. `mask <- enc_blk(key1, mask)`
 2. `mask <- update_mask(mask)`.
 - In States *Encrypt* and *Decrypt*, `ace.exec` instructions are expected to be of Form A.

“`ace.exec OUTPUT,  $K_n | K\{X_n\}$ , INPUT`” performs the following operations:

1. OUTPUT is computed as follows:
 1. In State *Encrypt*, `OUTPUT <- mask xor enc_blk(key1, INPUT xor mask)`.
 2. In State *Decrypt*, `OUTPUT <- mask xor dec_blk(key1, INPUT xor mask)`.
2. `mask <- update_mask(mask)`.

If ACELEN is a multiple of b , the above commands are executed ACELEN/b times, processing each b -bit block of INPUT into a b -bit block of OUTPUT.

Note that if `ace.setst` is issued to return to State *Initial*, the CC can be reused with a new tweak.

Encryption pseudocode is provided in [Pseudocode for encryption in any IV-based non-authenticated mode without associated data](#), decryption is similar

2.4.5. Generic Tweakable Block Ciphers

The specification of a generic tweakable block cipher is similar to that of a non-tweakable block cipher in ECB mode. Since `ace.exec` does not admit variants with two inputs, and we have a single input buffer, we set the tweak with a separate instruction. If block and tweak sizes are different, a reconfiguration of the vector length may be required between operations, unless the tweak is shorter than the block size, in which case the instruction setting the tweak will only consider the necessary bits.

Some commonly used TBCs are Deoxys cite:[Deoxys-141], Skinny cite:[DBLP-conf-crypto-BeierleJKLOPSSS16], QARMA cite:[DBLP-journals-tosc-Avanzi17], and `QARMA`{empty}`v2` cite:[DBLP-journals-tosc-AvanziBDEGNR23].

Parameters

- b = block size ($8|b$). For this algorithm, ACELEN must be an integer multiple of b .
- k = key size ($8|k$).
- t = tweak size ($8|t$).

Data Structures

- **Provisioning Input:**

Pos.	Size (Bits)	Field
i	128	MDH
ii	k or 64	key or System Key Identifier

- **Internal State:**

- Metadata Header.
- key: k bits.
- tweak: t bits.

- **Serialized Context:**

Pos.	Size (Bits)	Field
i	128	MDH
ii	k or 64	key or System Key Identifier
iii	t	tweak

State Machine

- **States:**
 - *Initial*, *Encrypt*, and *Decrypt* defined as before.
 - *Set_Aux_Value*, for setting the tweak value in the next encryption or decryption operation.
- **Allowed State Transitions:.**
 - *Initial* $\rightarrow$ *Set_Aux_Value*
 - *Set_Aux_Value* $\rightarrow$ *Encrypt*, if encryption is allowed.
 - *Set_Aux_Value* $\rightarrow$ *Decrypt*, if decryption is allowed.
- **Behavior:**
 - In State *Initial*:
 - Upon entering the state, the `tweak` field is cleared.
 - In each state, Form C `ace.setst` instructions can be issued, which perform:
 1. `tweak <- INPUT`, and,
 2. Instead of transitioning to State *Set_Aux_Value*, the State remains unchanged (can be initial, encrypt, or decrypt).
 - In State *Encrypt*, respectively, *Decrypt*, besides Form C `ace.setst` instructions, `ace.exec` instructions are expected to be of Form A, where “``ace.exec OUTPUT, Kn|K{Xn}, INPUT``” encrypts, respectively, decrypts `INPUT` into `OUTPUT`.
 - This algorithm does not automatically update the tweak value.

2.4.6. GCM Mode

The GCM mode was designed by David McGrew and John Viega cite:[DBLP-conf-indocrypt-McGrewV04] and was made into a NIST Recommendation in 2007 cite:[nist-SP-800-38D].

Parameters

- k = key size ($8|k$).
- b = block size ($8|b$). For this algorithm, `ACELEN` must be an integer multiple of b .
- c = counter size ($8|b$). It must be $c < b$.

Data Structures

- **Provisioning Input:**

Pos.	Size (Bits)	Field
i	128	MDH

Pos.	Size (Bits)	Field
ii	k or 64	key or System Key Identifier

- **Internal State:**

- Metadata Header.
- key: k bits.
- ctr: c bits.
- IV: b-c bits.
- auth_key, tag: b bits.

- **Serialized Context:**

Pos.	Size (Bits)	Field
i	128	MDH
ii	k or 64	key or System Key Identifier
iii	c	ctr
iv	b-c	IV
v	b	tag

i There are no fields for the lengths of the AD and of the plaintext, as they are maintained by the calling software. Also, hash_key is not serialized, and thus it is recomputed upon `ace.import`.

Algorithm-Specific Functions

- `enc_blk(key, p) -> c` encrypts block `p` using the key, and returns the result `c`.
- `absorb(data)` absorbs data into the hash function by performing:
 1. `tag <- tag xor INPUT`.
 2. `tag <- Galoismul(tag, hash_key)`.

State Machine

- **States:**

Besides *Initial*, *Success*, and *Failure*, the GCM Mode algorithms make use of the following states:

- *Hash_Absorb*: Process Associated Data (AD) into the tag.
- *Encrypt*: Encrypt the plaintext and absorb the ciphertext into the tag.
- *Enc_Tag_Finalize*: Finalize the encryption tag, prepare to return its value.
- *Decrypt*: Absorb the ciphertext into the tag and decrypt it.
- *Dec_Tag_Finalize*: Finalize the encryption tag and keep it internally.
- *Hash_Verify*: Compare the tag against a user supplied value.

- **Allowed State Transitions:**

- *Initial* $\rightarrow$ *Hash_Absorb*.

- *Hash_Absorb* → *Encrypt*, if encryption is allowed.
- *Hash_Absorb* → *Decrypt*, if decryption is allowed.
- *Encrypt* → *Enc_Tag_Finalize* → *Success*.
- *Decrypt* → *Dec_Tag_Finalize* → *Hash_Verify* → *Success* or *Failure*.

- **Behavior:**

- In State *Initial* :
 - No `ace.exec` operation is allowed.
 - The following initialization operations are performed:
 1. `hash_key <- enc_blk(key, zeros(b))`,
 2. `tag <- zeros(b)`,
 3. `ctr <- zeros(c-1) @ 1`, and
 4. `IV <- zeros(b)`.
- To transition to State *Hash_Absorb*: the `ace.setst` instruction must be a Form C to set the IV, i.e., the fixed part of the counter. Then:
 1. `IV <- INPUT[b-c-1:0]`.
 - If the `ace.setst` instruction is repeated, it will just update the counter again.
- If the AD is empty, the transition to State *Hash_Absorb* is necessary to initialize the counter.
- In State *Hash_Absorb*, an `ace.exec` instruction of Form B is expected, whereby “`ace.exec Kn|K{Xn}, INPUT``” absorbs a data block in `tag`, i.e.:

```
1. absorb(INPUT).
```

If $ACELEN > b$, the above operation is executed $ACELEN/b$ times.

- In State *Encrypt*, `ace.exec` instructions are expected to be of Form A. “`ace.exec OUTPUT, Kn|K{Xn}, INPUT``” encrypts `INPUT` into `OUTPUT`, and absorbs `OUTPUT` into the `tag`, namely:

```
1. If ctr = ones(c-1), then
  1. the CR is invalidated.
2. else
  1. tmp <- INPUT xor enc_blk(key, IV @ ctr).
  2. absorb(tmp).
  3. ctr <- ctr + 1 % 2c.
  4. OUTPUT <- tmp.
```

If $ACELEN$ is a multiple of b , the above operations are executed $ACELEN/b$ times, encrypting each b -bit block of `INPUT` into the corresponding b -bit block of `OUTPUT`.

- To transition to State *Enc_Tag_Finalize*, a Form C `ace.setst` instruction is issued, with the following value, as computed by the software itself:

```
• INPUT <- bin(bit_length(AD), b/2) @ bin(bit_length(plaintext), b/2) .
```

The authentication tag is then computed as follows:

1. `absorb(INPUT)`.
2. `tag <- tag xor enc_blk(key, IV @ zeros(c))`.

Finally, `tag` is returned in `OUTPUT`, and the state is changed to *Success*.

If `ACELEN > b`, only the `b` least significant bits of `INPUT` are considered.

- In State *Enc_Tag_Finalize*, `ace.exec` instructions are expected to be of Form C. “`ace.exec OUTPUT, Kn|K{Xn}`” writes the `tag` to `OUTPUT`, and the state transitions to *Success*.
-

In State *Decrypt*, `ace.exec` instructions are expected to be of Form A.

`ace.exec OUTPUT, Kn|K{Xn}`, `INPUT` absorbs the ciphertext `INPUT` into `tag`, then decrypts `INPUT` into `OUTPUT`, namely:

1. If `ctr = ones(c-1)`, then
 1. the CR is **invalidated**.
2. else
 1. `absorb(INPUT)`.
 2. `OUTPUT <- INPUT xor enc_blk(key, IV @ ctr)`.
 3. `ctr <- ctr + 1 % 2c`.

If `ACELEN` is a multiple of `b`, the same behavior applies as in State *Encrypt*, namely the above operations are executed `ACELEN/b` times, decrypting each `b`-bit block of `INPUT` into the corresponding `b`-bit block of `OUTPUT`.

- To transition to State *Dec_Tag_Finalize*, a Form C `ace.setst` instruction is expected to provide the following value, which must be set by the software:

• `INPUT <- bin(bit_length(AD), b/2) @ bin(bit_length(plaintext), b/2)`

The authentication tag is computed as follows:

1. `absorb(INPUT)`.
2. `tag <- tag xor enc_blk(key, ctr[b-1,c] @ zeros(c))`.

If `ACELEN > b`, only the `b` least significant bits of `INPUT` are considered.

- To transition to State *Hash_Verify*, a Form C `ace.setst` instruction must be used to provide the value against which the internally computed tag must be compared: If the values match, state transitions to *Success*, else it transitions to *Failure*.
- Upon completion of `ace.import`, the ACE unit recomputes `hash_key`, i.e.:
 1. `hash_key <- enc_blk(key, zeros(b))`.

[GCM Encryption](#) and [GCM Decryption](#) provide pseudocode for GCM encryption and decryption.

- i** GCM security is not compromised if $\mathbf{H}$ is revealed. Consequently, an implementation of GCM based on an ECB mode CC would be acceptable, for instance as in [Implementing GCM encryption using an ECB CC](#). However, this significantly increases code size and requires support for carry-less multiplication, which may not be available in all hardware.
- Furthermore, a GCM implementation is necessary to support the use case where the IV is also set in advance and not modifiable by the user. It must be possible to configure a GCM CC with both secret key and the IV. This is described in the next section.

2.4.7. GCM with Set IV

The mode of operation “GCM with a Set IV” is a variant of the GCM mode where the initialization vector (IV) is predefined and not generated randomly. This variant *cannot* be emulated with an ECB CC because it would reveal the internal data.

A key difference between GCM and GCM with Set IV is in the format of the Provisioning Input:

Data Structures

- **Provisioning Input:**

Pos.	Size (Bits)	Field
i	128	MDH
ii	k or 64	key or System Key Identifier
iii	b-c	IV
iv	(Padding)	Zero padding

- **Internal State:**

Same as for GCM.

- **Serialized Context:**

Same as for GCM.

- **States and Allowed Transitions:**

Same as for GCM.

- **State Machine Behavior:**

The differences with respect to GCM are:

- To transition to State *Hash_Absorb*: The `ace.setst` instruction shall be of Form A, i.e., *without any additional inputs*, since the IV is already included in the PI.
- When transitioning back to State *hash_Initial*: The CR's iv field is *not* reset to zero.

i For all the following IV- or nonce-based modes, versions with a set IV or nonce (such as for GCM-SIV) can also be similarly defined. For instance, for GCM-SIV ([GCM-SIV Mode](#)), nonce would be included in the PI exactly as iv is included in the PI for GCM, and the transition to *Hash_Absorb* would also use a Form A `ace.setst`. All other semantics would be unchanged.

2.4.8. GCM-SIV Mode

The GCM-SIV mode is described in cite:[DBLP-journals-iacr-GueronLL17] and cite:[RFC8452]. While GCM-SIV's decryption state machine is similar to the GCM's, the encryption state machine is different:

- During encryption, the *Hash_Absorb* state absorbs the entire associated data and plaintext. The *Enc_Tag_Finalize* state then absorbs the lengths and produces the SIV/tag. Finally, the *Encrypt* state performs a second pass to encrypt the plaintext using the SIV/tag as the initialization vector.
- During decryption, the *Hash_Absorb* state absorbs only the associated data. The *Decrypt* state decrypts the ciphertext and simultaneously absorbs the plaintext into the hash. Finally, the *Dec_Tag_Finalize* state absorbs the lengths, produces the SIV/tag, and internally compares it to the provided SIV/tag.

Parameters

- k = key size ($64|k$).
- b = block size, ($64|b$). For this algorithm, `ACELEN` must be an integer multiple of b .
- It must be $k = b$ or $k = 2b$.
- The authentication tag is b bits long.

Data Structures

• Provisioning Input:

Pos.	Size (Bits)	Field
i	128	MDH
ii	k or 64	key or System Key Identifier

• Internal State:

- Metadata Header.
- `key`, `enc_key`: k bits.
- `auth_key`: b bits.

- `nonce`: $3b/4$ bits.
- `ctr`: $b/4$ bits.
- `SIV`: b bits.
- `tmp`: b bits.

- **Serialized Context:**

Pos.	Size (Bits)	Field
i	128	MDH
ii	k or 64	key or System Key Identifier
iii	$3b/4$	nonce
iv	$b/4$	ctr
v	b	SIV
vi	b	tmp



The internal state fields `enc_key` and `auth_key` are not serialized, and must be recomputed upon each `ace.import`.

Algorithm-Specific Functions

- `enc_blk(key, p) -> c` encrypts block `p` using the key, and returns the result `c`.
- `absorb(data)` absorbs data into the hash function by performing a step of (a variant of) POLYVAL:
 1. `tmp <- tmp xor INPUT`.
 2. `tmp <- Montmul(tmp, hash_key)`.

State Machine

- **States:**

Besides *Initial*, *Success*, and *Failure*, the GCM-SIV algorithms make use of the following states:

- *Set_Aux_Value*: Set the value of nonce.
- *Hash_Absorb*: Set the SIV and process AD and, if encrypting, also the plaintext into the partial tag (this is accumulated in the state variable `tmp`).
- *Encrypt*: Encrypt the plaintext.
- *Enc_Tag_Finalize*: Finalize the SIV with the AD and PT lengths, prepare to return its value.
- *Decrypt*: Absorb the ciphertext into the partial tag and decrypt it.
- *Dec_Tag_Finalize*: Finalize the tag with the AD and CT lengths, then compare it against the SIV.

- **Allowed State Transitions:**

- *Initial* $\rightarrow$ *Set_Aux_Value* $\rightarrow$ *Hash_Absorb*.
- For encryption: *Hash_Absorb* $\rightarrow$ *Enc_Tag_Finalize* $\rightarrow$ *Encrypt*.
- For decryption: *Hash_Absorb* $\rightarrow$ *Decrypt* $\rightarrow$ *Dec_Tag_Finalize* $\rightarrow$ *Success* or *Failure*.
- Transition to *Encrypt* possible only if encryption is allowed.

- Transition to *Decrypt* possible only if decryption is allowed.

- **Behavior:**

- In State *Initial* :
 - Upon entering the state, the following initialization operations are performed:
 1. `(enc_key, auth_key) <- RFC8452_RFC8452_KeyDeriv(key)`,
 2. `nonce, ctr, tmp, SIV <- 0`.
 - No `ace.exec` operation is allowed.
 - Upon issuing a Form C ``ace.setst Kn|K{Xn}, #ace_state_set_aux_value, INPUT``, the value of `nonce` is set to `INPUT`.
(If the instruction is repeated, there is no harm, since it just overwrites the `nonce` before any other operation is performed.)
- In State *Set_Aux_Value*:
 - Upon issuing a Form C ``ace.setst Kn|K{Xn}, #ace_state_hash_absorb, INPUT``:

- The value of `SIV` is set to `INPUT`.

i This version of the instruction is meant to be used during decryption. It is harmless to invoke it during encryption, however, as the `SIV` field will be overwritten later with the correct synthetic value.

- Upon issuing a Form A ``ace.setst Kn|K{Xn}, #ace_state_hash_absorb``:

- The `SIV` is not modified (it is assumed to be already zero).

i This version of the instruction is meant to be used during encryption. If used during decryption, by leaving the `SIV` to zero it will most likely lead to an authentication error.

-

In State *Hash_Absorb*, Form B `ace.exec` instructions are expected.

Such an instruction absorbs its `INPUT` in the internal state variable `tmp`, i.e., it performs `absorb(INPUT)`.

If `ACELEN` is a multiple of `b`, `absorb()` is executed on each of the `ACELEN/b` blocks in `INPUT`.

- In State *Enc_Tag_Finalize*, an `ace.exec` instruction of Form A is expected. Its input value

- `INPUT <- bin(bit_length(AD), b/2) @ bin(bit_length(plaintext), b/2)`,

is set by the user.

Then, “``ace.exec``” finalizes the `SIV` computation by performing:

1. `absorb(INPUT)`.
2. `tmp <- enc_blk(enc_key, 0 @ (tmp xor N)[b-2:0])`
3. `SIV <- 0 @ tmp[b-2:0]` and `OUTPUT <- SIV`,

If $ACELEN > b$, only the b least significant bits of `INPUT` are considered.

- In State *Encrypt*, `ace.exec` instructions are expected to be of Form A, namely “``ace.exec OUTPUT, Kn|K{Xn}, INPUT``,” to encrypt `INPUT` into `OUTPUT` as follows:

1. If `ctr == 0`, then the CR is **invalidated**.
2. `OUTPUT <- INPUT`
`xor enc_blk(enc_key, 1 @ SIV[b-2:b/4] @ (SIV[b/4-1:0] + ctr % 2b/4)),`
3. `ctr <- ctr + 1`.

If $ACELEN > b$, the above operations are applied to each of the $ACELEN/b$ blocks in `INPUT`.

- In State *Decrypt*, `ace.exec` instructions are expected to be of Form A. “``ace.exec OUTPUT, Kn|K{Xn}, INPUT``” decrypts `INPUT` into `OUTPUT` and then absorbs the plaintext `OUTPUT` into the `tmp`:

1. If `ctr == 0`, then the CR is **invalidated**.
2. `OUTPUT <- INPUT`
`xor enc_blk(enc_key, 1 @ SIV[b-2:b/4] @ (SIV[b/4-1:0] + ctr % 2b/4)),`
3. `absorb(OUTPUT)`.
4. `ctr <- ctr + 1`.

If $ACELEN > b$, the above operations are applied to each of the $ACELEN/b$ blocks in `INPUT`.

- In State *Dec_Tag_Finalize*, a single `ace.exec` instruction of Form B is expected, namely “``ace.exec Kn|K{Xn}, INPUT``,” where it is the responsibility of the user to set

- `INPUT <- bin(bit_length(AD), b/2) @ bin(bit_length(plaintext), b/2)`.

`ace.exec` then performs the following operations:

1. `absorb(INPUT)` (which modifies `tmp`).
2. `tmp <- enc_blk(enc_key, 0 @ (tmp xor N)[b-2:0])`.
3. `tmp <- tmp[n-2:0]`,
4. If `tmp` and `SIV` match, then
 1. `State <- ace_state_success`
5. else
 1. `State <- ace_state_failure`

If $\text{ACELEN} > b$, only the b least significant bits of `INPUT` are considered.

- Upon completion of `ace.import`, the ACE unit recomputes `enc_key` and `auth_key`, i.e.:
 1. $(\text{enc_key}, \text{auth_key}) \leftarrow \text{RFC8452_RFC8452_KeyDeriv}(\text{key})$.

Encryption and decryption pseudocode is found in [GCM-SIV Mode](#).

2.4.9. OCB Mode

We only architect the OCB3 variant of OCB as specified in RFC 7253 [cite:\[RFC7253\]](#) and [cite:\[DBLP-journals-joc-KrovetzR21\]](#). While this algorithm is asymptotically more efficient than GCM and GCM-SIV, it has a much heavier setup and its exported state requires significantly more memory than the other modes. On the other hand, since it uses only a single key and no derived keys, with ciphers with an expensive key expansion like the AES it may use fewer internal resources.

The state machine of this algorithm closely follows RFC 7253 (except for the numbering of the bit fields, which is big endian in the RFC). The algorithm is identical to the RFC for 128-bit-wide block ciphers, with a straightforward generalization to other widths according to the security proof of the original algorithm and a lower bound on nonce length following [cite:\[cryptoeprint-2023-326\]](#).

Parameters

- b = block size. b is a power of two, and ≥ 64 . For OCB, ACELEN must be an integer multiple of b .
- k = key size, $b|k$.

Also, define $g = \log_2(b)$.

Data Structures

- **Provisioning Input:**

Pos.	Size (Bits)	Field
i	128	MDH
ii	k or 64	key or System Key Identifier

- **Internal State:**

- Metadata Header.
- key: k bits.
- N : $u+1$ bits (at most u bits are used).
- L , `offset`, `hash_A`, `hash_P`: b bits.
- N_len , `tag_len`, `last_blk_len`: g bits.

- **Serialized Context:**

Pos.	Size (Bits)	Field
i	128	MDH
ii	k or 64	key or System Key Identifier
iii	b	N
iv	b	L (only u+1 bits used)
v	b	offset
vi	b	hash_A
vii	b	hash_P
viii	16	tag_len
ix	16	last_blk_len
x	16	N_len
xi	16 or 80	Padding

Algorithm-Specific Functions

- `enc_blk(K,p)` -> `c` encrypts block `p` using the key `k` and returns the result `c`.
- `dec_blk(K,c)` -> `p` decrypts block `c` using the key `k` and returns the result `p`.
- `double(S)`:
Let `c` be `0x1B` for `b = 64`, `0x87` for `b = 128`, and `0x425` for `b = 256`, then:
 - If `S[b-1] == 0`, then `{ double(S) <- S << 1 }` else `{ double(S) <- (S << 1) xor C }`.

State Machine

• States:

Besides *Initial*, *Success*, and *Failure*, the OCB Mode algorithms make use of the following states:

- *set_aux_value*: Set the length and the value of the nonce.
- *Hash_Absorb*: Set the tag length, and process the AD.
- *Hash_Finalize*: Set the length of the final block of the AD and process it.
- *Encrypt*: Process the plaintext by absorbing it into the tag and encrypting it.
- *Enc_Last_Block*: Provide the last block of the plaintext, then process it.
- *Enc_Tag_Finalize*: Return the tag.
- *Decrypt*: Process the ciphertext by decrypting it and absorbing the plaintext into the tag.
- *Dec_Last_Block*: Provide the last block of the ciphertext, then process it.
- *Hash_Verify*: Compare a user-provided value with the internally computed tag.

• Allowed State Transitions:

- *Initial* → *Set_Aux_Value* → *Hash_Absorb* → *Hash_Finalize*
- *Hash_Finalize* → *Encrypt*, if encryption is allowed.
- *Hash_Finalize* → *Decrypt*, if decryption is allowed.

- *Encrypt* → *Enc_Last_Block* → *Enc_Tag_Finalize* → *Success*.
- *Decrypt* → *Dec_Last_Block* → *Hash_Verify* → *Success* or *Failure*.

- **Behavior:**

- In State *Initial*:
 - No `ace.exec` operation is allowed.
 - The following initialization operations are performed:
 1. `N <- 1 @ zeros(u)`.
 2. `hash_A <- zeros(b)`
 3. `hash_P <- zeros(b)`
- To move to *Set_Aux_Value*, a Form B `ace.setst` is expected, where the auxiliary argument `xs` sets the nonce length `N_len`. `N_len` must be at least `g-1`. If not, the CR transitions to Error State `ace_state_invalid`.
- In State *Set_Aux_Value*, `ace.exec` instructions are expected to be of Form B, namely “`ace.exec Kn|K{Xn}, INPUT``”, whereby `N` is set to `INPUT`, namely `N <- zeros(u-N_len) @ INPUT[N_len-1:0]`
 - If “`ace.exec Kn|K{Xn}, INPUT``,” is repeated, it will rewrite `N`.
 - If no such `ace.exec` is issued, `N` is not changed, which means that its most significant bit is 1. This condition results in an error when transitioning to State *Hash_Absorb*.
- To move to *Hash_Absorb*, a Form B `ace.setst` instruction is expected, where the auxiliary argument `xs` sets the tag length `tag_len`.
 - If this argument (interpreted as an unsigned value) is larger than `b` or is equal to 0 (the latter occurs also if `x0` is passed as the auxiliary argument), and the CR is invalidated.
 - If `N[u] == 1`, this means that `N` was not set in State *Set_Aux_Value*. Nonce setting cannot be skipped, as this mode is not secure under nonce reuse, therefore in this case the CR is invalidated.
 - Following operations are performed:
 1. `tag_len <- Xs`.
 2. `offset <- zeros(b)`
 3. `L <- enc_blk(key, zeros(b))`
 4. `L <- double(double(L))`
- In State *Hash_Absorb*, `ace.exec` instructions are expected to be of Form B, whereby “`ace.exec Kn|K{Xn}, INPUT``” absorbs a data block in `tag`, i.e.:

```

1. L <- double(L)
2. offset <- offset xor L
3. hash_A <- hash_A xor enc_blk(key, INPUT xor offset)

```

If `ACELEN > b`, the above operation is executed `ACELEN/b` times.

- To move to *Hash_Finalize*, a Form B `ace.setst` operation is expected, to set the length of the final block of the AD in `last_blk_len`.

- In State *Hash_Finalize*, only a single `ace.exec` instruction of Form B is expected if `last_blk_len != 0`. “``ace.exec Kn|K{Xn}, INPUT``” which absorbs the last data block in `tag`, i.e.:

```

• tmp : bits(b)

1. offset <- offset xor enc_blk(key, zeros(b))
2. tmp <- (zeros(b-last_blk_len-1) @ 1 @ INPUT[last_blk_len-1:0]) xor offset
3. hash_A <- hash_A xor enc_blk(K, tmp)

```

If `last_blk_len == 0`, however, no `ace.exec` can be executed, and only an `ace.setst` instruction is allowed to enter State *Encrypt* or *Decrypt*.

- Upon entering states *Encrypt* or *Decrypt*, the “setup” and “init” stages of the OCB algorithm are performed:

```

• N_ext : bits(b) (N_ext means “N, extended”)
• Stretch : bits(3b/2)
• bot : bits(g-1)
• Ktop : bits(b)

1. offset <- zeros(b)
2. L <- enc_blk(key, zeros(b))
3. L <- double(double(L))
4. N_ext <- N[N_len-1:0] @ 1 @ zeros(b-g-1-N_len) @ bin(tag_len,g)
5. bottom <- int(N_ext[b-1:b-g+1]) // Rogaway numbers the bits in the opposite
   order.
6. Ktop <- enc_blk(key, zeros(g-1) @ N_ext[b-g:0])
7. Stretch <- (Ktop[b/2-1:0] xor Ktop[b/2+7:8]) @ Ktop
8. offset <- Stretch[b+bottom:bottom]

```

- In State *Encrypt*, `ace.exec` instructions are expected to be of Form A:
“``ace.exec OUTPUT, Kn|K{Xn}, INPUT``” absorbs the plaintext `INPUT` into the `tag` and encrypts `INPUT` into `OUTPUT`, namely:

```

1. L <- double(L)
2. offset <- offset xor L
3. hash_P <- hash_P xor INPUT
4. OUTPUT <- offset xor enc_blk(key, INPUT xor offset)

```

If `ACELEN > b`, the above operation sequence is executed `ACELEN/b` times, encrypting each `b`-bit block of `INPUT` into the corresponding `b`-bit block of `OUTPUT`.

- To move to *Enc_Last_Block*, a Form B `ace.setst` instruction is expected to be issued to set the bit length of the final block of the plaintext in `last_blk_len`.
-

In State *Enc_Last_Block*, if `last_blk_len` is nonzero a single `ace.exec` instruction of Form A

“`ace.exec OUTPUT, Kn|K{Xn}`, INPUT` is expected.

In this case, following operations are performed:

- tmp : bits(last_blk_len)
 - tmp2 : bits(b)
 - aux : bits(b)
1. aux <- enc_blk(key, zeros(b))
 2. offset <- offset xor aux
 3. tmp <- enc_blk(key, offset)[last_blk_len-1:0]
 4. tmp <- INPUT[last_blk_len-1:0] xor tmp
 5. OUTPUT <- zeros(b-last_blk_len) @ tmp
 6. tmp2 <- (zeros(b-last_blk_len-1) @ 1 @ tmp) xor offset
 7. aux <- double(aux)
 8. hash_P <- enc_blk(key, hash_P xor tmp2 xor offset xor aux) xor hash_A

On the other hand, if last_blk_len is zero, only an ace.exec instruction of Form D, “`ace.exec Kn|K{Xn}`,” is expected, which performs:

- aux : bits(b)
1. aux <- enc_blk(key, zeros(b))
 2. aux <- double(aux)
 3. hash_P <- enc_blk(key, hash_P xor offset xor aux) xor hash_A

In both cases the state then transitions to *Enc_Tag_Finalize*.

- In State *Enc_Tag_Finalize*, only a single ace.exec instruction of Form C, “`ace.exec OUTPUT, Kn|K{Xn}`,” is expected, which performs OUTPUT <- zeros(b-tag_len) @ hash_P[tag_len-1:0].
- In State *Decrypt*, ace.exec instructions are expected to be of Form A. Such an ace.exec decrypts INPUT into OUTPUT, and absorbs the plaintext OUTPUT into the tag, namely:

- tmp : bits(b)
1. L <- double(L)
 2. offset <- offset xor L
 3. tmp <- offset xor enc_blk(key, INPUT xor offset)
 4. hash_P <- hash_P xor tmp
 5. OUTPUT <- tmp

If ACELEN > b, the same behavior applies as in State *Encrypt*.

- To move to State *Dec_Last_Block*, a Form B ace.setst operation must be used to also contextually set the bit length of the final block of the ciphertext in last_blk_len.
- In State *Dec_Last_Block*, only a single ace.exec instruction of Form A “ace.exec OUTPUT, Kn|K{Xn}, INPUT” is expected if last_blk_len is nonzero. In this case, the same operations as in State *Enc_Last_Block* are performed except for Line 6, which is instead

```
1. tmp2 <- (zeros(b-last_blk_len-1) @ 1 @ INPUT[last_blk_len-1:0]) xor offset
```

and the state is changed to *Hash_Verify*.

If `last_blk_len` is zero, only a single `ace.exec` instruction of Form D, “`ace.exec Kn|K{Xn}``,” is expected, which performs the same operations as in State *Enc_Last_Block*, except for the final state transition, which also in this case is to *Hash_Verify*.

- To complete the algorithm in the decryption branch, a Form C `ace.setst` operation is expected, to (confirm the) transition to State *Hash_Verify* and compare the `tag_len` least significant bits of `INPUT` with the `tag_len` least significant bits of `hash_P`:

```
1. If the values match, then the State transitions to Success, else it transitions to Failure.
```

Pseudocode for OCB encryption is given in [OCB Mode](#).

2.4.10. CMAC Functions

CMAC is an algorithm for message authentication codes constructed from a block cipher. It was designed by Tetsu Iwata and Kaoru Kurosawa and became a NIST Recommendation in May 2005 cite:[nist-SP-800-38B].

Parameters

- `b` = block size ($8|b$). For this algorithm, `ACELEN` must be an integer multiple of `b`.
- `k` = key size ($8|k$).

Data Structures

- **Provisioning Input:**

Pos.	Size (Bits)	Field
i	128	MDH
ii	k or 64	key or System Key Identifier

- **Internal State:**

- Metadata Header.
- key: k bits.
- hash: b bits.
- `last_block_len`: 64 bits.

- **Serialized Context:**

Pos.	Size (Bits)	Field
i	128	MDH

Pos.	Size (Bits)	Field
i	k or 64	key or System Key Identifier
ii	b	hash
iii	64	last_block_len

Algorithm-Specific Functions

- `enc_blk(key, p) -> c` encrypts block `p` using the key `key`, and returns the result `c`.
- `gen_subkeys(K) -> (K1, K2)` generates subkeys `K1` and `K2` from the master key `K` as follows:

Let `c` be `0x1B` for `b = 64`, `0x87` for `b = 128`, and `0x425` for `b = 256`, then

1. `L <- enc_blk(K, zeros(b))`
2. if `(msb(L) == 0)` then
 - { `K1 <- L << 1` }
 - else
 - { `K1 <- (L << 1) xor C` }.
3. if `(msb(K1) == 0)` then
 - { `K2 <- K1 << 1` }
 - else
 - { `K2 <- (K1 << 1) xor C` }.
4. return `(K1, K2)`.

State Machine

- **States:**
 - *Initial*: The CC is ready.
 - *Hash_Absorb*: Absorb data into the CMAC function.
 - *Hash_Last_Block*: Set the length of the last block to absorb, then absorb it.
 - *Hash_Output*: Output the CMAC.
 - *Success*: The CC has completed the CMAC operation.
- **Allowed State Transitions:**
 - *Initial* → *Hash_Absorb* → *Hash_Last_Block* → *Hash_Output* → *Success*.
 - From any valid state to State *Initial*.
- **Behavior:**
 - In State *Initial*:
 - `last_block_len <- zeros(64)`
 - `hash <- zeros(b)`
 - No ``ace.exec`` operation is allowed.
 - In State *Hash_Absorb*, `ace.exec` instructions are expected to be of Form B.
 “``ace.exec Kn|K{Xn}, #ace_state_hash_last_block, Xs``” absorbs the data block `INPUT` into the hash as follows:

```
1. hash <- enc_blk(key, hash xor INPUT).
```

If `ACELEN > b`, then the above operations are applied to each `b`-bit block of `INPUT`.

- Upon issuing a Form B “``ace.setst Kn|K{Xn}, #ace_state_hash_last_block, Xs``” where `Xs` contains the bit length of the last block, the following happens:

1. if `last_block_len != 0`, then the CR is **invalidated**. (The previous block is not complete.)
 2. if `Xs > b` then the CR is **invalidated**.
 3. `last_block_len <- Xs`.
- In State *Hash_Last_Block*, only a single `ace.exec` instruction of Form B may be issued, i.e., “`ace.exec Kn|K{Xn}, INPUT`”, to absorb the data block `INPUT`. If `ACELEN > b`, only the least significant `b` bits of `INPUT` are considered.

```

1. (K1, K2) <- gen_subkeys(key).
2. if last_block_len == b, then {
  1. tmp <- hash xor INPUT xor K1
3. } else {
  1. tmp <- INPUT
  2. Remove the b - last_block_len most significant bits of tmp and pad the resulting
     value with a 1 and sufficiently many 0 bits to make it b bits long.
  3. tmp <- hash xor tmp xor K2 }
4. hash <- enc_blk(key, tmp)
5. The state is changed to Hash_Output.

```

- In State *Hash_Output*, we can issue one `ace.exec` instruction of Form C, i.e., “`ace.exec OUTPUT, Kn|K{Xn}`”, to output the hash into `OUTPUT`.
If `ACELEN > b`, only the bits of `OUTPUT` beyond the `b`-th are cleared.
Upon issuing that instruction, the CC transitions to State *Success* with no error.

Pseudocode is given in [CMAC Functions](#).

2.4.11. Hash and MAC Functions, and XOFs

This section defines an interface for hash and MAC functions. A completely general interface cannot be defined because of the variety of constructions. However, the one given here is sufficiently general to allow describing most common functions, such as, for instance, the SHA2 cite:[nist-fips-180-4], SHA3 and SHAKE cite:[nist-fips-202], as well as HMAC cite:[nist-fips-198-1].

Parameters

- `b` = block size ($8|b$). This is the size of an absorbed block and may be different from the size of the internal state that is kept by the function. For a sponge function, this value is the *rate*.
- `t` = tag len ($8|t$). This is the size of the digest, or of the block that is output each time in a XOF. The present description assumes $t \leq b$.
- `n` = size of the internal state that is kept by the function ($8|n$). For a sponge function, $n-b$ is the *capacity* of the function. Either the block

size is an integer multiple of the vector length or the vector length is an integer multiple of the block size.

- k = key size ($8|k$). Present only if keyed, i.e., a MAC.

Data Structures

• Provisioning Input:

Pos.	Size (Bits)	Field
i	128	MDH
ii	If keyed: k , else 64	key or System Key Identifier

• Internal State:

- Metadata Header.
- `key`: k bits. Present only if keyed, i.e., a MAC.
- `state`: n bits.
- `last_block_len`: int. Present only if this Hash/MAC provides special treatment for a last block that goes beyond padding and appending the length. The latter must be handled by user software instead.
- `input_base`: int. Offset within input/output to read from/write to.
- `block_base`: int. Length of absorbed data so far for the current block. Only if needed by the algorithm.
- `cumul_len`: int. Cumulative length of data absorbed so far (if the algorithm must process and incorporate it in the hash on its own).
- `block`: b bits. Included only if a partial block must be maintained—for instance when the inputs are not directly absorbed into state.

• Serialized Context:

Pos.	Size (Bits)	Field	Notes
i	128	MDH	--
ii	If keyed: k or 64	key or SKID	--
	If unkeyed: 0	none	--
iii	n	state	--
iv	16	last_block_len	Present only if this Hash/MAC has special internal treatment for a fractional last block that goes beyond padding and appending the length, which are handled by user software.
v	16	block_base	Amount of data stored so far into block. Only if used by the algorithm, else zero padding.
vi	32	cumul_len	Cumulative length of all data absorbed so far. Only if the algorithm must process and incorporate it in

Pos.	Size (Bits)	Field	Notes
			the hash on its own, else zero padding.
vii	$64 \cdot \lceil \text{block_base}/64 \rceil$	block	Only if <code>block_base</code> is defined and $\neq 0$ and the block must be stored separately from the <code>state</code> .
viii	0 or 64	Padding	To reach a multiple of 128 bits.

- i** Regarding the `last_block_len/block_base/cumul_len` group: if any of these fields is not used, it is replaced by a corresponding padding block. If all three are not used by the hashing algorithm, then the 64-bit block is omitted, and `block` is clearly also

Algorithm-Specific Functions

- `absorb()`: Process `block` into the hash or MAC.
- `absorb_last_block()`: Process `block` into the hash or MAC, using a function for the last block.
- `finalize()`: Process the state into the digest value. We assume this either re-uses `block` (the entire digest, or a unit of XOF output should not be bigger than that) or the digest will be part of `state`.
- `update()`: Updates the state during the digest output stage.

State Machine

- **States:** *Initial*, *Hash_Absorb*, *Hash_Last_Block*, *Hash_Output* and *Success*.
 - *Initial*: The CC is ready.
 - *Hash_Absorb*: Absorb data into the hash function.
 - *Hash_Last_Block*: Set the length of the last block to `absorb`, then absorb it.
 - *Hash_Output*: Output the hash.
 - *Success*: The CC has completed the operation (XOFs do not have this state).
- **Allowed State Transitions:**
 - *Initial* → *Hash_Absorb* → *Hash_Last_Block* → *Hash_Output* → *Success*.
 - From any valid state to State *Initial*.
- **Behavior:**
 - In State *Initial*:
 - `last_block_len`, `block_base` and `block` are set to zero.
 - `state`: the initial value is set as specified by the algorithm.
 - Any `ace.exec` will result in the CR being **invalidated**.
 - In State *Hash_Absorb*, only `ace.exec` instructions of Form B may be issued, i.e., “`ace.exec Kn|K{Xn}, INPUT`”, to absorb the data block `INPUT` into the hash as follows.

```
1. input_base <- 0 // or astart upon resumption
2. while (input_base < ACELEN) do { // as long as we have input to process
```

```

1. amount <- min(ACELEN-input_base, b-block_base)
2. block[block_base + amount - 1 : block_base] <-
  <- INPUT[input_base + amount - 1 : input_base]
  (If there is no block field, could also be something like
   state[block_base + amount - 1 : block_base] <-
   <- state[block_base + amount - 1 : block_base] +
   + INPUT[input_base + amount - 1 : input_base],
   it could directly replace part of state, or some other function.)
3. input_base <- input_base + amount
4. block_base <- block_base + amount
5. if (block_base == b) then { // it's time to absorb the block
  1. absorb()
  2. block_base <- 0
  3. Here the instruction may be interrupted, with astart <- input_base.
3. }}

```

- Upon issuing a Form B “ace.setst $K_n|K\{X_n\}$, #ace_state_hash_last_block, x_s ” where x_s contains the length of the last block, the following happens:

```

1. if block_base != 0, then the CR is invalidated. (The previous block is not complete.)
2. if  $x_s > b$ , then the CR is invalidated.
3. if  $x_s == 0$ , then the CR is invalidated.
4. last_block_len <-  $x_s$ .

```

Then, state transitions to *Hash_Last_Block* as expected.

- State *Hash_Last_Block* is very similar to *Hash_Absorb*, the differences being:
 - last_block_len is used in place of b throughout.
 - absorb_last_block() is called instead of absorb() in Step 3.d.i.
 - The instruction is terminated after absorb_last_block()—only one block is absorbed—and the state transitions to *Hash_Output*.
- In State *Hash_Output*, the cases where the current function is a hash function or a XOF behave differently.
 - Upon entering the state, the following operations are first performed:


```

1. block[t - 1 : 0] <- finalize()
2. block_base <- 0

```
 - For each Form C ace.exec instruction “ace.exec OUTPUT, $K_n|K\{X_n\}$ ”, the following operations are performed:

```

• amount : int, amount of data to copy from block to OUTPUT
• #define output_base input_base

1. output_base <- 0 // or astart upon resumption
2. while (output_base < ACELEN) do { // as long as there is room in the OUTPUT

```

```

1. amount <- min(ACELEN - output_base, t - block_base)
2. OUTPUT[output_base + amount - 1 : output_base] <-
   <- block[block_base + amount - 1 : block_base + amount]
3. output_base <- output_base + amount
4. block_base <- block_base + amount
5. if (block_base == t) then {
  1. if we are in a Hash function, transition to State Success, and return.
  2. update()
  3. block_base <- 0
  4. Here the instruction can be interrupted, with astart <- output_base.
3. }}

```

2.4.12. HMAC

Hash function acceleration is sufficient to compute HMACs, and no dedicated algorithms are needed for either performance or security reasons.

2.4.13. SHA-3 Family Specific Rules

We describe here the algorithms for the primitives in the SHA-3 family, which currently includes SHA3-224, SHA3-256, SHA3-384, SHA3-512, SHAKE128 and SHAKE256. These primitives are designed around a special parameter choice of KECCAK cite:[DBLP-conf-eurocrypt-BertoniDPA13] standardized by NIST cite:[nist-fips-202]. These are instantiations of the algorithm specified in [Hash and MAC Functions, and XOFs](#), and we need to describe only the differences with respect to the behavior described there.

Parameters

- b = rate (8| b). It is $b = 1600 - c$, where $c = 448, 512, 768$, or 1024 is the capacity.
- $n = 1600$
- $t = c/2$

Data Structures

- **Provisioning Input:** Empty.

Pos.	Size (Bits)	Field
i	128	MDH

- **Internal State:**

- Metadata Header.
- state: n bits.
- last_block_len: int.
- input_base: int.
- block_base: int.

- **Serialized Context:**

Pos.	Size (Bits)	Field
i	128	MDH
ii	1600	state
iii	16	last_block_len
iv	16	block_base
v	96	Padding

Algorithm-Specific Functions

- `P()`: computes KECCAK-p[1600, 24] on state.
- `absorb()`: in this case there is no explicit operation to copy data into state, as the inputs will be directly XORed into it, however `absorb` applies `P()` to the state.
- `absorb_last_block()`: this is a NOP.
- `finalize()`: this is a NOP.
- `update()`: updates the state during the digest output stage. It applies `P()` to the state.

State Machine

- **States:** *Initial*, *Hash_Absorb*, *Hash_Last_Block*, *Hash_Finalize*, *Hash_Output* and *Success*.
- **Allowed State Transitions:**
 - *Initial* → *Hash_Absorb* → *Hash_Last_Block* → *Hash_Finalize* → *Hash_Output* → *Success*.
 - From any valid state to State *Initial*.
- **Behavior:**
 - In State *Initial*, state is zeroed.
 - In State *Hash_Absorb*, the input data is directly XORed to state.
 - In State *Hash_Last_Block*, the input data is directly XORed to state.
 - Upon transitioning to State *Hash_Output*:
 1. The suffix and padding string *s* is generated.
 2. If the bit length of *s* is *b*-*block_base*:
 1. *s* is directly XORed to rate part of the state, and
 2. `P()` is applied.
 3. If the bit length of *s* is *2b*-*block_base*:
 1. The first *b*-*block_base* bits are XORed to the rate part of the state,
 2. `P()` is applied,
 3. the remaining *b* bits are XORed to the rate part of the state, and
 4. `P()` is applied again.

The rest of the algorithm follows [Hash and MAC Functions, and XOFs](#).

2.4.14. Ascon-AEAD128

Ascon is a public permutation cite:[DBLP-journals-joc-DobraunigEMS21] standardized by the NIST cite:[nist-SP-800-232] together with various cryptographic algorithms designed around it.

! In all ASCON algorithms, the ordering of the bits must be verified!

Ascon-AEAD128 is an AEAD algorithm. In Ascon-AEAD128, it is the responsibility of the user to apply proper padding to the AD and plaintext. The inputs are therefore assumed to be properly padded. Similarly, it is the responsibility of the user to remove the padding from the decryption of the ciphertext to recover the original plaintext.

Parameters

- $b = 128$. For this algorithm, $ACELEN$ must be an integer multiple of b .
- $k = 128$.

Data Structures

• Provisioning Input:

Pos.	Size (Bits)	Field
i	128	MDH
ii	128 or 64	key or System Key Identifier

• Internal State:

- Metadata Header.
- key: 128 bits, or 64 bits if it is a System Key Identifier.
- `state[0 .. 4]`: five 64-bit words, i.e., 320 bits.
- `tag_len`: 8 bits as int.
- `last_block_len`: 9 bits as int.
- `AD_empty`: Boolean;

• Serialized Context:

Pos.	Size (Bits)	Field
i	128	MDH
ii	128 or 64 (padded to 128)	key or System Key Identifier
iii	64	<code>state[0]</code>
iv	64	<code>state[1]</code>
v	64	<code>state[2]</code>
vi	64	<code>state[3]</code>
vii	64	<code>state[4]</code>
viii	16	<code>last_block_len</code>

Pos.	Size (Bits)	Field
ix	16	tag_len
x	32	Padding

Algorithm-Specific Functions

- `ASCON(p)` applies p rounds of ASCON to the state.

State Machine

- **States:**
 - *Initial*: The CC is ready.
 - *Hash_Absorb*: Set the nonce, and process the AD.
 - *Encrypt*: Encrypt the plaintext.
 - *Enc_Last_Block*: Encrypt the last block of the plaintext.
 - *Hash_Output*: Return the tag.
 - *Decrypt*: Decrypt the ciphertext.
 - *Dec_Last_Block*: Decrypt the last block of the ciphertext.
 - *Hash_Verify*: Compare a user-provided value with the internally computed tag.
 - *Success*: The algorithm has completed its operation without errors.
 - *Failure*: The algorithm has completed its operation with an authentication failure.
- **Allowed State Transitions:**
 - *Initial* → *Hash_Absorb*.
 - *Hash_Absorb* → *Encrypt*, if encryption is allowed.
 - *Hash_Absorb* → *Decrypt*, if decryption is allowed.
 - *Encrypt* → *Enc_Last_Block* → *Hash_Output* → *Success*.
 - *Decrypt* → *Dec_Last_Block* → *Hash_Verify* → *Success* or *Failure*.
- **Behavior:**
 - In State *Initial* :
 - No `ace.exec` operation is allowed.
 - The following initialization operations are performed:
 1. `state[0] <- 0x00001000808c0001`
 2. `state[1] <- key[63:0]`
 3. `state[2] <- key[127:64]`
 4. `state[3] <- zeros(64)`
 5. `state[4] <- zeros(64)`
 6. `tag_len <- 128`
 7. `AD_empty <- true`
 - To transition to State *Hash_Absorb*, a Form C `ace.setst` is expected, whose INPUT sets the nonce of the algorithm as follows:

```

1. state[3] <- INPUT[63:0]
2. state[4] <- INPUT[127:64]
3. ASCON(12)
4. state[3] <- state[3] xor key[63:0]
5. state[4] <- state[4] xor key[127:64]
```

If $ACELEN > 128$, only the 128 least significant bits of `INPUT` are considered.

- In State *Hash_Absorb*, only `ace.exec` instructions of Form B are expected.
`“ace.exec Kn|K{Xn}, INPUT”` absorbs a data block into the state, i.e.:

```
1. AD_empty <- false
2. ASCON(8)
3. state[0] <- state[0] xor INPUT[63:0]
4. state[1] <- state[1] xor INPUT[127:64]
```

If $ACELEN > 128$, this sequence of four operations is executed $ACELEN/128$ times, once for each 128-bit block of `INPUT`.

- Upon entering State *Encrypt* or *Decrypt*, the following domain separation operation is performed:
 1. `lsb(state[4]) <- lsb(state[4]) xor 1`
- In State *Encrypt*, `ace.exec` instructions are expected to be of Form A, whereby
`“ace.exec OUTPUT, Kn|K{Xn}, INPUT”` encrypts `INPUT` into `OUTPUT`, namely:

```
1. state[0] <- state[0] xor INPUT[63:0]
2. state[1] <- state[1] xor INPUT[127:64]
3. OUTPUT <- state[1] @ state[0]
4. ASCON(8)
```

If $ACELEN$ is a multiple of 128, these operations are executed $ACELEN/128$ times, encrypting each 128-bit block of `INPUT` into the corresponding 128-bit block of `OUTPUT`.

- In State *Enc_Last_Block*, only one `ace.exec` instruction of Form A is expected to be issued, to
 encrypt the last block.
`“ace.exec OUTPUT, Kn|K{Xn}, INPUT”` performs

```
1. state[0] <- state[0] xor INPUT[63:0]
2. state[1] <- state[1] xor INPUT[127:64]
3. OUTPUT <- state[1] @ state[0] // only the 128 lsbs of OUTPUT are written to.
4. The state machine then transitions to State Hash_Output.
```

In this state only one 128-bit block is processed.

The differences between the operation of `ace.exec` in this state and in State *Encrypt* are the absence of the final `ASCON(8)` and the automatic transition to State *Hash_Output*.

- In State *Hash_Output*, only one `ace.exec` instruction of Form C is expected to be issued, to compute the tag.
`“ace.exec OUTPUT, Kn|K{Xn}”` performs

```
1. state[2] <- state[2] xor key[63:0]
2. state[3] <- state[3] xor key[127:64]
```


3. ASCON(12)
4. `state[3] <- state[3] xor key[63:0]`
5. `state[4] <- state[4] xor key[127:64]`
6. `OUTPUT <- state[4] @ state[3]` // only the 128 lsbs of OUTPUT are written to.
7. The state machine then transitions to State *Success*.

In this state only one 128-bit block is processed.

- In State *Decrypt*, `ace.exec` instructions are expected to be of Form A, i.e., `ace.exec OUTPUT, Kn|K{Xn}, INPUT`. This operation performs:

- `tmp : bits(128)`
- 1. `tmp <- (state[0] xor INPUT[63:0]) @ (state[1] xor INPUT[127:64])`
- 2. `state[0] <- INPUT[63:0]`
- 3. `state[1] <- INPUT[127:64]`
- 4. ASCON(8)
- 5. `OUTPUT <- tmp`

If ACELEN is a multiple of 128, these operations are executed ACELEN/128 times, decrypting each 128-bit block of INPUT into the corresponding 128-bit block of OUTPUT.

- In State *Dec_Last_Block*, only a Form A `ace.exec` instruction is expected. “`ace.exec OUTPUT, Kn|K{Xn}, INPUT`” performs:

- `tmp : bits(128)`
- 1. `tmp <- (state[1] xor INPUT[127:64]) @ (state[0] xor INPUT[63:0])`
- 2. `state[0] <- INPUT[63:0]`
- 3. `state[1] <- INPUT[127:64]`
- 4. The state machine transitions to State *Hash_Verify*
- 5. `OUTPUT <- tmp`

- In State *Dec_Tag_Finalize*, a Form B `ace.setst Kn|K{Xn}, #ace_state_dec_tag_finalize, Xs` instruction may be executed, setting the tag len `tag_len` to the value of `Xs`, provided it is not smaller than 32 and not larger than 128. If the value is out of bounds, the CR is **invalidated**.
- In State *Hash_Verify*, only one `ace.exec` instruction of Form D is expected to be issued, to verify the tag. “`ace.exec Kn|K{Xn}, INPUT`” performs

1. `state[2] <- state[2] xor key[63:0]`
2. `state[3] <- state[3] xor key[127:64]`
3. ASCON(12)
4. `state[3] <- state[3] xor key[63:0]`
5. `state[4] <- state[4] xor key[127:64]`
6. `INPUT[tag_len-1:0]` is compared to `(state[4] @ state[3])[tag_len-1:0]`.

7. If the values match, then the State transitions to *Success*, else it transitions to *Failure*.

2.4.15. Ascon-AEAD128 with set Nonce

Similarly to GCM with Set IV, Ascon-AEAD128 can also be made into a version with a set nonce.

In this case the State *Set_Aux_Value* is not present and is thus skipped, and the PI has the following form.

Data Structures

- **Provisioning Input:**

Pos.	Size (Bits)	Field
i	128	MDH
ii	128 or 64	key or System Key Identifier
iii	128	nonce

In State *Initial*, words 3 and 4 of state are initialized thus:

1. `state[3] <- nonce[63:0]`
2. `state[4] <- nonce[127:64]`

Everything else is as for Ascon-AEAD128.

2.4.16. Ascon-AEAD128 with Nonce Masking

The NIST specification also describes a version of Ascon-AEAD128 with *Nonce Masking*. This variant is described as follows:

Data Structures

- **Provisioning Input:**

Pos.	Size (Bits)	Field
i	128	MDH
ii	128 or 64	k1 or System Key Identifier
iii	128 or 0	k2 or System Key Identifier in Pos. ii

- **Internal State:**

- Metadata Header.
 1. k1: 128 bits, or 64 bits if it is a System Key Identifier.
 2. k2: 128 bits, omitted if a System Key is used.
- `state[0 .. 4]`: five 64-bit words, i.e., 320 bits.
- `tag_len`: 8 bits.

- **Serialized Context:**

Pos.	Size (Bits)	Field
i	128	MDH
ii	128 or 64	k1 or System Key Identifier
iii	128 or 0	k2 or System Key Identifier in Pos. ii
iv	64	state[0]
v	64	state[1]
vi	64	state[2]
vii	64	state[3]
viii	64	state[4]
ix	16	tag_len
x	48 or 112	Padding

The algorithm has the same states as Ascon-AEAD128 as described in [Ascon-AEAD128](#), with the key `key` equal to `k1` and the nonce `N` replaced throughout by `N xor K2`.

For encryption and decryption with both nonce masking and a set nonce, the algorithm described in [Ascon-AEAD128 with set Nonce](#) is used, giving `k1` as the key `key` and providing `N xor K2` as the nonce value.

2.4.17. Ascon-Hash256

In this mode it is the responsibility of the user to apply proper padding to the inputs.

Parameters

- `b = 64`. For this algorithm, `ACELEN` must be an integer multiple of `b`.

Data Structures

- **Provisioning Input:**

Pos.	Size (Bits)	Field
i	128	MDH

- **Internal State:**

- Metadata Header.
- `state[0 .. 4]`: five 64-bit words, i.e., 320 bits.
- A countdown, an integer going from 3 down to 0, is stored in bits [1:0] of *AlgorithmUse*

- **Serialized Context:**

Pos.	Size (Bits)	Field
i	128	MDH (includes countdown)
ii	64	state[0]
iii	64	state[1]

Pos.	Size (Bits)	Field
iv	64	state[2]
v	64	state[3]
vi	64	state[4]

Algorithm-Specific Functions

- `ASCON(p)` applies p rounds of ASCON to the state.

State Machine

- **States:** *Initial*, *Hash_Absorb*, *Hash_Finalize*, and *Success*.
- **Allowed State Transitions:**
 - *Initial* $\rightarrow$ *Hash_Absorb* $\rightarrow$ *Hash_Finalize* $\rightarrow$ *Success*.
- **Behavior:**
 - In State *Initial* :
 - No `ace.exec` operation is allowed.
 - The following initialization operations are performed:
 1. `state[0] <- 0x0000080100cc0002`
 2. `state[1] <- zeros(64)`
 3. `state[2] <- zeros(64)`
 4. `state[3] <- zeros(64)`
 5. `state[4] <- zeros(64)`
 6. `ASCON(12)`
 - In State *Hash_Absorb*, only `ace.exec` instructions of Form B are expected to be issued. “`ace.exec Kn|K{Xn}, INPUT``” absorbs a data block into the state, i.e.:
 1. `state[0] <- state[0] xor INPUT`
 2. `ASCON(12)`

If $\text{ACELEN} > 64$, these operations are executed $\text{ACELEN}/64$ times, once for each 64-bit block of `INPUT`.

- Upon entering *Hash_Finalize*, `countdown` is set to 3.
- In State *Hash_Finalize*, at most four `ace.exec` instructions of Form C are expected to be issued. To compute the tag, “`ace.exec OUTPUT, Kn|K{Xn}`” performs

```

1. ASCON(12)
2. OUTPUT <- state[0]
3. if countdown == 0 then transition to State Success.
4. countdown <- countdown - 1

```

If $\text{ACELEN} \geq 128$, these operations are executed $\text{ACELEN}/64$ times, writing to `OUTPUT[63:0]` the first time, then `OUTPUT[127:64]`, `OUTPUT[191:128]` and `OUTPUT[255:192]`. If more than one `ace.exec` instructions are required, writing restarts from `OUTPUT[63:0]`. If `countdown` reaches zero before the entire output registers is written to, the state of the unwritten bits is

undefined: for instance this may occur if $\text{ACELEN} = 192$ or $\text{ACELEN} > 256$. This is usually not a problem, since the output vectors will be usually configured for 64-bit, 128-bit, or 256-bit blocks, in which case four, two, or one `ace.exec` instructions are required, respectively.

- i** There is no need to provide keyed versions of this algorithm. This can be done by a trusted entity by providing a CC after absorbing some secret content in it first.

2.4.18. Ascon-XOF128

The algorithm is the same as Ascon-Hash256 ([Ascon-Hash256](#)) with the following differences:

1. The IV written to `state[0]` in State *Initial* is `0x0000080100cc0002`.
2. There is no `countdown` field (and therefore also no padding: the last 64 bits of the Serialized Context of Ascon-Hash256 are not present).
3. As much output as desired can be produced in State *Hash_Finalize*, and the algorithm never transitions to State *Success*.

2.4.19. Ascon-CXOF128

Ascon-CXOF128 differs from Ascon-XOF128 in that the message is prepended with the customization string. Except for the initialization of the state, Ascon-CXOF128 is identical to Ascon-XOF128 with customization string and message concatenated. The IV for Ascon-CXOF128 is `0x0000080000cc0004` (in place of `0x0000080000cc0003`), and it is set by the algorithm. The states and their semantics are otherwise identical to Ascon-XOF128, and the management of the customization string is left to the user.

2.5. Elliptic Curve Asymmetric Algorithms

Elliptic Curve Asymmetric Algorithms provide following functionality:

1. Scalar multiplication with known and unknown base point.
2. ECDSA/EdDSA signature generation and verification.

- i** Support for Key Encapsulation Mechanisms (KEMs) and Public-Key Encryption (PKE) schemes is difficult to provide in full generality, given the variety of options for KDFs, hash functions, and input formats. Covering all combinations would be impractical. Amongst all elliptic curve standards, only the Chinese SM2 standards fully specify an integrated encryption scheme cite:[GMT-0003-4-2012]. We therefore leave the implementation of specific KEM and PKE schemes to user code, based on the scalar multiplication feature. The user must be aware that any

derived key value will be present in the environment's memory, and must be used immediately to provision a CR, and then erased.

When an elliptic curve is provisioned into a CR, the default base point (“generator”) according to the standards listed is configured as the `Generator` field in the CR. This field may be overwritten, and when `ace.setst` is issued on the CR with *Initial* destination state, the default base point may be restored.

All values (integers, coordinates, etc.) are in little-endian format.

Parameters

The parameters that are used in this section are:

1. The bit length b of the main fields, such as coordinates, signature R and S fields, scalars;
2. The bit length h of the hash used in the signature scheme;
3. The bit length j of the random number used as nonce in signature generation;
4. The number u of coordinates used to represent a point (u can be 1 or 2); and
5. The number v of elements used to represent a signature (v can be 1 or 2).

The parameters for the defined curves are:

- NIST cite:[NIST-SP-800-186,nist-fips-186-5]:
 - secp256r1 (NIST P-256): $b = h = j = 256$, $u = v = 2$.
 - secp384r1 (NIST P-384): $b = h = j = 384$, $u = v = 2$.
 - secp521r1 (NIST P-521): $b = 576$ (55 msbs set to zero), $h = j = 512$, $u = v = 2$.
- Edwards cite:[RFC7748]:
 - ed25519: $b = h = j = 256$; $u = v = 1$.
 - ed448: $b = h = j = 448$; $u = v = 1$.
- Brainpool cite:[RFC5639,RFC8734]:
 - brainpoolP256r1: $b = h = j = 256$; $u = v = 2$.
 - brainpoolP384r1: $b = h = j = 384$; $u = v = 2$.
 - brainpoolP512r1: $b = h = j = 512$; $u = v = 2$.
- SM2 cite:
[GMT-0003-1-2012,GMT-0003-2-2012,GMT-0003-3-2012,GMT-0003-4-2012]: $b = h = j = 256$; $u = v = 2$.

Depending on the specifications of the algorithms, non-compressed or compressed points may be used.

All curves currently used are defined over fields of odd characteristic. Since the all-ones strings do not represent any valid field value, these are used as coordinate(s) values of a point to represent the point at infinity.

Data Structures

• MDH:

The elliptic curve algorithm use the *StateExtension* field as follows:

StateExtension bits used by the elliptic curve algorithms

Bit	Name	Description
0	HasSecondPt	A second point has been set.
1	HasSignature	A signature has been set.
2	HasHash	A hash has been set.
3	HasRndNum	A random number has been set.

The elliptic curve algorithm use the *AlgorithmUse* field as follows:

AlgorithmUse bits used by the elliptic curve algorithms

Bits	Name	Description
0	OutputType	Determines whether the output is a point (and thus comes from SecondPt) or a signature (and thus comes from Signature).
[19:1]	Progress	May be used by the implementation to track the progress of long running operations in a non-portable way.

For ECC algorithms, the flags in *AlgorithmPolicy* enable signature generation (*AlgorithmPolicy*[0]) and verification (*AlgorithmPolicy*[1]), where all four combinations are admissible.

Data Structures• **Provisioning Input:**

Pos.	Size (Bits)	Field
i	128	MDH

• **Internal State:**

- Metadata Header.
- OutputType: Bool.
- Progress: integer (16 bits, MDH field *AlgorithmUse* is used).
- block_base: integer.
- Generator (X, {Y}): one or two b-bit strings representing the X {and Y} coordinate{s}.
- SecondPt (X, {Y}): one or two b-bit strings representing the X {and Y} coordinate{s}.
- Scalar: b-bit string.
- If signature generation or verification is allowed:
 - PublicKey (X, {Y}): one or two b-bit strings representing the X {and Y} coordinate{s}.
 - Signature (R,S): two b-bit strings representing the R and s components.
 - Hash: a h-bit string representing the hash.
- If signature generation or a public-key encryption mechanism are allowed:
 - RndNum: a h-bit string representing the random number.

- Additional data, for instance precomputed multiples of the Generator.

- **Serialized Context:**

Pos.	Size (Bits)	Field
i	128	MDH
ii	32	Padding
iii	ub	Generator, X coordinate, followed by Y coordinate if $u=2$.
iv	b	Scalar: b-bit string. Can be the Private Key.

The following fields are also optionally included in the `Content1[]` field of the SCC

Pos.	Size (Bits)	Field
v	If <code>HasSecondPt</code> : ub, else: 0	SecondPt, X coordinate, followed by Y coordinate if $u=2$. Can be the Public Key.
vi	If <code>HasSignature</code> : vb, else: 0	Signature, R value, followed by S coordinate if $v=2$.
vii	If <code>HasHash</code> : h, else: 0	Hash: a h-bit string representing the hash.
viii	If <code>HasRndNum</code> : j, else: 0	RndNum. not empty.

`SCC_content_size()` returns the length of the content from Position i to Position viii, and thus the elliptic curve algorithms make use of the *StateExtension* MDH field to determine the length of the SCC.

The `HasRndNum` flag is set and the `RndNum` field is used only during signature generation. `RndNum` is exported only if the signature operation is interrupted.

The size in bits of Fields vii to x is:

$$\text{varlen} \leftarrow 8 + (\text{HasSecondPt} \neq 0) \cdot \text{ub}/8 + (\text{HasSignature} \neq 0) \cdot \text{vb}/8 + (\text{HasHash} \neq 0) \cdot \text{h}/8 + (\text{HasRndNum} \neq 0) \cdot \text{j}/8.$$

If $\text{ImpDataLen} > \text{varlen}$, Position x contains the microarchitectural dependent part, which starts with the Implementation Qualifier `IMPQUAL`.

State Machine

- **States:**

Beside

Initial (1),

Success (21),

Failure (22),

ECC algorithms define:

- *Set_Generator* (2)
- *Set_Scalar* (3)
- *Gen_Rnd_Scalar* (4)
- *Point_Mul* (5)

- *Sign_Generate* (6)
- *Sign_Verify* (7)
- *Set_Hash* (8)
- *Set_SecondPt* (9)
- *Set_Signature* (10)
- *Output* (11)

• **Allowed State Transitions:**

- *Initial* → *Set_Generator*, *Set_Scalar*, *Set_Hash*, *Set_SecondPt*, *Sign_Generate*, *Set_Signature*.
- One can freely transition between *Set_Generator*, *Set_Scalar*, *Set_Hash*, and *Set_SecondPt*.
- From any of *Initial*, *Set_Generator*, *Set_Scalar*, *Set_Hash*, *Set_SecondPt* → one of:
 - *Point_Mul*;
 - *Sign_Generate* only if HasSecondPt, and HasHash;
 - *Sign_Verify* only if HasSecondPt, HasHash, and HasSignature.
- *Point_Mul* → *Output*.
- *Sign_Generate* → *Output* → *Success*.
- *Sign_Verify* → *Success* or *Failure*.
- In State *Set_Generator* we can set a group generator, or a default one.
- In State *Set_Scalar* we can either set a scalar for scalar multiplication or private key.
- For each *Var* ∈ {Generator, Scalar, Hash, SecondPt, Signature}, the Boolean variable Has*Var* is set when all operations required in State *set_Var* have been successfully executed.

• **Behavior:**

- When the CC is created, i.e., provisioned, the *Generator* is set to the default parameter, and *Scalar* is set to zero.
- All the “set” and “output” operations work like the input and output of blocks in the Hash functions, in other words, they use *Progress* (just as the Hash/MAC functions use *block_base*) to track how much information has been read in or written out.
- To transition to State *Set_Generator*, a Form A or B *ace.setst* is used.
 - If no parameter is given, or the value of the GPR used in the Form B *ace.setst* is zero:
 1. The default generator of the curve is set.
 2. A state change operation can be issued, including to states that require a set *Generator*.
 - Otherwise:
 1. *Progress* ← 0.
 2. A value for *Generator* is loaded, using one or more Form B *ace.exec* operations, each one increasing *Progress* by the amount of data provided by *ace.exec*.

3. Once `Progress` has reached the length of the generator point (excess data in the last `ace.exec` is ignored):
 - No further `ace.exec` instructions may be issued.
 - A state change operation can be issued, including to states that require a set `Generator`. This state change also clears `Progress`.
- To transition to any of the States *Set_Scalar*, *Set_Hash*, *Set_SecondPt*, or *Set_Signature* a Form A `ace.setst` is used.
 1. If there is a current value in the corresponding field $X = \text{Scalar, Hash, SecondPt, OR Signature}$, it is erased, and the Boolean `Has_X_` is set to `False`.
 2. `Progress` $\leftarrow$ 0, and a value for Field X is loaded, using one or more Form B `ace.exec` operations (similarly to `Generator`).
 3. Once the values `Progress` has reached the length of field X (excess data in the last `ace.exec` is ignored):
 - The corresponding Boolean `Has_X_` is set to `True`.
 - No further `ace.exec` instructions may be issued.
 - A state change operation can be issued, including to states that require a value for X to be set. This state change also clears `Progress`.
- It is allowed to transition to *Set_Scalar* also using a Form B `ace.setst` with a non-zero value in the GPR.
In this case, a random value is set.
- When the operations in states *Point_Mul* and *Sign_Generate* are completed:
 1. If the output is a point, then `OutputType` $\leftarrow$ `False`, and if the output is a signature, then `OutputType` $\leftarrow$ `True`.
This allows the ACE implementation to determine not only which output must be written out, but also its length.
 - If `OutputType == False`, the output is in `SecondPt`.
 - If `OutputType == True`, the output is in `Signature`.
 2. `Progress` $\leftarrow$ 0.
 3. One or more Form C `ace.exec` instructions may be issued to export sections of the result to memory, each time increasing `Progress` by the length of the output of `ace.exec`.
 4. Data beyond the length of the output is replaced with zeros in the last exported part.
 5. Once `Progress` has reached the length of the output, the state transitions to *Success*.
- If, after any computation, either `Generator` OR `SecondPt` is the point at infinity, then the corresponding field is set to the special value that encodes it.
- Upon returning to State *Initial*:
 - If a Form B `ace.setst` is used, the parameter `xs` will determine whether the fields `Generator`, `Scalar`, and `SecondPt` are reset/erased or kept for further operations. This is determined by Bits 0, 1, 2, and 3 of `xs` as follows:

- If Bit 0, 1, 2, resp., 3 is set, then Generator is reset to the default value, SecondPt, Scalar, resp., SecondPt is erased, and in the second case flag HasSecondPt is unset as well.
- If Bit 4 is set, then Generator is copied onto SecondPt. If Bit 0 is set, then Generator is set to the default value.
- If Bit 5 is set, then SecondPt is copied onto Generator. If Bit 1 is set, then SecondPt is erased afterwards and HasSecondPt is set to False.
- A Form A `ace.setst` does not modify these states.
- Signature and the corresponding Boolean HasSignature are always reset.
-

Progress can be used by operations such as Point Multiplication, Signature Generation or Verification, to track the progress of the operations in case they are interrupted.

In this case, if the CR is exported, the Additional Data Section may contain an implementation-specific section that contain sufficient information to allow resumption, such as certain precomputed multiples of the base point.

The importing implementation, after comparing `acevendorid`, `acearchid` and `aceimpid` with the values in `IMPQUAL`, decides whether it can retain the additional data and trust the random number generator of the originating implementation, in which case it can then resume the computation.

Otherwise, it will discard the additional data at Position `x` and `RndNum` (Position `ix`, if present), and restart the operation.

2.6. Post Quantum Public-Key Cryptography

2.6.1. ML-KEM

The *Module-Lattice-Based Key-Encapsulation Mechanism Standard* was announced by NIST in August 2024 cite:[nist-fips-203].

The standard, defines several procedures, including:

- ML-KEM.KeyGen: Generates an encapsulation key `encapsk` and a corresponding decapsulation key `decapsk`.
- ML-KEM.Encaps: Uses the encapsulation key `encapsk` to generate a shared secret key `sharedkey` and an associated ciphertext.
- ML-KEM.Decaps: Uses the decapsulation key `decapsk` to produce a shared secret key `sharedkey` from a ciphertext.

ML-KEM exists in three different subtypes, called ML-KEM-512, ML-KEM-768, and ML-KEM-1024, each with different cryptographic strength.

These three versions require RBGs (Random Bit Generators) of varying strength, namely, 128, 192, and 256 bits, respectively.

The sizes (in bytes) of keys and ciphertexts of the three subtypes of ML-KEM are summarized in [Sizes \(in bytes\) of keys and ciphertexts of ML-KEM](#).

Sizes (in bytes) of keys and ciphertexts of ML-KEM

Subtype	encapsk	decapsk	ciphertext	sharedkey
ML-KEM-512	800	1632	768	32
ML-KEM-768	1184	2400	1088	32
ML-KEM-1024	1568	3168	1568	32

The ML-KEM algorithms also support the `ace.derive` instruction to derive a shared secret key from a ciphertext and the decapsulation key and use it to create a new cryptographic context. The auxiliary input in a GPR (or GPR pair for RV32) is used to provide the symmetric algorithm that will be created with the derived shared secret key.

Data Structures

- **Provisioning Input:**

Pos.	Size (Bits)	Field
i	128	MDH

- **Internal State:**

- Metadata Header.
- encapsk: 6400 / 9472 / 12544 bits (800 / 1184 / 1568 bytes).
- decapsk: 13056 / 19200 / 25344 bits (1632 / 2400 / 3168 bytes).
- ciphertext: 6144 / 8704 / 12544 bits (768 / 1088 / 1568 bytes).
- sharedkey 256 bits.

ML-KEM algorithms use the *AlgorithmUse* field of the metadata header to indicate how much of the content has been read so far from memory into the internal values of the encapsk, resp., decapsk in the *encapsk_Input* or *decapsk_Input* internal states, or written to memory out of them or the sharedkey in the *encapsk_Output*, *decapsk_Output*, *ciphertext_Output*, or *sharedkey_Output* states. This is similar to what is done in the Hashing and Elliptic Curve algorithms.

For the three subtypes, the size of the internal state, without considering the metadata information, is 3232, 4704, and 6336 bytes.

- **Serialized Context:**

Pos.	Size (Bits), for ML-KEM-512/768/1024	Field
i	128	MDH
ii	6400 / 9472 / 12544	encapsk
iii	13056 / 19200 / 25344	decapsk
iv	6144 / 8704 / 12544	ciphertext
v	256	sharedkey

For the three subtypes, the sizes of the Serialized Context is 3248, 4720, and 6352 bytes.

State Machine

- **States:**

Beside

Initial (1),

Success (21),

Failure (22),

ML-KEM algorithms define:

- *GenerateKeyPair* (2)
- *Encapsulate* (3)
- *Decapsulate* (4)
- *encapsk_Input* (5)
- *decapsk_Input* (6)
- *encapsk_Output* (7)
- *decapsk_Output* (8)
- *ciphertext_Output* (9)
- *sharedkey_Output* (10)

- **Allowed State Transitions:**

- State transitions are allowed between any two valid states.

- **Behavior:**

- In State *Initial*, no `ace.exec` instruction is allowed.
Upon transitioning to *Initial*, the fields `encapsk`, `decapsk`, `ciphertext` and `sharedkey` are cleared.
- All uses of `ace.setst` do not require an auxiliary parameter.
- The ML-KRM algorithms do not carry state information as to whether `encapsk`, `decapsk`, `ciphertext` and `sharedkey` have been assigned values.
If the values are not valid, the encapsulation and decapsulation procedures may return unpredictable results or fail with a transition to Error State `ace_state_invalid`.
- In State *GenerateKeyPair*, a random seed is generated and used to derive the `encapsk` and `decapsk` values using the procedure `ML-KEM.KeyGen` described in the ML-KEM specification.
If the procedure `ML-KEM.KeyGen` fails, the state machine transitions to state *Failure*.
- In particular, `ML-KEM.KeyGen`—as well as `ML-KEM.Encaps` and `ML-KEM.Decaps`—must perform an examination of their inputs for the purpose of determining whether it conforms to certain requirements, as described in the ML-KEM specification.
- In State *Encapsulate*, a shared key is randomly generated as described in the ML-KEM specification and stored in the `sharedkey` state field.
The corresponding ciphertext is generated using the procedure `ML-KEM.Encaps` described in the ML-KEM specification and stored in the `ciphertext` state field.
If the procedure `ML-KEM.Encaps` fails, the state machine transitions to state *Failure*.
- In State *Decapsulate*, the content of the `ciphertext` state field is decrypted using the procedure `ML-KEM.Decaps` described in the ML-KEM specification and the result is stored in the `sharedkey` state field.
If the procedure `ML-KEM.Decaps` fails, the state machine transitions to state *Failure*.

- In States *encapsk_Input*, resp., *decapsk_Input*, one or more Form B `ace.exec` instructions are used to load a value of *encapsk*, resp., *decapsk* from memory. The amount of bits loaded into the state fields so far is stored in the *AlgorithmUse* field.
When the value of the *AlgorithmUse* field reaches the size of the corresponding field, further `ace.exec` instructions are not allowed.
If, as a result of an `ace.exec` instruction, the value of the *AlgorithmUse* field would become greater the size of the corresponding field, an invalid instruction exception is raised.
- In States *encapsk_Output*, *decapsk_Output*, *ciphertext_Output*, resp., *sharedkey_Output* one or more Form C `ace.exec` instructions are used to store values of *encapsk*, *decapsk*, *ciphertext*, resp., *sharedkey* to memory.
The amount of bits saved to memory so far is stored in the *AlgorithmUse* field.
- Upon entering any of the **_Input* or **_Output* states by using `ace.setst`, the *AlgorithmUse* field is cleared.
- At any moment, the `ace.derive` instruction can be used to use the shared key from the state into a new CC in a CR.
The `ace.derive` instruction must be of Form B, with bits [63:0] of the MDH of the new CC in a GPR (RV64) or a GPR pair (RV32).
The MDH must define a suitable supported single-key algorithm, that can accept keys up to 256 bits in length.
If the key used by the destination algorithm is $m < 256$ bits long, the m least significant bits of the shared key are used.
- For certain long-running operations, such as *GenerateKeyPair*, *Encapsulate*, and *Decapsulate*, an implementation may use the *AlgorithmUse* field to store a progress value, so that these operations can be interrupted and resumed later.
In this case the use of a mandatory Additional Data Section is required to store the Implementation Qualifier, as well as any implementation-specific state needed to resume the operation.

2.6.2. ML-DSA

The *Module-Lattice-Based Digital Signature Standard* was announced by NIST in August 2024 cite:[nist-fips-204].

It exists in three subtypes: ML-DSA-44, ML-DSA-65, and ML-DSA-87.

The principal procedures offered by the ML-KEM are:

- ML-DSA.KeyGen: Generates a public-private key pair.
- ML-DSA.Sign: Generates a digital signature for a given message.
- ML-DSA.Verify: Verifies a digital signature for a given message.
- HashML-DSA.Sign: Generates a digital signature for a given pre-hashed message.
- HashML-DSA.Verify: Verifies a digital signature for a given pre-hashed message.

The ACE algorithm exposes the signing, and verification operations in a slightly different format that requires the caller to compute the values tr (hash of the public key) and μ (hash of the formatted message). In other words, the actual functions exposed by ACE are

ML-DSA.Sign_internal and ML-DSA.Verify_internal as defined in the specification, except that tr and μ are provided by the caller and not computed from the message (M) or the pre-hashed message (M').

The main reason is that it would not be possible to support all hash functions in a generic way, especially in ACE units which, for instance, implement hash functions in hardware and expose ML-DSA through software emulation. While this approach may give a malicious user some limited potential for input manipulation, it still does not expose the private key.

Sizes (in bytes) of keys and signatures of ML-DSA

Subtype	Private Key	Public Key	Signature Size
ML-DSA-44	2560	1312	2420
ML-DSA-65	4032	1952	3309
ML-DSA-87	4896	2592	4627

Data Structures

• Provisioning Input:

Pos.	Size (Bits)	Field
i	128	MDH

The *AlgorithmPolicy* field in the MDH has the following meaning:
AlgorithmPolicy[0] is set to 1 if the Algorithm can generate signatures, and
AlgorithmPolicy[1] is set to 1 if the Algorithm can verify signatures.
 An *AlgorithmPolicy* of 0 is not valid, and it will cause the CR to immediately transition to State `ace_state_invalid`.

The *AuxInfo* field specifies the hash function to use in the signature algorithms. This parameter has the same format as the *Algorithm* and *AlgorithmPolicy* Fields and the next two Reserved bits.
 If the value does not correspond to a hash function supported by the ACE implementation *and* by ML-DSA, the state machine transitions to Error State `ace_state_not_impl`.

• Internal State:

- Metadata Header. The field *AlgorithmUse* is used to keep track of input and output of long fields in an architectural and to track long running operations in an implementation-specific way.
 The field *AuxInfo* contains the description of the Hash Function used by the algorithm.
- privkey: 2560/4032/4896 bytes (20480/32256/39168 bits) for ML-DSA-44/65/87, resp.
- pubkey : 1312/1952/2592 bytes (10496/15616/20736 bits) for ML-DSA-44/65/87, resp.

- **signature:** 2420/3309/4627 bytes (19360/26472/37016 bits) for ML-DSA-44/65/87, resp.
- **ctxlen:** 1 byte (8 bits)
- **ctx:** 255 bytes (2040 bits)
- **mu:** 64 bytes (512 bits), contains the hashed formatted message
- **tr:** 64 bytes (512 bits), contains the hash of the public key

The ML-DSA Algorithm in ACE does not hash the message itself before signing, but only performs the subsequent hash operations.

Therefore it is the responsibility of the caller to compute the public-key hash *tr* and compute the 64-byte message representative μ using the same hash function.

The internal state must contain several additional variables, for instance a 256-bit random number, intermediate values of the computation of the NTT, etc.

The definition of all these values is implementation dependent, as is their occupancy. They must be saved in an SCC in the Additional Data Section, allowing resumption of an interrupted operation on the same implementation. In general, key pair generation, signing, and verification operations will restart on a different implementation.

- **Serialized Context:**

Pos.	Size (Bits), for ML-DSA-44/65/87	Field
i	128	MDH
ii	20480 / 32256 / 39168	privkey
iii	10496 / 15616 / 20736	pubkey
iv	19360 / 26472 / 37016	signature
v	8	ctxlen
vi	2040	ctx
vii	512	mu
viii	512	tr

State Machine

- **States:**
 Beside
Initial (1),
Success (21),
Failure (22),
 ML-KEM algorithms define:
 - *GenerateKeyPair* (2)
 - *pubkey_Output* (3)
 - *pubkey_Input* (4)
 - *ctx_Input* (5)
 - *mu_Input* (6)

- *tr_Input* (7)
- *Sign_Generate* (8)
- *Sign_Output* (9)
- *Sign_Verify* (10)
- *Sign_Input* (11)

- **Allowed State Transitions:**

- State transitions are allowed between any two valid states.

- **Behavior:**

- Upon provisioning, all fields *ii* to *viii* are cleared and the state machine transitions to State *Initial*.
- In State *Initial*, no `ace.exec` instruction is allowed.
Upon transitioning to *Initial*, the fields *signature*, *ctxlen*, *ctx*, and *mu* are cleared.
- To transition to states *Initial*, *pubkey_Input*, *decapsk_Input*, *mu_Input*, *tr_Input*, *Sign_Input*, *pubkey_Output*, *Sign_Generate*, *Sign_Verify*, *Sign_Output*, a Form A `ace.setst` instruction must be used.
- To transition to state *ctx_Input*, a Form B `ace.setst` instruction must be used where the GPR contains the parameter *ctxlen*. Only values of *ctxlen* from 0 to 255 are valid. Passing an invalid *ctxlen* value will transition the CR to Error State `ace_state_invalid`.
- The ML-DSA algorithms do not carry state information as to whether *privkey*, *pubkey*, *signature*, *ctxlen*, *ctx*, *mu* and *tr* have been assigned values.
If the values are not valid, the signing and verification procedures may return unpredictable results or fail with a transition to Error State `ace_state_invalid`.
- Transitioning to State *GenerateKeyPair* will generate a new key pair and replace the *encapsk* and *decapsk* values.
This will also clear the Fields *ctxlen*, *ctx*, and *mu*. *tr* will be recomputed.
The `ace.setst` instruction must be of Form A, i.e, without optional parameters.
- In States *pubkey_Input*, *decapsk_Input*, *ctx_Input*, *mu_Input*, *tr_Input*, and *Sign_Input*, one or more Form B `ace.exec` instructions are used to load a value of *pubkey*, *decapsk*, *ctx*, *mu*, *tr*, and *signature`_Input_*, respectively.
The amount of bits loaded into the state fields so far is stored in the *AlgorithmUse* field.
When the value of the *AlgorithmUse* field reaches the size of the corresponding field, further `ace.exec` instructions are not allowed.
If, as a result of an `ace.exec` instruction, the value of the *AlgorithmUse* field would become greater the size of the corresponding field, an invalid instruction exception is raised.
- In States *pubkey_Output* and *Sign_Output*, one or more Form C `ace.exec` instructions are used to store values of *encapsk*, *decapsk*, *ciphertext*, resp., *sharedkey* to memory.
The amount of bits saved to memory so far is stored in the *AlgorithmUse* field.
When the value of the *AlgorithmUse* field reaches the size of the corresponding field, further `ace.exec` instructions are not allowed.
If, as a result of an `ace.exec` instruction, the value of the *AlgorithmUse* field would

become greater the size of the corresponding field, an invalid instruction exception is raised.

- Upon entering any of the **_Input* or **_Output* states by using `ace.setst`, the *AlgorithmUse* field is cleared.
- *AlgorithmPolicy* can be modified using `ace.restrict1`.
The value 0 is not admissible.
An attempt to clear the field using `ace.restrict1` will transition the CR to Error State `ace_state_invalid`.
- For certain long-running operations, such as *GenerateKeyPair*, *Sign_Generate*, and *Sign_Verify*, an implementation may use the *AlgorithmUse* field to store a progress value, so that these operations can be interrupted and resumed later.
In this case the use of a mandatory Additional Data Section is required to store the Implementation Qualifier, as well as any implementation-specific state needed to resume the operation.
- **Anything else?**

3. Book 3: Privileged Architecture

The ACE Privileged Architecture consists of CSRs and of optional custom instructions to implement parts of the unprivileged ISA.

We refer to the ACE Unprivileged Architecture for the definitions of the ACE instruction and the categories of *configuration* and *usage* instructions, including the meaning of “to use a CR”.

3.1. Extensions Overview

Extension	Features	Defined in:	Minimum version	Lifecycle state
<code>Smacestatus</code>	Defines the ACES per-CR status tracking CSRs.	<code>scrstatus</code> for Dirtiness Tracking , <code>vscrstatus</code> for Dirtiness Tracking , and <code>mcrstatus</code> for Dirtiness Tracking	v0.0.0	--
<code>SmaceCSK</code>	Defines the CSRs group to configure the Context Sealing Key.	<code>macecsk</code> Group	v0.0.0	--

Extension	Features	Defined in:	Minimum version	Lifecycle state
Smacephysbootscrt	Defines the CSRs group to configure the OS Secret Locality (cf. Locality Field).	macePhysBootScrt Group	v0.0.0	--
Smacevirtbootscrt	Defines the CSRs group to configure the Boot Secret Locality (cf. Locality Field).	haceVirtBootScrt Group	v0.0.0	--
Smaceclt	Defines the CSRs group to configure the software binding Localities (cf. Locality Field).	maceLocality Group , haceLocality Group , and saceLocality Group	v0.0.0	--

If Zlv or Zlio are implemented: `misa` must implement the `L` bit, `*status` must implement ACES.

3.2. Dependencies on Other Extensions

The ACE Privileged Architecture may depend (this must be decided together with the ARC) on the Indirect CSR Access mechanism defined in `Smcsrind`/`Sscsrind`.

3.3. Exceptions Raised by ACE

With reference to [State and StateExtension Fields](#), we discuss here error conditions that raise exceptions.

When an exception is raised, the CSR `mcause` will reflect the error condition.

[Exception causes and their meanings, in decreasing priority order \(also relative to RISC-V exceptions\)](#) lists `mcause` values defined by ACE.

The error codes are listed in decreasing priority order.

The actual numbers are TBD.

Exception causes and their meanings, in decreasing priority order (also relative to RISC-V exceptions)

Value	Mnemonic/RISC-V type	Examples
	Instruction address misaligned	
	Instruction access fault	
	Load address misaligned	For <code>ace.prov</code> , <code>ace.import</code> , and <code>ace.input</code>

Value	Mnemonic/RISC-V type	Examples
	Load access fault	For <code>ace.prov</code> , <code>ace.import</code> , and <code>ace.input</code>
	Store address misaligned	For <code>ace.export</code> and <code>ace.output</code>
	Store access fault	For <code>ace.export</code> and <code>ace.output</code>
	Instruction page fault	
	Load page fault	
	Store page fault	
TBD	<code>ace_guru_meditation</code>	Fatal and unrecoverable system failure in the ACE unit.
TBD	<code>ace_exc_CR_unconf</code>	Use of an unconfigured, or partially configured CR as a source in any instruction except <code>ace.size</code> .
TBD	<code>ace_exc_CR_other</code>	Use of a CR as a source in any instruction (including <code>ace.size</code>), when the status of the CR is “Other/Lazy”, as per <code>scrstatus</code> for Dirtiness Tracking .
TBD	<code>ace_exc_out_of_mem</code>	Insufficient CRF memory for configuration instruction. This may be set by privileged code in case they do not handle the CR allocation themselves.
TBD	<code>ace_exc_buf_unconf</code>	Use of <code>ACEIOBUF</code> as an input or an output while unconfigured.
TBD	<code>ace_exc_invalid</code>	Invalid parameters in metadata, including expired CR state. In the latter case, we recall that the hardware also sets the <i>State</i> field of the CR’s metadata to <code>ace_state_expired</code> .



Merge `ace_exc_CR_unconf` into `ace_exc_invalid`, so that we save one code?

3.4. CSRs

ACE extends `*status` and `misa`, and provides up to 3 direct and up to 28 indirect CSRs.

ACE Privileged CSRs.

Address	Privilege	Field	Description
0xXXX	SRW	<code>scrstatus</code>	Per-CR dirtiness and laziness tracking
0xXXX	HRW	<code>vscrstatus</code>	Per-CR dirtiness and laziness tracking
0xXXX	MRW	<code>mcrstatus</code>	Per-CR dirtiness and laziness tracking

Address	Privilege	Field	Description
(Indirect)	MRW (RZ)	macecsk{0..7}	ACE Context Sealing Key
(Indirect)	MRW (RZ)	macePhysBootScrt{0..3}	Locality: Physical (host) Boot Session Secret
(Indirect)	HRW (RZ)	haceVirtBootScrt{0..3}	Locality: Virtual (guest) Boot Session Secret
(Indirect)	MRW (RZ)	maceLocality{0..3}	Locality: Mutually isolate processes below M-mode
(Indirect)	HRW (RZ)	haceLocality{0..3}	Locality: Mutually isolate processes below HS-mode
(Indirect)	SRW (RZ)	saceLocality{0..3}	Locality: Mutually isolate U-mode processes

3.4.1. ACE field in `misa`

Bit L (11) of `misa` indicates that the ACE extension is present. If ACE is implemented, this bit shall not be read-only zero.

3.4.2. ACE Context Status in `mstatus` and `sstatus`

The ACES field is added to `mstatus`[26:25] and shadowed in `sstatus`[26:25] (see `mcrstatus` for [Dirtiness Tracking](#)).

ACES is defined analogously to the floating-point context status field FS and the vector context status field VS.

It tracks the dirtiness of the entire ACE state, including CRs, ACEIOBUF, and all ACE-specific CSRs.

If ACE is implemented, ACES shall not be read-only zero.

Encodings are given in [Encoding of the ACES field](#).

Encoding of the ACES field

Value	Name	Meaning
0	Off	ACE disabled. The ACE state is inaccessible. Executing any ACE instruction raises an illegal instruction exception.
1	Initial	ACE enabled.
2	Clean	ACE enabled. At least one CR or ACEIOBUF is configured; state matches the last context swap.
3	Dirty	ACE enabled. At least one CR, ACEIOBUF, or ACE-specific CSR may have changed since the last context swap. Unconfigured CRs that were previously configured also cause ACES to become Dirty, since context-switching code must erase any stale saved state.

ACES state transitions follow the same rules as FS, VS, and XS (see cite:[RISCV-ISA-Priv]).

i Precise management of `mstatus.ACES` is an optimization that software may use to reduce context-switch overhead.

When `mstatus.ACES` is Initial or Clean, any instruction that modifies ACE state sets `mstatus.ACES` to Dirty. Implementations may also set `mstatus.ACES` to Dirty from Initial or Clean at any time, even in the absence of a state change.

When `mstatus.ACES` (resp. `sstatus.ACES`) is Dirty, `mstatus.SD` (resp. `sstatus.SD`) is 1; otherwise, `mstatus.SD` is set per existing specifications.

Writes to ACES do not modify any other ACE state. Setting ACES to Off, Initial, or Clean does not clear CRs or ACEIOBUF; any state reset must be explicit and must precede such a write.

In implementations with a writable `misa.L`, the `mstatus.ACES` field may be present even when `misa.L` is clear, analogously to `mstatus.FS` and `mstatus.VS`.

When `TSR` is set to 1, all ACE memory instructions cause an illegal-instruction exception in S-mode.

i ACE memory instructions are allowed to trap so that implementations have more flexibility when loading/storing large ACE contexts. Simpler implementations may defer the load/store process to software using scalar or vector loads/stores followed by an `ace.exec` move, whereas more performant implementations may have dedicated hardware to speedup the instructions.

3.4.3. ACE Context Status in `vsstatus`

When the H extension is present, an ACES field is added to `vsstatus[26:25]`, defined analogously to `mstatus.FS` and `mstatus.VS`, and tracking the dirtiness of CRs, ACEIOBUF, and all ACE-specific CSRs.

When `V=1`:

- Both `vsstatus.ACES` and `mstatus.ACES` are in effect.
- Any ACE instruction raises an illegal instruction exception if either field is Off.
- If neither field is Off, any instruction that modifies ACE state sets both `mstatus.ACES` and `vsstatus.ACES` to Dirty.
- Implementations may set ACES to Dirty from Initial or Clean at any time, even without a state change.
- When `vsstatus.ACES` is Dirty, `vsstatus.SD` is 1; else, `vsstatus.SD` is set per existing specifications.

In implementations with a writable `misa.L`, `vsstatus.ACES` may be present even when `misa.L` is clear.

When `VTSR` is set to 1, all ACE memory instructions cause a virtual-instruction exception in VS-mode.

3.4.4. `scrstatus` for Dirtiness Tracking

The 64-bit register `scrstatus` tracks per-CR status, including dirtiness.

$\text{scrstatus}(i) = \text{scrstatus}[2 \cdot i + 1 : 2 \cdot i]$ is a two-bit field that encodes four possible statuses for $\kappa(i)$, as described in the following table. Note that there are conditions for the values of the *State* metadata field in a CR and the corresponding `*crstatus` field.

Encoding of the `*crstatus` bits

Value	Name	Meaning
0	<i>Off</i>	The CR is unconfigured. If the metadata record's <i>State</i> field of a CR is <i>Off</i> , then this <code>scrstatus</code> field shall be <i>Off</i> or <i>Other</i> . Attempts to use it will raise exception <code>ace_exc_CR_unconf</code> . The CR can be configured.
1	<i>Other (Lazy)</i>	The CR is may be configured or unconfigured. The metadata record's <i>State</i> field of a CR can assume any admissible value. Attempts to export or use the CR will raise exception <code>ace_exc_CR_other</code> . This <code>scrstatus</code> can be used for lazy context switching at per-CR granularity. It is the responsibility of the exception handler to ensure that the CR is configured before returning to caller.
2	<i>Clean</i>	The CR is configured and its state matches the last context swap.
3	<i>Dirty</i>	The CR is configured and its state may have been modified since the last context swap.

On RV32, 32-bit CSRs `scrstatush` shadows bits [63:32] of `scrstatus`, allowing these bits to be accessed.

When the privilege mode is S, VS, or U:

- Any instruction that modifies $\kappa(i)$ sets `scrstatus(i)` to Dirty.
- Implementations may set `scrstatus(i)` to Dirty from Clean at any time, even without a state change.

Status Off is used only when a CR is not configured.

i `scrstatus` is independent of the S extension. In an M-U system, M-mode may use it to optimize U-mode context switches.

i `sstatus.ACES` is also in effect.

3.4.5. `vscrstatus` for Dirtiness Tracking

The 64-bit register `scrstatus` tracks per-CR status, including dirtiness, for virtualized contexts.

`vscrstatus` exists only when the H extension is implemented.

`vscrstatus(i)` = `scrstatus[2·i+1:2·i]` is a two-bit field that encodes four statuses for $\kappa(i)$, as in [Encoding of the `\*crstatus` bits](#).

On RV32, 32-bit CSRs `scrstatush` shadows bits [63:32] of `scrstatus`, allowing these bits to be accessed.

When the privilege mode is VS or VU:

- `vscrstatus` shadows `scrstatus`; accesses that would target `scrstatus` instead access `vscrstatus`.
- Any instruction that modifies $\kappa(i)$ sets `vscrstatus(i)` to Dirty.
- Implementations may set `vscrstatus(i)` to Dirty from Clean at any time, even without a state change.

i `sstatus.ACES`, `vsstatus.ACES`, and `scrstatus` are also in effect when `V=1`.

3.4.6. `mcrstatus` for Dirtiness Tracking



A `mcrstatus` register may be defined, *mutatis mutandis*, in the same manner as `scrstatus` and `vscrstatus`, to allow M-mode to optimize context switches across Supervisor domains. In such a case, the `ACES` field in `sstatus` shall be unshadowed and be independent instead.

3.4.7. Use of `*crstatus`'s “Other” Value for Lazy Context Switching and CR Loading

This section is non-normative.

VM migration can become an issue when the destination machine has a smaller CRF than the source machine. In such cases, migration may fail, or guest software may be forced to terminate processes. A lazy-loading mechanism for CRs can mitigate this issue. In addition to reducing context-switch overhead, it enables CRF oversubscription and supports migration to systems with smaller CRFs.

When a context-switch handler chooses not to restore a configured register $\kappa(i)$ during context restoration, it sets `X_crstatus(__i_)` to Lazy instead. While a CR is marked Lazy,

any operation that requires its current value—including export and cryptographic use—raises exception `ace_exc_CR_other`.

1. Operations that overwrite the CR, such as clear, provisioning, or import, do not cause a trap.

Lazy context switching defers restoration of a CR from its saved SCC.

The authoritative value remains stored in the SCC, while the CR itself remains unconfigured.

When software subsequently accesses the CR, the trap handler restores its value by importing the SCC.

If there is no sufficient CRF available when the SCC must be imported, the trap handler selects another configured CR, exports its state to an SCC, marks it Lazy and frees it, and then restores the requested CR. The policy used to select the CR to be evicted is implementation-defined or software-defined.

As execution may be interrupted by any higher-privilege mode, the architecture must be able to determine which privilege mode currently holds the authoritative SCC for a lazily restored CR:

1. Let B be a privilege mode and A a higher privileged mode.
Suppose an exception handler executing in A -Mode has saved CR $\kappa(i)$ on behalf of software running in B -mode.
At context restore, the exception handler may choose not to restore the CR, but to set $Acrstatus(i)$ to *Lazy* instead.
Then, it unconfigures $\kappa(i)$, causing the hardware to set $Xcrstatus(i)$ to Dirty for all privilege modes X above A .
2. Consequently, the authoritative SCC for $\kappa(i)$ resides in the highest privilege mode C such that $Ccrstatus(i) = \text{Lazy}$.
3. If the software in B -mode later accesses $\kappa(i)$, the lazy-reload trap handlers search privilege modes from M-mode downward. The first mode X for which $Xcrstatus(i) = \text{Lazy}$ holds the authoritative SCC. That mode thus restores $\kappa(i)$ from its saved SCC.

Besides being a context-switching optimization, this feature enables support for VM migration when the destination platform lacks sufficient CRF capacity to restore all SCCs saved by the originating hypervisor.

3.4.8. Propagation of `*crstatus` Status Values

The per-CR status in `vscstatus` (resp. `scstatus`) propagates from less-privileged to more-privileged modes, analogously to the FS, VS, and ACES fields in `*status`.

3.4.9. `macecsk` Group

The optional `macecsk` group consists of $XLEN$ -bit M-mode WARZ CSRs: eight in RV32, four in RV64.

A write to the i -th CSR sets bits $[(i+1) \cdot XLEN - 1 : i \cdot XLEN]$ of the CSK.

Access outside M-mode raises an illegal instruction exception.

- i** M-mode firmware must be aware of whether the CSK is configurable, ephemeral, or fixed.
Persistence mechanisms for configurable CSKs are implementation-specific. Transferring a CSK to another endpoint—such as the M-mode firmware of another device, a local TPM (for instance to preserve a CSK upon suspension or host hibernation), or a cloud-based HSM—requires an authenticated public-key key-wrapping mechanism.
Alternative reconfiguration mechanisms (e.g., Caliptra [cite:\[caliptra\]](#)) are permitted.

3.4.10. macePhysBootScri Group

The macePhysBootScri group consists of M-mode WARZ XLEN-bit CSRs, accessed via Indirect CSR Access: four in RV32, two in RV64.

A write to the i -th CSR sets bits $[(i+1)XLEN-1 : i \cdot XLEN]$ of the Locality Secret PhysBootScri ([Architected Localities. Bits refer to the *Locality* field in the MDH.](#)).

Access outside M-mode or Debug mode raises an illegal instruction exception.

3.4.11. haceVirtBootScri Group

The haceVirtBootScri group consists of HS- and VS-mode WARZ XLEN-bit CSRs, accessed via Indirect CSR Access: four in RV32, two in RV64.

A write to the i -th CSR sets bits $[(i+1)XLEN-1 : i \cdot XLEN]$ of the Locality Secret VirtBootScri ([Architected Localities. Bits refer to the *Locality* field in the MDH.](#)).

Access below VS-mode raises an illegal instruction exception.

3.4.12. maceLocality Group

The maceLocality group consists of M-mode WARZ XLEN-bit CSRs, accessed via Indirect CSR Access: four in RV32, two in RV64.

A write to the i -th CSR sets bits $[(i+1)XLEN-1 : i \cdot XLEN]$ of the Locality Secret MLocality ([Architected Localities. Bits refer to the *Locality* field in the MDH.](#)).

Access outside M-mode and Debug mode raises an illegal instruction exception.

3.4.13. haceLocality Group

The haceLocality group consists of HS-mode WARZ XLEN-bit CSRs, accessed via Indirect CSR Access: four in RV32, two in RV64.

A write to the i -th CSR sets bits $[(i+1)XLEN-1 : i \cdot XLEN]$ of the Locality Secret HLocality ([Architected Localities. Bits refer to the *Locality* field in the MDH.](#)).

This group shall be implemented whenever the H extension is implemented. Access below HS-mode raises an illegal instruction exception.

3.4.14. saceLocality Group

The saceLocality group consists of (V)S-mode WARZ XLEN-bit CSRs, accessed via Indirect CSR Access: four in RV32, two in RV64.

A write to the i -th CSR sets bits $[(i+1)XLEN-1 : i \cdot XLEN]$ of the Locality Secret SLocality ([Architected Localities. Bits refer to the *Locality* field in the MDH.](#)).

The group is accessible in S/HS-mode, VS-mode, M-mode, and Debug mode; access from any other mode raises an illegal instruction exception.

4. Book 4: Pseudocode Examples

4.1. Electronic Codebook Modes (Direct Encryption/Decryption)

The following pseudocode illustrates how to use an ECB CC to encrypt a data using a block cipher.

In this example, as with most other examples, we assume that the block cipher has a width of 128 bits (IOLEN is also 128).

The upper case IMPORT instruction is a placeholder for an import code snippet, library or API call.

Simple Example: Encryption with no Authentication

```
- Call OS API to get the necessary SCC. Assume X5 points to the SCC.
- Allocate memory for ciphertext buffer.

vsetvli zero, 4, e32, m1, ta, ma           // Use 128-bit vectors
IMPORT K0, (X5)                             // Import the SCC into CR K0
ace.getst X1, K0
if (X1 >= 24) then { handle error }

ace.setst K0, #ace_state_encrypt             // Configure CR for encryption
ace.getst X1, K0
if (X1 >= 24) then { handle error }

// assume plaintext in whole blocks
foreach(i from 0 to len_in_bytes(plaintext)/16-1) {
    V0 ← plaintext[16*i .. 16*i+15]
    ace.exec V0, K0, V0                      // Encrypt one block of plaintext
    ciphertext[16*i .. 16*i+15] ← V0
}
```

4.2. CTR/LFSR/XCTR/XLFSR Keystream Generation

Encrypt with Keystream Generation

```
// Acquire payload, initialize
- Info: plaintext = *PT
- Process gets a Keystream Generation CC from an API. X5 = pointer to the CC.
- Allocate memory for ciphertext buffer CT (if not same as PT).

vsetvli zero, 4, e32, m1, ta, ma          // Use 128-bit vectors
IMPORT K0, (X5)

// The next operations set the nonce
V1 ← nonce
ace.setst K0, #ace_state_operate, V1

// Compute keystream, use it to encrypt plaintext
foreach(i from 0 to len_in_bytes(PT)/16-1) {
    V0 ← PT[16*i .. 16*i+15]
    ace.exec V1, K0
    V1 ← V1 xor V0
    CT[16*i .. 16*i+15] ← V1
}
```

4.3. XEX/XTS modes (one or two keys)

We provide only encryption pseudocode, decryption is similar (also this example assumes $b = \text{IOLEN} = 128$).

Pseudocode for encryption in any IV-based non-authenticated mode without associated data

```
// Acquire payload and SCC, initialize
- Info: plaintext = *PT
- Call an API to get the corresponding SCC. X5 = pointer to it.
- Allocate memory for ciphertext buffer CT (if not same as PT).

vsetvli zero, 4, e32, m1, ta, ma          // Use 128-bit vectors
IMPORT K0, (X5)

V0 ← IV
ace.setst K0, #ace_state_encrypt, V1      // Configure CR for encryption
foreach(i from 0 to ceil(len_in_bytes(PT)/16)-1) {
    V0 ← PT[16*i .. 16*i+15]
    ace.exec V0, K0, V0                    // Encrypt one block of plaintext
    CT[16*i .. 16*i+15] ← V0
}
```

To implement the XTS mode, the ciphertext stealing part is easy to realize using the XEX algorithm together with vector masking and shifting operations.

4.4. GCM Mode

In [GCM Encryption](#) we show pseudocode for GCM encryption, again with $b = \text{IOLEN} = 128$, and $c = 32$).

The code assumes that both AD and plaintext (or the ciphertext for the decryption) are zero-padded to a multiple of b bits, in this case 128 bits.

GCM Encryption

```
// Acquire payload, initialize
- Info: authenticated data = *AD, plaintext = *PT
- Process gets a GCM encryption CC from an API. X5 = pointer to the CC.
- Allocate memory for ciphertext buffer CT (if not same as PT).

vsetvli zero, 4, e32, m1, ta, ma          // Use 128-bit vectors
IMPORT K0, (X5)                            // Import, and compute the derived Auth Key

// The next operations set the nonce
ace.setst K0, #ace_state_set_aux_value
V0 ← nonce
ace.exec K0, V0

// Compute AD contribution to Auth Data
ace.setst K0, #ace_state_hash_absorb
foreach(i from 0 to len_in_bytes(AD)/16-1) {
    V0 ← AD[16*i .. 16*i+15]
    ace.exec K0, V0                        // Absorb it into hash
}

// Encrypt plaintext and continue tag computation
ace.setst K0, #ace_state_encrypt
foreach(i from 0 to len_in_bytes(PT)/16-1) {
    V0 ← PT[16*i .. 16*i+15]
    ace.exec V1, K0, V0
    CT[16*i .. 16*i+15] ← V1
}

// Finally, we absorb the lengths into tag, finalize it, and output it
V1 ← bin(len_in_bits(AD), 64) @ bin(len_in_bits(PT), 64)
ace.setst K0, #ace_state_enc_tag_finalize, V1
ace.exec V2, K0                          // V2 contains the finalized hash
```

GCM decryption differs from encryption as follows:

the roles of PT and CT are reversed,

states *encrypt* and *enc_tag_finalize* are replaced by states *decrypt* and *dec_tag_finalize*,

and the last lines are as follows:

GCM Decryption

```
// Finally, we absorb the lengths into tag, finalize it, and verify it
ace.setst K0, #ace_state_dec_tag_finalize
V1 ← bin(len_in_bits(AD), 64) @ bin(len_in_bits(CT), 64)
ace.exec V2, K0, V1                      // V2 contains the finalized hash

V1 ← tag_from_payload
```

```

ace.setst K0, #ace_state_hash_verify, V1
ace.getst X1, K0
if (X1 != ace_state_success) then { handle error } // Do not use the plaintext if
error

```

Implementing GCM encryption using an ECB CC

```

// Acquire payload, initialize
- Info: authenticated data = *AD, plaintext = *PT
- Process gets an ECB encryption CC from an API. Assume X5 is the pointer to the
  CC.
- Allocate memory for ciphertext buffer CT (if not same as PT).

vsetvli zero, 4, e32, m1, ta, ma
IMPORT K0, (X5)
V5[127:32] ← IV
V5[31:0] ← zeros(31) @ 1

V3 ← zeros(b) // "Auth Data 1"
ace.exec V4, K0, V3 // compute H
foreach(i from 0 to ceil(len_in_bytes(AD)/16)-1) {
  V1 ← AD[16*i .. 16*i+15]
  V3 ← V3 xor V1
  V3 ← Galoismul(V4, V3)
}

counter ← 1
foreach(i from 0 to ceil(len_in_bytes(PT)/16)-1) {
  V5[32:0] ← counter
  V1 ← PT[16*i .. 16*i+15]
  ace.exec V2, K0, V5 // create mask by encrypting the counter
  V1 ← V1 xor V2 // CTR encrypt
  V3 ← V3 xor V1
  V3 ← Galoismul(V4, V3)
  CT[16*i .. 16*i+15] ← V1
  counter ← counter + 1
}

V3 ← V3 xor (len_in_bits(AD) @ len_in_bits(PT))
V3 ← Galoismul(V4, V3)
V5[31:0] ← zeros(31) @ 1 // Prepare IV @ 1 for tag encryption
ace.exec V2, K0, V5
tag ← V3 xor V2

```

4.5. GCM-SIV Mode

We show in [GCM-SIV Encryption](#) how to perform AEAD encryption with a GCM-SIV CC (with $b = \text{IOLEN} = 128$).

[GCM-SIV Decryption](#) is the corresponding pseudocode for decryption.

GCM-SIV Encryption

```

// Acquire payload, initialize
- Info: authenticated data = *AD

```

- Info: plaintext = *PT
- Call an API to get the corresponding key from a key management applet or HW block.
- The API returns the SCC. Assume X5 is the pointer to it.
- Allocate memory for ciphertext buffer CT (if not same as PT).

```

vsetvli zero, 4, e32, m1, ta, ma          // Use 128-bit vectors
IMPORT K0, (X5)

// Set the nonce (and the derived Authentication Key).
ace.setst K0, #ace_state_set_aux_value
V0 ← nonce
ace.exec K0, V0

foreach(i from 0 to len_in_bytes(AD)/16-1) {
    V0 ← AD[16*i .. 16*i+15]                // Load a block of associated data
    ace.exec K0, V0                          // Absorb it into hash
}

foreach(i from 0 to len_in_bytes(PT)/16-1) {
    V0 ← PT[16*i .. 16*i+15]                // Load a block of plaintext
    ace.exec K0, V0                          // Absorb it into hash
}

V0 ← bin(len_in_bits(AD), 64) @ bin(len_in_bits(PT), 64)
ace.exec K0, V0                             // Absorb lengths into hash

// Finalize the SIV computation, set and return it
ace.setst K0, #ace_state_enc_tag_finalize
ace.exec V2, K0
tag ← V2

// Encrypt the plaintext
ace.setst K0, #ace_state_encrypt
foreach(i from 0 to len_in_bytes(PT)/16-1) {
    V0 ← PT[16*i .. 16*i+15]                // Load a block of plaintext
    ace.exec V1, K0, V0                     // Encrypt the block
    CT[16*i .. 16*i+15] ← V1                // Store a block of ciphertext
}

```

GCM-SIV Decryption

```

// Acquire payload, initialize
- Info: authenticated data = *AD
- Info: ciphertext = *CT
- Call an API to get the corresponding key from a key management entity.
- The API returns the SCC. Assume X5 is the pointer to it.
- Allocate memory for plaintext buffer PT (if not same as CT).

vsetvli zero, 4, e32, m1, ta, ma          // Use 128-bit vectors
IMPORT K0, (X5)

// The next operations set the nonce and SIV.
V0 ← nonce
ace.setst K0, #ace_state_set_aux_value, V0

```

```

// Compute AD contribution to Auth Data
V0 ← SIV
ace.setst K0, #ace_state_hash_absorb, V0
foreach(i from 0 to len_in_bytes(AD)/16-1) {
    V0 ← AD[16*i .. 16*i+15]
    ace.exec K0, V0 // Absorb it into hash
}

// Decrypt ciphertext and continue computation of tag
foreach(i from 0 to len_in_bytes(CT)/16-1) {
    V0 ← CT[16*i .. 16*i+15]
    ace.exec V1, K0, V0
    PT[16*i .. 16*i+15] ← V1
}

// Finally, we absorb the lengths of AD and plaintext into tag AND verify
ace.setst K0, #ace_state_enc_tag_finalize
V0 ← bin(len_in_bits(AD), 64) @ bin(len_in_bits(PT), 64)
ace.exec K0, V0
ace.getst X1, K0
if (X1 != ace_state_success) then { handle error } // Do not use the plaintext if
error

```

4.6. OCB Mode

In this section we show pseudocode for OCB encryption and decryption ($b = \text{IOLEN} = 128$).

OCB Encryption

```

// Acquire payload, initialize
- Info:
  - authenticated data = *AD with bit length AD_len
  - plaintext = *PT with bit length PT_len
  - nonce N with bit length N_len
  - tag length tag_len

- Process gets an OCB encryption CC from an API. X5 = pointer to the CC.
- Allocate memory for ciphertext buffer CT (if not same as PT).

vsetvli zero,4,e32,m1,ta,ma // Use 128-bit vectors
IMPORT K0, (X5)

// The next operations set the nonce
X1 ← N_len
ace.setst K0, #ace_state_set_aux_value, X1
V0 ← N
ace.exec K0, V0

// The next state change also sets the tag length
X1 ← tag_len
ace.setst K0, #ace_state_hash_absorb, X1

AD_fullblocks ← floor(AD_len/128)
PT_fullblocks ← floor(PT_len/128)

```



```

// Compute AD contribution to tag
foreach(i from 0 to AD_fullblocks-1) {
    V0 ← AD[16*i .. 16*i+15]
    ace.exec K0, V0 // Absorb it into hash
}
X1 ← AD_len mod 128 // Can be zero
ace.setst K0, #ace_state_hash_finalize, X1
if (X1 mod 128 != 0) then {
    V0 ← zeros(128)
    V0 ← AD[AD_fullblocks*16 .. AD_fullblocks*16+ceil(X1/8)]
    ace.exec K0, V0 // Absorb it into hash and encrypt
}

// Encrypt plaintext and continue computation of tag
ace.setst K0, #ace_state_encrypt
foreach(i from 0 to PT_fullblocks-1) {
    V0 ← PT[16*i .. 16*i+15]
    ace.exec V1, K0, V0
    CT[16*i .. 16*i+15] ← V1
}
X1 ← PT_len mod 128 // Can be zero
ace.setst K0, #ace_state_last_block, X1
if (X1 mod 128 != 0) then {
    V0 ← zeros(128)
    V0 ← PT[PT_fullblocks*16 .. PT_fullblocks*16+ceil(X1/8)]
    ace.exec V1, K0, V0 // Absorb it into hash and encrypt
    CT[PT_fullblocks*16 .. PT_fullblocks*16+ceil(X1/8)] ← V1
} else
    ace.exec K0

ace.exec V1, K0 // output tag

```

OCB decryption differs from encryption as follows:

the roles of PT and CT are reversed,

states *encrypt* and *enc_tag_finalize* are replaced by states *decrypt* and *dec_tag_finalize*,

and the last lines are as follows:

OCB Decryption

```

V1 ← tag
ace.setst K0, #ace_hash_verify, V1
ace.getst X1, K0
if (X1 != ace_state_success) then { handle error } // do not use the plaintext if
error

```

4.7. CMAC Functions

CMAC Computation

- Info: CMAC input = *M with bit length M_len
- Process gets a CMAC CC from an API. Assume X5 is the pointer to the CC.

```

vsetvli zero, 4, e32, m1, ta, ma // Use 128-bit vectors
IMPORT K0, (X5)

```

```

ace.setst K0, #ace_hash_absorb
blocks = floor(M_len/128)
remainder = M_len % 128
if remainder == 0 then
    blocks ← blocks - 1
    remainder ← 128

foreach(i from 0 to blocks-1) {
    V0 ← M[16*i .. 16*i+15]
    ace.exec K0, V0
}

X1 ← remainder
ace.setst K0, #ace_hash_last_block, X1
V0 ← M[16*blocks .. 16*blocks+15]
ace.exec K0, V0

ace.setst K0, #ace_hash_hash_finalize
ace.exec V0, K0 // this is the value of the CMAC

```

Index

Bibliography

bibliography::[]